

CLARITI

Documentação Técnica do Notebook Analítico

Building Data-Driven Applications for Data Science — Sprint 3

Enterprise Challenge FIAP + Oracle

Integrantes

NOMES:	RM:
Filipe Santos de Oliveira	RM572828
Giovanni Pascon Corrêa	RM571546
Nicolas Fois Lima	RM571619
Vitor Matias do Nascimento	RM572761
Yasmin Yumi Tsunokawa (Representante)	RM569408

Nome do Grupo: Seraph

Nome do Projeto: Clariti

Turma: 1TSCPF

Nome do Professor: Patrícia Maura Angelini

Data de Entrega: 29/08

Sumário

1. Objetivo e escopo

Este documento explica o código do notebook analítico do projeto CLARITI: o que cada bloco faz, quais comandos foram usados e por que aqueles comandos e não outros. Ele acompanha o arquivo `clariti_aed_icsap_uti_sp.py`, que contém o mesmo código em formato de script executável.

O objeto de análise são as internações com UTI ocorridas no estado de São Paulo entre 2022 e 2024, classificadas quanto a serem ou não ICSAP (Internações por Condições Sensíveis à Atenção Primária). ICSAP é a categoria de internação que, em tese, poderia ter sido evitada por atendimento primário adequado, e por isso funciona como indicador indireto da qualidade da atenção básica de um município.

O escopo deste documento é o código e a sua justificativa técnica. A interpretação estatística dos resultados e as conclusões de negócio estão no próprio notebook, entregue à disciplina de Statistical Methods for Data Science & Machine Learning.

2. Visão geral do pipeline

O fluxo tem cinco arquivos de entrada e três níveis de análise, e a decisão mais importante do projeto é justamente qual nível responde qual pergunta.

Entrada: `dataset_unificado` (município x diagnóstico x ano x mês), `painel_municipio_ano` (município x ano, com SIOPS e CNES), `perfil_paciente` (uma linha por internação), `regiao_municipios` (regionalização IBGE) e `rotas_polos` (distância e tempo de viagem até o polo regional mais próximo).

Nível 1, município x ano: AED completa (etapas 1 a 8) e teste de hipótese (etapas 9 e 10). É onde a maior parte das perguntas do projeto vive, porque o gestor decide por município.

Nível 2, mês x estado: análise de sazonalidade (etapa 6). Precisa de um grão próprio porque métrica calculada por município não pode ser somada para virar métrica estadual: a soma de razões não é a razão das somas.

Nível 3, paciente: modelo de reinternação e análise de viés de seleção (etapa 11). É o único nível em que a unidade é a pessoa, e é também onde aparece o problema de a base só conter quem chegou ao hospital.

3. Ambiente e dependências

O notebook foi desenvolvido e executado em JupyterHub com Python 3.12. As bibliotecas usadas e suas versões, conforme registrado na saída de execução da célula 56, são: `scikit-learn` 1.9.0, `statsmodels` 0.14.6, `seaborn` 0.13.2, `xgboost` 3.4.1, `shap` 0.52.0, `pandas` 2.3.3, `numpy` 2.5.2, `scipy` 1.18.0 e `matplotlib` 3.11.1.

Todos os caminhos de arquivo passam por uma única variável de configuração, `DATA_DIR`, definida na primeira célula do notebook e no topo do script. Nenhuma outra célula contém caminho fixo, o que permite mover o projeto de máquina alterando uma linha.

4. Do notebook ao script: o que muda e por quê

O código do notebook e o do script são o mesmo código, com quatro diferenças de estrutura. Nenhuma delas altera resultado: todos os números do script conferem com os do notebook. As diferenças existem porque notebook e script resolvem problemas diferentes.

1. Imports consolidados no topo. No notebook, cada bloco importa o que usa, porque uma célula precisa poder rodar sozinha depois de um restart de kernel. Em script, a execução é sempre de cima para baixo, então import repetido é ruído. Os 21 imports espalhados pelas células viraram um bloco único.
2. Carga de dados única. As células 34, 38, 52, 53, 54, 56, 57 e 58 releem os mesmos arquivos, pelo mesmo motivo de isolamento de célula. No script, os cinco arquivos são lidos uma vez só. Isso corta 11 leituras redundantes de disco, sendo que perfil_paciente sozinho tem 10,8 MB.
3. Instalação de dependências fora do código. O notebook tem chamadas `!pip install` dentro de célula de execução. Instalar dependência é responsabilidade de ambiente, não de processamento, então no script isso virou um bloco de requisitos no cabeçalho.
4. Figuras salvas em arquivo. `plt.show()` depende de sessão interativa e não funciona em servidor sem display. O script usa o backend Agg e grava os quatro gráficos em figuras/, o que também os transforma em artefato verificável da execução.

Uma quinta diferença é sintática: no notebook, a última expressão de uma célula é exibida automaticamente. Em script isso não acontece, então as 29 expressões finais soltas foram envolvidas em `print()`.

5. Explicação do código, bloco a bloco

Cada bloco abaixo corresponde a uma célula do notebook, identificada pelo número de execução entre colchetes. A evidência de execução vem logo depois da explicação.

Etapas 1. Perfil geral do dataset

[3] Perfil temporal e granularidade da base

Confere o período coberto, se todos os meses aparecem em todos os anos, e quantas chaves distintas existem. É a primeira coisa a fazer com qualquer base nova: descobrir qual é a unidade de uma linha antes de contar qualquer coisa.

Comandos e por que estes:

`df["ano"].min() / .max()` — Delimita a janela real do dado em vez de confiar no nome do arquivo.

`sorted(int(m) for m in df["mes"].unique())` — `unique()` não vem ordenado; o `sorted` expõe mês faltante de imediato.

`nunique()` — Conta valores distintos sem materializar a lista, o que importa em 463 mil linhas.

```
[3]: print("Período:", df["ano"].min(), "a", df["ano"].max())
print("Meses por ano:", sorted(int(m) for m in df["mes"].unique()))
print()
print("Combinações (município, diag_princ, ano, mês) = linhas do dataset:", f"{len(df):,}")
print("Códigos de diagnóstico distintos (diag_princ):", df["diag_princ"].nunique())
print("Códigos de município distintos (município_6):", df["município_6"].nunique())

Período: 2022 a 2024
Meses por ano: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

Combinações (município, diag_princ, ano, mês) = linhas do dataset: 463,299
Códigos de diagnóstico distintos (diag_princ): 6242
Códigos de município distintos (município_6): 2175
```

Evidência da célula [3] (código e saída)

[4] Volume real de interações

Soma a coluna de quantidade em vez de contar linhas. A base é agregada por município, diagnóstico, ano e mês, então uma linha pode representar dezenas de interações.

Comandos e por que estes:

`df["qtd_internacoes_uti"].sum()` — Contar com `len(df)` daria 463.299 e o número correto é 813.422. É o erro mais comum em base agregada.

`groupby("ano").sum()` — Quebra por ano para checar se o crescimento é plausível antes de seguir.

```
[4]: total_internacoes = df["qtd_internacoes_uti"].sum()
print(f"Total de interações com UTI (soma de qtd_internacoes_uti, não linha): {total_internacoes:,}")
print()
print("Por ano:")
print(df.groupby("ano")["qtd_internacoes_uti"].sum().apply(lambda x: f"{x:,}"))

Total de interações com UTI (soma de qtd_internacoes_uti, não linha): 813,422

Por ano:
ano
2022    251,743
2023    271,679
2024    290,000
Name: qtd_internacoes_uti, dtype: object
```

```
[3]: print("Período:", df["ano"].min(), "a", df["ano"].max())
print("Meses por ano:", sorted(int(m) for m in df["mes"].unique()))
print()
print("Combinações (município, diag_princ, ano, mês) = linhas do dataset:", f"{len(df):,}")
print("Códigos de diagnóstico distintos (diag_princ):", df["diag_princ"].nunique())
print("Códigos de município distintos (município_6):", df["município_6"].nunique())
```

Evidência da célula [4] (código e saída)

[5] Separação entre residente de SP e de fora

Divide a base pela flag de residência e mede peso de cada lado, em interações e em municípios distintos.

Comandos e por que estes:

`df.loc[~df["reside_fora_sp"], ...]` — O `.loc` com máscara booleana filtra e seleciona coluna numa operação só; o `~` inverte a máscara.

`nunique()` nos dois recortes — Mostra que os 645 municípios de SP estão todos presentes, o que valida a cobertura do dado.

```
[5]: residentes_sp = df.loc[~df["reside_fora_sp"], "qtd_internacoes_uti"].sum()
    residentes_fora = df.loc[df["reside_fora_sp"], "qtd_internacoes_uti"].sum()

    municipios_sp = df.loc[~df["reside_fora_sp"], "municipio_6"].nunique()
    municipios_fora = df.loc[df["reside_fora_sp"], "municipio_6"].nunique()

    print(f"Municípios de SP que aparecem no dataset: {municipios_sp} (de 645 no total do estado)")
    print(f"Municípios de fora de SP que aparecem: {municipios_fora}")
    print()
    print(f"Internações de residente de SP: {residentes_sp;} ({residentes_sp/total_internacoes:.1%})")
    print(f"Internações de residente de fora de SP: {residentes_fora;} ({residentes_fora/total_internacoes:.1%})")

    print()
    print(f"Municípios de SP que aparecem no dataset: 645 (de 645 no total do estado)")
    print(f"Municípios de fora de SP que aparecem: 1530")

    print()
    print(f"Internações de residente de SP: 805,118 (99.0%)")
    print(f"Internações de residente de fora de SP: 8,304 (1.0%)")
```

Evidência da célula [5] (código e saída)

[6] Proporção ICSAP, o indicador central do projeto

Calcula quanto das internações são por condição sensível à atenção primária, no total e por ano.

Comandos e por que estes:

`df.loc[df["csap"] == "sim", ...].sum()` — Filtra e soma direto, sem criar DataFrame intermediário.

`groupby(["ano", "csap"]).sum().unstack(fill_value=0)` — O `unstack` vira a tabela de comprido para largo, deixando sim/não como colunas; o `fill_value` evita nulo onde a combinação não existe.

```
[6]: icsap = df.loc[df["csap"] == "sim", "qtd_internacoes_uti"].sum()
    nao_icsap = df.loc[df["csap"] == "nao", "qtd_internacoes_uti"].sum()

    print(f"ICSAP: {icsap;} ({icsap/total_internacoes:.1%})")
    print(f"Não-ICSAP: {nao_icsap;} ({nao_icsap/total_internacoes:.1%})")
    print()
    print("% ICSAP por ano:")
    tab = df.groupby(["ano", "csap"])[["qtd_internacoes_uti"].sum().unstack(fill_value=0)
    tab["%_icsap"] = (tab["sim"] / (tab["sim"] + tab["nao"])) * 100).round(1)
    print(tab)

    ICSAP: 152,069 (18.7%)
    Não-ICSAP: 661,353 (81.3%)

    % ICSAP por ano:
    csap    não    sim    %_icsap
    ano
    2022  205763  45980    18.3
    2023  219019  52660    19.4
    2024  236571  53429    18.4
```

Evidência da célula [6] (código e saída)

Etapa 2. Missing (valores ausentes)

[7] Inventário de valores ausentes

Conta nulo por coluna, ordena e converte em percentual. É o ponto de partida da etapa 2, e serve para decidir quais colunas merecem investigação de mecanismo.

Comandos e por que estes:

`df.isna().sum()` — Soma booleano por coluna: True vale 1.

`nulos[nulos > 0]` — Descarta coluna sem nulo, deixando só o que interessa.

`pd.DataFrame({...})` — Junta contagem e percentual numa tabela única, mais legível que dois prints.

```
[7]: nulos = df.isna().sum()
nulos = nulos[nulos > 0].sort_values(ascending=False)
pct = (nulos / len(df) * 100).round(2)

resumo_nulos = pd.DataFrame({"linhas_nulos": nulos, "pct_do_total": pct})
resumo_nulos
```

```
[7]:
```

	linhas_nulos	pct_do_total
grupo_csap_cod	384762	83.05
grupo_csap_nome	384762	83.05
qtd_leitos_nao_sus	49737	10.74
qtd_leitos_existentes	49737	10.74
qtd_leitos_sus	49737	10.74
qtd_leitos_contratados	49737	10.74
qtd_estabelecimentos	7833	1.69
nome_municipio	7833	1.69
populacao	7833	1.69

Evidência da célula [7] (código e saída)

[8] Teste da regra de nulo por design

Verifica se o grupo CSAP é nulo exatamente quando a internação não é ICSAP. Se a regra vale, aqueles 83% de nulo não são dado faltando, são ausência estrutural.

Comandos e por que estes:

`((df["csap"] == "não") != df["grupo_csap_cod"].isna()).sum()` — O `!=` entre duas máscaras é um XOR: conta as linhas onde as duas condições discordam. Resultado 0 prova a regra, o que é mais forte do que só olhar a proporção.

```
[8]: # confere a regra: grupo_csap_cod é nulo se e só se csap == "não"
inconsistente = ((df["csap"] == "não") != df["grupo_csap_cod"].isna()).sum()
print(f"Linhas onde a regra 'grupo_csap_cod nulo <=> csap=não' NÃO vale: {inconsistente}")

Linhas onde a regra 'grupo_csap_cod nulo <=> csap=não' NÃO vale: 0
```

Evidência da célula [8] (código e saída)

[9] Decomposição do nulo de população

Separa os 7.833 nulos de população em duas causas: quem mora fora de SP (esperado) e quem mora em SP sem população (bug de merge).

Comandos e por que estes:

`(pop_nula & df["reside_fora_sp"]).sum()` — O `&` combina duas máscaras elemento a elemento, isolando cada causa.

Separar em vez de somar — Nulo com causas diferentes precisa de tratamento diferente. Os 9 registros restantes ficam documentados como bug conhecido em vez de descartados em silêncio.

```
[9]: pop_nula = df["populacao"].isna()

fora_sp = (pop_nula & df["reside_fora_sp"]).sum()
bug_sp = (pop_nula & ~df["reside_fora_sp"]).sum()

print(f"Total nulo: {pop_nula.sum():,}")
print(f" Residente de fora de SP (esperado – não faz sentido ter população/CNES de SP pra quem não mora aqui): {fora_sp:,}")
print(f" Residente de SP sem população (bug de ordem de merge, documentado no .docx seção 2.8, não corrigido): {bug_sp:,}")

Total nulo: 7,833
Residente de fora de SP (esperado – não faz sentido ter população/CNES de SP pra quem não mora aqui): 7,824
Residente de SP sem população (bug de ordem de merge, documentado no .docx seção 2.8, não corrigido): 9
```

Evidência da célula [9] (código e saída)

[10] Decomposição do nulo de leitos

Mesma lógica da célula anterior, agora nos 49.737 nulos de leito: quanto vem do grupo já explicado e quanto é gap de cadastro entre CNES-LT e CNES-ST.

Comandos e por que estes:

`leito_nulo & df["qtd_estabelecimentos"].isna()` — Cruza os dois nulos para ver se são a mesma linha.

`.notna()` — O complemento do isna, para o outro lado da conta. Os 41.904 restantes são o gap real de cadastro.

```
[10]: leito_nulo = df["qtd_leitos_existentes"].isna()
      junto_com_grupo_781 = (leito_nulo & df["qtd_estabelecimentos"].isna()).sum()
      so_leito = (leito_nulo & df["qtd_estabelecimentos"].notna()).sum()

      print(f"Total nulo: {leito_nulo.sum():,}")
      print(f"Mesmas linhas do grupo anterior (fora de SP + bug): {junto_com_grupo_781:,}")
      print(f"Só leito nulo, estabelecimento preenchido (gap CNES-LT vs CNES-ST): {so_leito:,}")

Total nulo: 49,737
Mesmas linhas do grupo anterior (fora de SP + bug): 7,833
Só leito nulo, estabelecimento preenchido (gap CNES-LT vs CNES-ST): 41,904
```

Evidência da célula [10] (código e saída)

[11] Teste de padrão monotônico

Checa se existe alguma linha com estabelecimento nulo e leito preenchido. Se o resultado é zero, o padrão de nulo é monotônico: um nulo sempre engloba o outro, nunca o contrário.

Comandos e por que estes:

Contar o caso impossível — Padrão monotônico é uma propriedade que se prova procurando o contraexemplo. Achar zero é o resultado esperado e é o que valida a classificação.

```
[11]: so_estabelecimento_nulo_leito_preenchido = (
      df["qtd_estabelecimentos"].isna() & df["qtd_leitos_existentes"].notna()
    ).sum()
      print(f"Linhas com estabelecimento nulo MAS leito preenchido: {so_estabelecimento_nulo_leito_preenchido}")
      print(f"Esperado 0 se o padrão for monotônico - nulo de leito 'engloba' o de estabelecimento")

Linhas com estabelecimento nulo MAS leito preenchido: 0
Esperado 0 se o padrão for monotônico - nulo de leito 'engloba' o de estabelecimento
```

Evidência da célula [11] (código e saída)

[12] Tabela cruzada de residência x nulo de população

Mostra em 2x2 o que a célula 9 já contou, em formato de tabela de contingência.

Comandos e por que estes:

`pd.crosstab(a, b)` — Monta a tabela de frequência cruzada em uma linha; fazer o mesmo com groupby exigiria três comandos.

Renomear colunas e índice — Crosstab nomeia as colunas como True/False; renomear deixa a tabela legível sem consultar o código.

```
[12]: tabela_cruzada = pd.crosstab(df["reside_fora_sp"], df["populacao"].isna())
      tabela_cruzada.columns = ["populacao_preenchida", "populacao_nula"]
      tabela_cruzada.index.name = "reside_fora_sp"
      tabela_cruzada
```

```
[12]:
```

	populacao_preenchida	populacao_nula
reside_fora_sp		
False	455466	9
True	0	7824

Evidência da célula [12] (código e saída)

[13] Nulo de leito por porte de município

Divide os municípios em quartis de número de estabelecimentos e mede a taxa de nulo em cada um. É o teste que decide o mecanismo do nulo.

Comandos e por que estes:

`pd.qcut(serie, q=4, labels=..., duplicates="drop")` — `qcut` corta por quantil (grupos de tamanho igual), diferente de `cut`, que corta por valor. O `duplicates="drop"` evita erro quando há empate nas bordas.

`groupby("porte", observed=True)["leito_nulo"].mean()` — A média de uma máscara booleana é a proporção. O `observed=True` impede que categorias vazias apareçam.

O que o resultado prova — 36,3% de nulo no quartil menor e 0% nos outros três descarta MCAR: o nulo depende de uma variável observada, ou seja, é MAR.

```
[13]: sub = df[df["qtd_estabelecimentos"].notna()].copy()
      sub["leito_nulo"] = sub["qtd_leitos_existentes"].isna()
      sub["porte"] = pd.qcut(
          sub["qtd_estabelecimentos"], q=4,
          labels=["Q1 (menor)", "Q2", "Q3", "Q4 (maior)"], duplicates="drop"
      )

      taxa_nulo_por_porte = (sub.groupby("porte", observed=True)["leito_nulo"].mean() * 100).round(1)
      taxa_nulo_por_porte

[13]: porte
      Q1 (menor)    36.3
      Q2          0.2
      Q3          0.0
      Q4 (maior)    0.0
      Name: leito_nulo, dtype: float64
```

Evidência da célula [13] (código e saída)

Etapa 3. Distribuição das métricas centrais

[14] Agregação para o grão município x ano

Constrói a primeira tabela analítica, somando internações, valor e dias de UTI por município e ano, e traz a contagem de ICSAP por um `merge` à parte.

Comandos e por que estes:

`groupby(...).agg(nome=(coluna, funcao))` — A sintaxe de agregação nomeada gera colunas já com o nome final, evitando índice hierárquico.

`populacao=("populacao", "median")` — População se repete por linha dentro do mesmo município-ano; `median` devolve o valor sem somar indevidamente.

`merge(..., how="left") + fillna(0)` — Município sem nenhuma ICSAP não aparece no agrupamento filtrado. O `left` preserva a linha e o `fillna` registra zero, que aqui é informação real e não ausência.

```
[14]: df_sp_mun = df[~df["reside_fora_sp"]].copy()

metricas_centrais = (
    df_sp_mun.groupby(["municipio_6", "ano"])
    .agg(
        n_internacoes=("qtd_internacoes_uti", "sum"),
        soma_val_tot=("soma_val_tot", "sum"),
        soma_uti_mes_tot=("soma_uti_mes_to", "sum"),
        populacao=("populacao", "median"),
        nome_municipio=("nome_municipio", "first"),
    )
    .reset_index()
)

n_icsap = (
    df_sp_mun[df_sp_mun["icsap"] == "sim"]
    .groupby(["municipio_6", "ano"])["qtd_internacoes_uti"]
    .sum()
    .rename("n_icsap")
)

metricas_centrais = metricas_centrais.merge(n_icsap, on=["municipio_6", "ano"], how="left")
metricas_centrais["n_icsap"] = metricas_centrais["n_icsap"].fillna(0)

print(f"len(metricas_centrais):, combinações município x ano (máximo possível: 645 x 3 = 1.935)")
metricas_centrais.head()

1,935 combinações município x ano (máximo possível: 645 x 3 = 1.935)
[14]:
```

	municipio_6	ano	n_internacoes	soma_val_tot	soma_uti_mes_to	populacao	nome_municipio	n_icsap
0	350010	2022	357	2409622.85	2436	34687.0	Adamantina - SP	81.0
1	350010	2023	340	2034558.55	1740	34687.0	Adamantina - SP	61.0
2	350010	2024	335	2116637.02	1873	35642.0	Adamantina - SP	59.0
3	350020	2022	64	624747.47	381	4351.0	Adolfo - SP	10.0
4	350020	2023	53	431688.78	242	4351.0	Adolfo - SP	20.0

Evidência da célula [14] (código e saída)

[15] Anualização de leitos e estabelecimentos

Converte as séries mensais de capacidade instalada em valor anual, seguindo a convenção CONASS/DATASUS (soma dos 12 meses dividida por 12). É a célula com a decisão técnica mais delicada da AED.

Comandos e por que estes:

drop_duplicates(subset=[municipio, ano, mes]) — Leito e estabelecimento se repetem em toda linha de diagnóstico do mesmo mês. Sem deduplicar, o mesmo leito seria contado dezenas de vezes.

sum(min_count=1) — O padrão do pandas soma um grupo inteiramente nulo como 0. Com min_count=1, o resultado é NaN, preservando o nulo. Sem esse argumento, 853 combinações município-ano (44,1%) virariam 'zero leito' fabricado.

mean() para estabelecimentos — mean já ignora nulo por padrão, então aqui não é preciso o mesmo cuidado.

```
[15]: mensal = df_sp_mun.drop_duplicates(subset=["municipio_6", "ano", "mes"])[
    ["municipio_6", "ano", "mes", "qtd_leitos_existentes", "qtd_estabelecimentos"]
]

por_municipio_ano = mensal.groupby(["municipio_6", "ano"])

# Leitos: convenção oficial CONASS/DATASUS (soma dos 12 meses / 12).
# min_count=1 é essencial: sem isso, um grupo com os 12 meses nulos soma "0" por
# padrão do pandas, e viraria "0 leitos" fabricado em vez de nulo de verdade.
leitos_soma_sem_min_count = por_municipio_ano["qtd_leitos_existentes"].sum()
leitos_soma = por_municipio_ano["qtd_leitos_existentes"].sum(min_count=1)
leitos_anual = (leitos_soma / 12).rename("leitos_anual_media")

# estabelecimentos: média dos meses disponíveis (mean já ignora nulo por padrão)
estabelecimentos_anual = por_municipio_ano["qtd_estabelecimentos"].mean().rename("estabelecimentos_anual_media")

anualizado = pd.concat([leitos_anual, estabelecimentos_anual], axis=1).reset_index()

todos_nulos = (leitos_soma_sem_min_count == 0) & (leitos_soma.isna())
total_combinacoes = por_municipio_ano.ngroups
print(f"Combinações município-ano com os 12 meses de leito nulos: {todos_nulos.sum():,} de {total_combinacoes:,} ({todos_nulos.sum()/total_combinacoes:.1%})")
print(f"Sem min_count=1, essas linhas virariam '0 leitos' por engano.")
anualizado.head()

Combinações município-ano com os 12 meses de leito nulos: 853 de 1,935 (44.1%)
Sem min_count=1, essas linhas virariam '0 leitos' por engano.
[15]:
```

	municipio_6	ano	leitos_anual_media	estabelecimentos_anual_media
0	350010	2022	242.833333	153.00
1	350010	2023	233.416667	160.25
2	350010	2024	206.000000	186.75
3	350020	2022	NaN	6.00
4	350020	2023	NaN	5.25

Evidência da célula [15] (código e saída)

[16] Cálculo das cinco métricas centrais

Junta as duas tabelas anteriores e deriva taxa por 10 mil habitantes, percentual ICSAP, custo médio e dias médios de UTI.

Comandos e por que estes:

merge(on=[municipio, ano], how="left") — Left preserva todas as combinações município-ano mesmo onde a capacidade instalada é nula.

Normalizar por população — Número absoluto de internação mede tamanho de cidade. A taxa por 10 mil habitantes é o que permite comparar municípios entre si.

```
[16]: tabela_municipio_ano = metricas_centrais.merge(anualizado, on=["municipio_6", "ano"], how="left")

tabela_municipio_ano["taxa_internacao_uti_10k"] = (
    tabela_municipio_ano["n_internacoes"] / tabela_municipio_ano["populacao"] * 10000
)
tabela_municipio_ano["taxa_icsap_10k"] = (
    tabela_municipio_ano["n_icsap"] / tabela_municipio_ano["populacao"] * 10000
)
tabela_municipio_ano["pct_icsap"] = (
    tabela_municipio_ano["n_icsap"] / tabela_municipio_ano["n_internacoes"] * 100
)
tabela_municipio_ano["custo_medio_internacao"] = (
    tabela_municipio_ano["soma_val_tot"] / tabela_municipio_ano["n_internacoes"]
)
tabela_municipio_ano["dias_medios_uti_internacao"] = (
    tabela_municipio_ano["soma_uti_mes_to"] / tabela_municipio_ano["n_internacoes"]
)

print(f"({len(tabela_municipio_ano):,}) linhas, {tabela_municipio_ano.shape[1]} colunas")
tabela_municipio_ano.head()

1,935 linhas, 15 colunas
```

	municipio_6	ano	n_internacoes	soma_val_tot	soma_uti_mes_to	populacao	nome_municipio	n_icsap	leitos_anual_media	estabelecimentos_anual_media	taxa_internacao_uti_10k	taxa_icsap_10k	pct_icsap	custo_medio_internacao
0	350010	2022	357	2409622.85	2436	34687.0	Adamantina - SP	81.0	242.833333	153.00	102.920402	23.351688	22.689076	
1	350010	2023	340	2034558.55	1740	34687.0	Adamantina - SP	61.0	233.416667	160.25	98.019431	17.585839	17.941176	
2	350010	2024	335	2116637.02	1873	35642.0	Adamantina - SP	59.0	206.000000	186.75	93.990236	16.553504	17.611940	
3	350020	2022	64	624747.47	381	4351.0	Adolfo - SP	10.0	NaN	6.00	147.092622	22.983222	15.625000	
4	350020	2023	53	431688.78	242	4351.0	Adolfo - SP	20.0	NaN	5.25	121.811078	45.966444	37.735849	

Evidência da célula [16] (código e saída)

[17] Distribuição das métricas centrais

Gera mediana, quartis, mínimo, máximo e amplitude interquartil das cinco métricas.

Comandos e por que estes:

describe(percentiles=[0.25, 0.5, 0.75]).T — O .T transpõe para colocar métricas nas linhas, formato mais legível quando são poucas colunas e muitas medidas.

iqr calculado à mão — O describe não devolve IQR; ele é a base do critério de outlier da etapa seguinte, então é calculado aqui.

```
[17]: metricas_distribuicao = [
    "taxa_internacao_uti_10k", "taxa_icsap_10k", "pct_icsap",
    "custo_medio_internacao", "dias_medios_uti_internacao",
]

distribuicao = tabela_municipio_ano[metricas_distribuicao].describe(percentiles=[0.25, 0.5, 0.75]).T
distribuicao["iqr"] = distribuicao["75%"] - distribuicao["25%"]
distribuicao[["50%", "25%", "75%", "iqr", "min", "max"]].round(2)
```

	50%	25%	75%	iqr	min	max
taxa_internacao_uti_10k	67.28	50.71	91.44	40.73	11.11	233.67
taxa_icsap_10k	12.05	7.68	19.16	11.47	0.00	64.25
pct_icsap	18.32	14.14	23.53	9.39	0.00	58.33
custo_medio_internacao	7933.72	6726.19	9144.27	2418.08	3463.27	20396.80
dias_medios_uti_internacao	6.35	5.48	7.32	1.84	2.09	17.67

Evidência da célula [17] (código e saída)

Etapa 4. Outliers

[18] Cercas de Tukey e contagem de outliers

Aplica o critério clássico de 1,5 vezes o IQR em cada métrica e conta quantos município-ano ficam fora, separando cauda inferior e superior.

Comandos e por que estes:

`quantile([0.25, 0.75])` — Devolve os dois quartis numa chamada, evitando percorrer a série duas vezes.

Criar coluna outlier_<métrica> — Guardar a máscara em coluna permite, na célula 20, contar em quantas métricas cada linha é extrema.

Separar outliers_baixo e outliers_alto — A direção do outlier é informação: em taxa ICSAP não existe cauda inferior, porque o mínimo já é zero.

```
[18]: metricas_distribuicao = [
    "taxa_internacao_uti_10k", "taxa_icsap_10k", "pct_icsap",
    "custo_medio_internacao", "dias_medios_uti_internacao",
]

fences = []
outlier_cols = []
for m in metricas_distribuicao:
    q1, q3 = tabela_municipio_ano[m].quantile([0.25, 0.75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    col = f"outlier_{m}"
    tabela_municipio_ano[col] = (tabela_municipio_ano[m] < lower) | (tabela_municipio_ano[m] > upper)
    outlier_cols.append(col)
    fences.append({
        "metrica": m, "fence_inferior": round(lower, 2), "fence_superior": round(upper, 2),
        "outliers_total": int(tabela_municipio_ano[col].sum()),
        "outliers_baixo": int((tabela_municipio_ano[m] < lower).sum()),
        "outliers_alto": int((tabela_municipio_ano[m] > upper).sum()),
    })

pd.DataFrame(fences)
```

```
[18]:
```

	metrica	fence_inferior	fence_superior	outliers_total	outliers_baixo	outliers_alto
0	taxa_internacao_uti_10k	-10.39	152.53	40	0	40
1	taxa_icsap_10k	-9.53	36.37	86	0	86
2	pct_icsap	0.04	37.62	40	17	23
3	custo_medio_internacao	3099.07	12771.39	26	0	26
4	dias_medios_uti_internacao	2.73	10.08	47	9	38

Evidência da célula [18] (código e saída)

[19] Boxplot das cinco métricas

Visualiza as distribuições lado a lado, confirmando graficamente o que a tabela de cercas mostrou em número.

Comandos e por que estes:

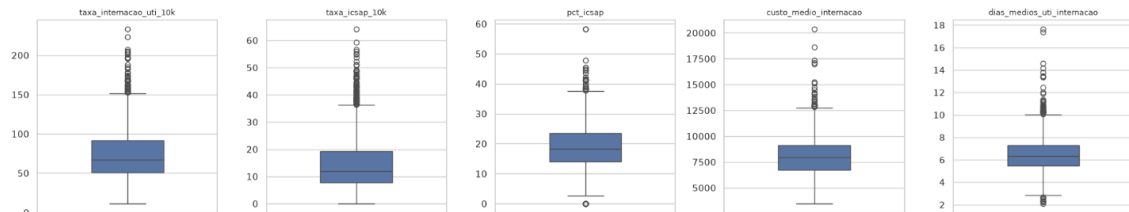
`plt.subplots(1, 5)` — Cria a grade de eixos numa chamada e devolve o array de eixos para o laço.

`sns.boxplot(y=..., ax=eixo)` — O parâmetro `ax` direciona cada gráfico para o eixo certo; sem ele, todos iriam para a mesma figura.

`tight_layout(rect=...)` — Ajusta o espaçamento reservando a faixa superior para o título geral.

```
[19]: fig, eixos = plt.subplots(1, 5, figsize=(19, 4.5))
for eixo, m in zip(eixos, metricas_distribuicao):
    sns.boxplot(y=tabela_municipio_ano[m], ax=eixo, color="#4C72B0", width=0.4)
    eixo.set_title(m, fontsize=9)
    eixo.set_ylabel("")
fig.suptitle("Distribuição das 5 métricas centrais (boxplot Tukey, 1,5×IQR)", y=1.04)
plt.tight_layout(rect=[0, 0, 1, 0.94])
plt.show()
```

Distribuição das 5 métricas centrais (boxplot Tukey, 1,5×IQR)



`taxa_icsap_10k` é a métrica com mais outlier (86 município-ano), todos pra cima (nenhum fence inferior, porque o mínimo da distribuição já é 0 ICSAP, não dá pra ter menos que isso). `custo_medio_internacao` é a que tem menos (26), e é a única sem nenhum outlier pra baixo — custo muito baixo não é incomum o bastante pra passar a fence.

Evidência da célula [19] (código e saída)

[20] Concentração de outliers por linha

Conta em quantas das cinco métricas cada município-ano é extremo. Responde se o extremo é isolado ou se existe um conjunto de municípios problemáticos.

Comandos e por que estes:

`tabela[outlier_cols].sum(axis=1)` — O `axis=1` soma ao longo das colunas, ou seja, por linha.

`value_counts().sort_index()` — O `sort_index` ordena por número de métricas (0, 1, 2, 3) em vez de por frequência, que é o padrão.

```
[20]: tabela_municipio_ano["n_metricas_outlier"] = tabela_municipio_ano[outlier_cols].sum(axis=1)
tabela_municipio_ano["n_metricas_outlier"].value_counts().sort_index()

[20]: n_metricas_outlier
0    1750
1     134
2      48
3        3
Name: count, dtype: int64

1.750 município-ano (90.4%) não estoura fence em nenhuma métrica. Só 3 estouram 3 das 5 ao mesmo tempo, nenhum estoura mais que isso. A maioria do "extremo" é isolado numa métrica só, não um padrão geral de município problemático.
```

Evidência da célula [20] (código e saída)

[21] Inspeção dos casos extremos

Lista os três município-ano que estouram três métricas ao mesmo tempo, com população ao lado.

Comandos e por que estes:

Ordenar por população — Coloca a explicação à vista: são municípios de 1,5 mil a 5,5 mil habitantes, onde poucos casos movem muito a taxa.

```
[21]: cols_show = ["nome_municipio", "ano", "populacao", "n_internacoes", "n_icsap"] + metricas_distribuicao
tres_metricas = tabela_municipio_ano[tabela_municipio_ano["n_metricas_outlier"] == 3]
tres_metricas[cols_show].sort_values("populacao")

[21]:
```

	nome_municipio	ano	populacao	n_internacoes	n_icsap	taxa_internacao_uti_10k	taxa_icsap_10k	pct_icsap	custo_medio_internacao	dias_medios_uti_internacao
543	Flora Rica - SP	2022	1487.0	4	0.0	26.899798	0.000000	0.000000	15223.680000	12.000000
878	Jumirim - SP	2024	3123.0	10	0.0	32.020493	0.000000	0.000000	13870.760000	13.400000
1862	Ubarana - SP	2024	5456.0	91	30.0	166.788856	54.985337	32.967033	12838.155604	8.186813

Evidência da célula [21] (código e saída)

[22] Volume mediano de quem é e de quem não é outlier

Compara a mediana de internações dos dois grupos, para testar a hipótese de que outlier é efeito de denominador pequeno.

Comandos e por que estes:

Comparar medianas em vez de médias — Mediana não é arrastada pela própria cauda longa que está sendo investigada.

O que o resultado mostra — 42 contra 98 internações. O extremo está concentrado em município de baixo volume, o que sustenta manter as linhas em vez de removê-las.

```
[22]: tem_algun_outlier = tabela_municipio_ano["n_metricas_outlier"] > 0

mediana_n_outlier = tabela_municipio_ano.loc[tem_algun_outlier, "n_internacoes"].median()
mediana_n_nao_outlier = tabela_municipio_ano.loc[~tem_algun_outlier, "n_internacoes"].median()

print(f"Mediana de n_internacoes | é outlier em alguma métrica: {mediana_n_outlier:.0f}")
print(f"Mediana de n_internacoes | não é outlier em nenhuma: {mediana_n_nao_outlier:.0f}")

Mediana de n_internacoes | é outlier em alguma métrica: 42
Mediana de n_internacoes | não é outlier em nenhuma: 98
```

Evidência da célula [22] (código e saída)

[23] Caso Tupã, o outlier explicável

Isola um município específico nos três anos para entender por que ele aparece no topo de várias métricas.

Comandos e por que estes:

.str.contains("Tupã", na=False) — Busca por parte do texto, dispensando o nome exato com sufixo de UF. O `na=False` evita erro em nome nulo.

Investigar o caso em vez de descartar — Tupã é polo regional com 58 leitos por 10 mil habitantes contra mediana estadual de 21. O extremo tem causa estrutural.

```
[23]: colunas_tupa = [
    "nome_municipio", "ano", "populacao", "n_internacoes", "n_icsap",
    "pct_icsap", "taxa_internacao_uti_10k", "taxa_icsap_10k",
]
tabela_municipio_ano[tabela_municipio_ano["nome_municipio"].str.contains("Tupã", na=False)][colunas_tupa]

[23]:
```

	nome_municipio	ano	populacao	n_internacoes	n_icsap	pct_icsap	taxa_internacao_uti_10k	taxa_icsap_10k
1848	Tupã - SP	2022	63928.0	956	235.0	24.581590	149.543236	36.760105
1849	Tupã - SP	2023	63928.0	1202	294.0	24.459235	188.024027	45.989238
1850	Tupã - SP	2024	65416.0	1339	291.0	21.732636	204.689984	44.484530

Evidência da célula [23] (código e saída)

Etapa 5. Correlação

[24] Matriz de correlação com capacidade instalada

Correlaciona as cinco métricas centrais com leitos e estabelecimentos, por Spearman e por Pearson.

Comandos e por que estes:

corr(method="spearman") — Spearman é sobre postos, então não assume linearidade nem normalidade e é robusto às caudas longas já documentadas.

Reportar quantas linhas entram — Só 55,9% têm leito válido, por causa do gap CNES-LT. Declarar o N da correlação evita que ela seja lida como se cobrisse a base inteira.

```
[24]: colunas_correlacao = metricas_distribuicao * ["leitos_anual_media", "estabelecimentos_anual_media"]

n_leito_valido = tabela_municipio_ano["leitos_anual_media"].notna().sum()
print(f"Correlações com leitos_anual_media usam {n_leito_valido} de {len(tabela_municipio_ano):,} linhas "
      f"({n_leito_valido/len(tabela_municipio_ano):.1%}) - o resto é nulo (achado da etapa 3, gap CNES-LT).")
print(f"Correlações com estabelecimentos_anual_media usam {tabela_municipio_ano['estabelecimentos_anual_media'].notna().sum():,} linhas (sem nulo relevante).")

correlacao_spearman = tabela_municipio_ano[colunas_correlacao].corr(method="spearman")
correlacao_pearson = tabela_municipio_ano[colunas_correlacao].corr(method="pearson")

correlacao_spearman.loc[metricas_distribuicao, ["leitos_anual_media", "estabelecimentos_anual_media"]].round(3)

Correlações com leitos_anual_media usam 1,082 de 1,935 linhas (55.9%) - o resto é nulo (achado da etapa 3, gap CNES-LT).
Correlações com estabelecimentos_anual_media usam 1,935 linhas (sem nulo relevante).
```

```
[24]:
```

	leitos_anual_media	estabelecimentos_anual_media
taxa_internacao_uti_10k	0.016	-0.240
taxa_icsap_10k	0.090	-0.120
pct_icsap	0.145	0.017
custo_medio_internacao	-0.092	-0.013
dias_medios_uti_internacao	0.120	0.207

Evidência da célula [24] (código e saída)

[25] Divergência entre Spearman e Pearson

Mede a diferença absoluta entre os dois métodos em cada par de variáveis.

Comandos e por que estes:

Subtrair as duas matrizes e aplicar `.abs()` — Operação elemento a elemento entre DataFrames de mesmo formato.

Por que reportar a divergência — Diferença máxima de 0,186 indica que a relação não é linear. Publicar isso é mais honesto do que escolher o método que dá o número desejado.

```
[25]: diferenca_metodo = (
      correlacao_spearman.loc[metricas_distribuicao, ["leitos_anual_media", "estabelecimentos_anual_media"]]
      - correlacao_pearson.loc[metricas_distribuicao, ["leitos_anual_media", "estabelecimentos_anual_media"]]
      ).abs()

print(f"Maior diferença absoluta entre Spearman e Pearson: {diferenca_metodo.values.max():.3f}")
diferenca_metodo.round(3)
```

Maior diferença absoluta entre Spearman e Pearson: 0.186

```
[25]:
```

	leitos_anual_media	estabelecimentos_anual_media
taxa_internacao_uti_10k	0.052	0.186
taxa_icsap_10k	0.123	0.080
pct_icsap	0.159	0.028
custo_medio_internacao	0.114	0.018
dias_medios_uti_internacao	0.075	0.169

Evidência da célula [25] (código e saída)

[26] Leitos por 10 mil habitantes

Normaliza a capacidade instalada pela população e compara Tupã com a mediana estadual.

Comandos e por que estes:

Normalizar antes de comparar — Leito absoluto mede tamanho de cidade; per capita mede oferta relativa, que é o conceito em discussão.

```
[26]: tabela_municipio_ano["leitos_10k_hab"] = (
    tabela_municipio_ano["leitos_anual_media"] / tabela_municipio_ano["populacao"] * 10000
)

mediana_leitos_10k = tabela_municipio_ano["leitos_10k_hab"].median()
tupa_leitos_10k = tabela_municipio_ano[tabela_municipio_ano["nome_municipio"].str.contains("Tupã", na=False)][
    ["ano", "leitos_10k_hab", "taxa_icsap_10k"]
]

print(f'Mediana geral de leitos por 10 mil hab: (mediana_leitos_10k:.1f)')
tupa_leitos_10k
```

Mediana geral de leitos por 10 mil hab: 21.2

```
[26]:
```

	ano	leitos_10k_hab	taxa_icsap_10k
1848	2022	59.389730	36.760105
1849	2023	58.190464	45.989238
1850	2024	57.019689	44.484530

Evidência da célula [26] (código e saída)

Etapas 6. Sazonalidade

[27] Série temporal mês a mês do estado

Muda a unidade de análise de município para ano e mês do estado inteiro, e recalcula as mesmas métricas nesse novo grão.

Comandos e por que estes:

drop_duplicates antes de somar população — População é atributo do município, não da linha de diagnóstico; sem deduplicar, o denominador estadual explodiria.

Reconstruir as métricas no novo grão — Métrica calculada por município não pode ser somada para virar métrica estadual: a soma de razões não é a razão das somas.

Checar 36 linhas — 3 anos vezes 12 meses. É um teste de integridade barato e imediato.

```
[27]: import calendar

pop_dim_sp = (
    df[~df["reside_fora_sp"]]
    .drop_duplicates(subset=["municipio_6", "ano"])[["municipio_6", "ano", "populacao"]]
)
populacao_sp_ano = pop_dim_sp.groupby("ano")["populacao"].sum()

tabela_ano_mes = (
    df.groupby(["ano", "mes"])
    .agg(
        n_internacoes=("qtd_internacoes_uti", "sum"),
        soma_val_tot=("soma_val_tot", "sum"),
        soma_uti_mes_to=("soma_uti_mes_to", "sum"),
    )
    .reset_index()
)

n_icsap_mes = (
    df[df["icsap"] == "sim"].groupby(["ano", "mes"])[["qtd_internacoes_uti"].sum().rename("n_icsap")]
)

tabela_ano_mes = tabela_ano_mes.merge(n_icsap_mes, on=["ano", "mes"], how="left")
tabela_ano_mes["n_icsap"] = tabela_ano_mes["n_icsap"].fillna(0)

tabela_ano_mes = tabela_ano_mes.merge(populacao_sp_ano.rename("populacao_sp"), on="ano", how="left")

tabela_ano_mes["taxa_internacao_uti_10k"] = tabela_ano_mes["n_internacoes"] / tabela_ano_mes["populacao_sp"] * 10000
tabela_ano_mes["taxa_icsap_10k"] = tabela_ano_mes["n_icsap"] / tabela_ano_mes["populacao_sp"] * 10000
tabela_ano_mes["pct_icsap"] = tabela_ano_mes["n_icsap"] / tabela_ano_mes["n_internacoes"] * 100
tabela_ano_mes["custo_medio_internacao"] = tabela_ano_mes["soma_val_tot"] / tabela_ano_mes["n_internacoes"]
tabela_ano_mes["dias_medios_uti_internacao"] = tabela_ano_mes["soma_uti_mes_to"] / tabela_ano_mes["n_internacoes"]

metricas_sazonais = ["taxa_internacao_uti_10k", "taxa_icsap_10k", "pct_icsap", "custo_medio_internacao", "dias_medios_uti_internacao"]

print(f'Tabela ano+mês: {tabela_ano_mes.shape[0]} linhas (esperado 36 = 3 anos x 12 meses).')
tabela_ano_mes[["ano", "mes"] + metricas_sazonais].round(2)

Tabela ano+mês: 36 linhas (esperado 36 = 3 anos x 12 meses).
```

Evidência da célula [27] (código)

Tabela ano+mês: 36 linhas (esperado 36 = 3 anos x 12 meses).

[27]:

	ano	mes	taxa_internacao_uti_10k	taxa_icsap_10k	pct_icsap	custo_medio_internacao	dias_medios_uti_internacao
0	2022	1	4.79	0.80	16.72	7811.55	7.24
1	2022	2	4.39	0.65	14.74	8721.79	7.46
2	2022	3	4.67	0.77	16.51	7838.84	7.18
3	2022	4	4.55	0.85	18.62	7849.55	7.02
4	2022	5	4.65	0.89	19.22	7789.39	6.92
5	2022	6	4.50	0.84	18.56	7662.70	6.86
6	2022	7	4.97	0.94	18.83	7930.83	7.18
7	2022	8	4.84	0.90	18.69	7929.25	7.06
8	2022	9	4.76	0.91	19.15	8108.75	7.06
9	2022	10	4.97	0.96	19.36	8091.38	7.06
10	2022	11	4.77	0.93	19.54	8079.80	7.09
11	2022	12	4.84	0.91	18.91	7876.55	7.00
12	2023	1	5.00	0.96	19.13	7920.38	6.99
13	2023	2	4.74	0.96	20.23	7869.65	6.85
14	2023	3	5.24	1.00	19.08	7785.59	6.72
15	2023	4	4.96	0.93	18.71	7794.23	6.77
16	2023	5	5.10	1.03	20.14	7672.24	6.72
17	2023	6	5.23	1.03	19.73	7942.01	6.86
18	2023	7	5.22	1.03	19.75	8065.99	6.98
19	2023	8	5.42	1.09	20.15	8065.60	6.85
20	2023	9	5.01	0.94	18.81	8027.39	6.68
21	2023	10	5.15	1.00	19.44	8097.49	6.83
22	2023	11	4.96	0.94	19.02	7975.71	6.70
23	2023	12	5.15	0.95	18.37	8099.48	6.67
24	2024	1	5.11	0.95	18.60	7893.28	6.80
25	2024	2	4.96	0.88	17.83	8059.31	6.56
26	2024	3	5.41	0.99	18.32	8185.43	6.65

Evidência da célula [27] (saída, parte 1 de 2)

27	2024	4	5.23	0.99	18.89	8128.76	6.60
28	2024	5	5.25	0.93	17.65	8359.89	6.78
29	2024	6	5.20	0.95	18.23	8231.18	6.82
30	2024	7	5.34	0.97	18.10	8291.71	6.83
31	2024	8	5.42	1.04	19.25	8298.34	6.74
32	2024	9	5.23	0.95	18.22	8379.28	6.62
33	2024	10	5.50	1.01	18.42	8331.29	6.63
34	2024	11	5.17	0.96	18.54	8318.36	6.65
35	2024	12	5.27	1.00	19.00	8078.31	6.63

Evidência da célula [27] (saída, parte 2 de 2)

[28] Padrão sazonal por mês

Colapsa os três anos em um perfil mensal usando a mediana, e registra explicitamente que N=3 por mês.

Comandos e por que estes:

`calendar.monthrange(ano, mes)[1]` — Devolve o número de dias do mês, considerando ano bissexto.

Preparação para a normalização da célula seguinte.

`median()` em vez de `mean()` — Com apenas três observações por mês, a mediana é menos sensível a um ano atípico.

Registrar `n_anos` na tabela — Deixa o tamanho da amostra visível no próprio resultado, evitando que o número seja lido como teste estatístico.

```
[28]: tabela_ano_mes["dias_no_mes"] = tabela_ano_mes.apply(
      lambda r: calendar.monthrange(int(r["ano"]), int(r["mes"]))[1], axis=1
    )

padrao_sazonal = tabela_ano_mes.groupby("mes")[metricas_sazonais].median()
padrao_sazonal["n_anos"] = tabela_ano_mes.groupby("mes").size()

print("Padrão sazonal por mês (mediana dos 3 anos, N=3 por mês - leitura exploratória, não teste estatístico):")
padrao_sazonal.round(3)

Padrão sazonal por mês (mediana dos 3 anos, N=3 por mês - leitura exploratória, não teste estatístico):
[28]: taxa_internacao_uti_10k  taxa_icap_10k  pct_icap  custo_medio_internacao  dias_medios_uti_internacao  n_anos
mes
1          5.003          0.951    18.595          7893.275          6.990      3
2          4.735          0.885    17.833          8059.315          6.852      3
3          5.243          0.991    18.318          7838.838          6.718      3
4          4.955          0.927    18.708          7849.548          6.767      3
5          5.095          0.926    19.219          7789.388          6.783      3
6          5.203          0.948    18.556          7942.005          6.860      3
7          5.223          0.966    18.825          8065.992          6.977      3
8          5.423          1.044    19.246          8065.596          6.848      3
9          5.008          0.942    18.810          8108.752          6.684      3
10         5.147          1.001    19.359          8097.485          6.835      3
11         4.965          0.944    19.020          8079.805          6.700      3
12         5.147          0.945    18.907          8078.311          6.673      3
```

Evidência da célula [28] (código e saída)

[29] Correção pelo número de dias do mês

Divide a taxa mensal pelos dias do mês antes de comparar. Fevereiro tem 10% menos dias que março, e sem isso a comparação mede calendário, não sazonalidade.

Comandos e por que estes:

Dividir pela coluna `dias_no_mes` — Transforma volume mensal em taxa diária, que é comparável entre meses.

`(max - min) / median * 100` — Amplitude relativa: expressa a variação sazonal como percentual da mediana, comparável entre métricas de escalas diferentes.

```
[29]: tabela_ano_mes["taxa_internacao_uti_10k_dia"] = tabela_ano_mes["taxa_internacao_uti_10k"] / tabela_ano_mes["dias_no_mes"]
tabela_ano_mes["taxa_icsap_10k_dia"] = tabela_ano_mes["taxa_icsap_10k"] / tabela_ano_mes["dias_no_mes"]

padrao_sazonal_dia = tabela_ano_mes.groupby("mes")[["taxa_internacao_uti_10k_dia", "taxa_icsap_10k_dia"]].median()
variacao_pct = (padrao_sazonal_dia.max() - padrao_sazonal_dia.min()) / padrao_sazonal_dia.median() * 100

print("Taxa por DIA (corrige o viés de mês com menos dias, ex: fevereiro):")
print(padrao_sazonal_dia.sort_values("taxa_internacao_uti_10k_dia").round(4))
print()
print("Variação (mês mais alto - mês mais baixo) / mediana:")
print(variacao_pct.round(1))

Taxa por DIA (corrige o viés de mês com menos dias, ex: fevereiro):
taxa_internacao_uti_10k_dia taxa_icsap_10k_dia
mes
1      0.1614      0.0307
5      0.1644      0.0299
4      0.1652      0.0309
11     0.1655      0.0315
10     0.1660      0.0323
12     0.1660      0.0305
9      0.1669      0.0314
7      0.1685      0.0312
2      0.1691      0.0305
3      0.1691      0.0320
6      0.1734      0.0316
8      0.1749      0.0337

Variação (mês mais alto - mês mais baixo) / mediana:
taxa_internacao_uti_10k_dia      8.1
taxa_icsap_10k_dia             12.1
dtype: float64
```

Evidência da célula [29] (código e saída)

[30] Sazonalidade por grupo de causa

Repete a análise anterior separando por grupo CSAP, porque o agregado pode estar escondendo padrões opostos que se cancelam.

Comandos e por que estes:

isin(dicionario) — Ao passar um dicionário, o isin usa as chaves; filtra os quatro grupos em uma expressão.

Laco sobre .items() — Percorre código e nome juntos, mantendo o rótulo legível no resultado.

idxmin() e **idxmax()** — Devolvem o índice (o mês) do menor e do maior valor, não o valor. É o que interessa aqui.

O que muda — No agregado a variação é de 8%; em pulmonares, 67,9%. O agregado estava escondendo o achado.

```
[30]: grupos_sazonalidade = {"g12": "Cerebrovasculares", "g11": "Insuf. cardiaca", "g10": "Angina", "g08": "Pulmonares"}

sub_grupos = df[df["grupo_csap_cod"].isin(grupos_sazonalidade)].copy()
tabela_grupos_mes = (
    sub_grupos.groupby(["grupo_csap_cod", "ano", "mes"])[["qtd_internacoes_uti"]].sum().reset_index()
)
tabela_grupos_mes["dias_no_mes"] = tabela_grupos_mes.apply(
    lambda r: calendar.monthrange(int(r["ano"]), int(r["mes"]))[1], axis=1
)
tabela_grupos_mes["internacoes_dia"] = tabela_grupos_mes["qtd_internacoes_uti"] / tabela_grupos_mes["dias_no_mes"]

resumo_grupos = []
for cod, nome in grupos_sazonalidade.items():
    pool_grupo = tabela_grupos_mes[tabela_grupos_mes["grupo_csap_cod"] == cod].groupby("mes")["internacoes_dia"].median()
    variacao = (pool_grupo.max() - pool_grupo.min()) / pool_grupo.median() * 100
    resumo_grupos.append({
        "grupo": f"{cod}: {nome}",
        "mes_mais_baixo": int(pool_grupo.idxmin()),
        "mes_mais_alto": int(pool_grupo.idxmax()),
        "variacao_pct": round(variacao, 1),
    })

resumo_grupos_df = pd.DataFrame(resumo_grupos).sort_values("variacao_pct")
resumo_grupos_df
```

```
[30]:
```

	grupo	mes_mais_baixo	mes_mais_alto	variacao_pct
0	g12 (Cerebrovasculares)	5	8	14.2
1	g11 (Insuf. cardiaca)	4	9	19.6
2	g10 (Angina)	5	2	24.6
3	g08 (Pulmonares)	2	5	67.9

Evidência da célula [30] (código e saída)

Etapa 7. Recorte geográfico

[31] Ranking de capacidade instalada

Ordena os municípios por leitos por 10 mil habitantes, usando a mediana dos três anos.

Comandos e por que estes:

`groupby([...]).median()` — Mediana dos três anos evita que um ano de cadastro incompleto defina a posição do município.

`sort_values(ascending=False)` — Ranking decrescente; o `head(15)` limita a exibição sem perder a tabela completa.

```
[31]: ranking_leitos = (
    tabela_municipio_ano.groupby(["municipio_6", "nome_municipio"])["leitos_10k_hab"]
    .median()
    .reset_index()
    .sort_values("leitos_10k_hab", ascending=False)
)

mediana_geral_leitos = ranking_leitos["leitos_10k_hab"].median()
print(f"Mediana geral de leitos por 10 mil hab: {mediana_geral_leitos:.1f}")
print(f"Total de municípios no ranking: {len(ranking_leitos)}")
ranking_leitos.head(15).reset_index(drop=True)
```

Mediana geral de leitos por 10 mil hab: 21.3
Total de municípios no ranking: 645

```
[31]:
```

	municipio_6	nome_municipio	leitos_10k_hab
0	352450	Jaci - SP	283.725207
1	351390	Divinolândia - SP	166.696541
2	351080	Casa Branca - SP	144.571449
3	352260	Itapira - SP	132.181833
4	353260	Nhandeara - SP	130.937881
5	353890	Pirajui - SP	128.728100
6	354750	Santa Rita do Passa Quatro - SP	117.954603
7	351518	Espírito Santo do Pinhal - SP	110.696705
8	353620	Parquera-Açu - SP	102.168148
9	350290	Araçoiaba da Serra - SP	101.716857
10	352070	Indiaporã - SP	87.612558
11	352530	Jau - SP	78.547333
12	350170	Américo Brasiliense - SP	76.132358
13	355660	Vera Cruz - SP	70.754717
14	351450	Duartina - SP	70.571058

Evidência da célula [31] (código e saída)

[32] Correlação entre oferta e demanda

Testa se município com mais leito por habitante interna mais, e se leito por habitante está associado ao porte.

Comandos e por que estes:

`dropna()` antes de correlacionar — Deixa explícito que a correlação usa 364 dos 645 municípios, por causa do gap de leitos.

`.corr(...).iloc[0, 1]` — Extrai o único coeficiente de interesse da matriz 2x2.

```
[32]: resumo_municipio = (
    tabela_municipio_ano.groupby(["municipio_6", "nome_municipio"])
    .agg(leitos_10k_hab=("leitos_10k_hab", "median"), n_internacoes=("n_internacoes", "median"), populacao=("populacao", "median"))
    .dropna()
    .reset_index()
)

corr_leito_volume = resumo_municipio[["leitos_10k_hab", "n_internacoes"]].corr(method="spearman").iloc[0, 1]
corr_leito_populacao = resumo_municipio[["leitos_10k_hab", "populacao"]].corr(method="spearman").iloc[0, 1]

print(f"N de municípios com leito válido nos 3 anos: {len(resumo_municipio)} de {tabela_municipio_ano['municipio_6'].nunique()}")
print(f"Correlação (Spearman) leitos_10k_hab x n_internacoes: {corr_leito_volume:.3f}")
print(f"Correlação (Spearman) leitos_10k_hab x população: {corr_leito_populacao:.3f}")

N de municípios com leito válido nos 3 anos: 364 de 645
Correlação (Spearman) leitos_10k_hab x n_internacoes: -0.012
Correlação (Spearman) leitos_10k_hab x população: -0.120
```

Evidência da célula [32] (código e saída)

[33] Identificação dos polos regionais

Define polo como município simultaneamente no quartil superior de leito per capita e de volume de internação.

Comandos e por que estes:

quantile(0.75) nas duas variáveis — Corte por posição relativa, não por valor absoluto arbitrário.

Combinar as duas condições com & — Exigir as duas evita dois falsos positivos: cidade pequena com hospital grande (alta per capita, baixo volume) e capital (alto volume, per capita comum).

Resultado — 20 municípios de 364, incluindo São Paulo, Campinas, Ribeirão Preto e Tupã.

```
[33]: p75_leito = resumo_municipio["leitos_10k_hab"].quantile(0.75)
      p75_volume = resumo_municipio["n_internacoes"].quantile(0.75)

      candidatos_polo = resumo_municipio[
        (resumo_municipio["leitos_10k_hab"] >= p75_leito) & (resumo_municipio["n_internacoes"] >= p75_volume)
      ].sort_values("n_internacoes", ascending=False)

      print(f"Candidatos a polo regional (top 25% em leito per capita E top 25% em volume): {len(candidatos_polo)} de {len(resumo_municipio)} municípios")
      candidatos_polo.round(1).reset_index(drop=True)
```

Candidatos a polo regional (top 25% em leito per capita E top 25% em volume): 20 de 364 municípios

```
[33]:
```

	municipio_6	nome_municipio	leitos_10k_hab	n_internacoes	populacao
0	355030	São Paulo - SP	33.0	60093.0	11451999.0
1	350950	Campinas - SP	31.0	6865.0	1139047.0
2	354340	Ribeirão Preto - SP	38.5	3771.0	698642.0
3	354980	São José do Rio Preto - SP	53.7	3570.0	480393.0
4	354850	Santos - SP	47.5	3183.0	418608.0
5	350600	Bauru - SP	41.8	2610.0	379146.0
6	352900	Marília - SP	43.2	2448.0	237627.0
7	354880	São Caetano do Sul - SP	59.7	2030.0	165655.0
8	352530	Jau - SP	78.5	1396.0	133497.0
9	350550	Barretos - SP	50.0	1281.0	122485.0
10	351840	Guaratinguetá - SP	32.8	1250.0	118044.0
11	354140	Presidente Prudente - SP	57.3	1237.0	225668.0
12	355500	Tupã - SP	58.2	1202.0	63928.0
13	350400	Assis - SP	33.2	1200.0	101409.0
14	352220	Itapeirica da Serra - SP	44.3	979.0	158522.0
15	352390	Itu - SP	31.9	973.0	168240.0
16	350750	Botucatu - SP	64.4	824.0	145155.0
17	351110	Catanduva - SP	58.5	791.0	115791.0
18	350330	Araras - SP	36.9	715.0	130866.0
19	351640	Franco da Rocha - SP	34.5	669.0	144849.0

Evidência da célula [33] (código e saída)

[34] Junção com a regionalização do IBGE

Traz a Região Intermediária de cada município e confere que a cobertura é total.

Comandos e por que estes:

dtype={"municipio_6": str} — Sem isso o pandas lê o código como número e perde o zero à esquerda, quebrando o merge silenciosamente. É a causa mais comum de junção com resultado vazio.

Checar isna().sum() após o merge — Zero nulo prova que a chave casou em 100% das linhas, o que valida o merge antes de qualquer análise em cima dele.

```
[34]: regioao_municipios = pd.read_csv(DATA_DIR / "regiao_municipios_sp.csv", dtype={"municipio_6": str})

      tabela_municipio_ano_regiao = tabela_municipio_ano.merge(
        regioao_municipios[["municipio_6", "regiao_intermediaria"]], on="municipio_6", how="left"
      )

      print(f"Municípios sem região identificada: {tabela_municipio_ano_regiao['regiao_intermediaria'].isna().sum()} de {len(tabela_municipio_ano_regiao)} linhas.")
      print(f"Regiões intermediárias cobrindo SP: {tabela_municipio_ano_regiao['regiao_intermediaria'].nunique()}")

      Municípios sem região identificada: 0 de 1935 linhas.
      Regiões intermediárias cobrindo SP: 11
```

Evidência da célula [34] (código e saída)

[35] Métricas por Região Intermediária

Agrega as cinco métricas por região e junta a contagem de municípios de cada uma.

Comandos e por que estes:

median() na agregação — Mediana por região resiste ao município extremo dentro da região.

Trazer n_municipios — Sem o tamanho da região, uma mediana de 26 municípios pareceria tão sólida quanto uma de 100.

```
[35]: metricas_por_regiao = (
    tabela_municipio_ano_regiao.groupby("regiao_intermediaria")[metricas_distribuicao].median().round(2)
)
n_municipios_regiao = regiao_municipios.groupby("regiao_intermediaria").size().rename("n_municipios")
metricas_por_regiao = metricas_por_regiao.merge(n_municipios_regiao, on="regiao_intermediaria")
metricas_por_regiao.sort_values(["taxa_icsap_10k", ascending=False])
```

```
[35]:
```

regiao_intermediaria	taxa_internacao_uti_10k	taxa_icsap_10k	pct_icsap	custo_medio_internacao	dias_medios_uti_internacao	n_municipios
São José do Rio Preto	99.42	17.41	18.83	9054.57	5.81	100
Marília	90.77	17.40	19.91	7649.95	6.35	54
São José dos Campos	75.04	15.28	20.50	7398.70	6.78	39
Araraquara	69.44	15.23	22.17	6671.43	5.51	26
Araçatuba	70.01	14.00	22.59	7101.18	5.75	44
Presidente Prudente	63.18	12.17	20.00	7644.17	6.08	55
Bauru	73.74	12.00	16.76	7183.66	6.42	48
Ribeirão Preto	63.15	11.23	19.05	8496.09	5.96	64
Campinas	57.03	9.36	16.90	7961.66	6.46	87
São Paulo	50.68	8.48	16.34	7983.36	7.31	50
Sorocaba	48.34	7.65	16.39	7745.58	7.15	78

Evidência da célula [35] (código e saída)

[36] Dispersão interna de cada região

Calcula o IQR da taxa ICSAP dentro de cada região, para separar diferença entre regiões de heterogeneidade dentro delas.

Comandos e por que estes:

apply(lambda s: s.quantile(0.75) - s.quantile(0.25)) — O groupby não tem método de IQR pronto; o apply recebe a série de cada grupo e devolve um escalar.

Comparar com o IQR geral — O IQR geral é 11,5. Regiões abaixo disso são internamente homogêneas, o que reforça a região como recorte útil.

```
[36]: dispersao_regiao = (
    tabela_municipio_ano_regiao.groupby("regiao_intermediaria")["taxa_icsap_10k"]
    .apply(lambda s: s.quantile(0.75) - s.quantile(0.25))
    .rename("iqr_taxa_icsap_10k")
    .sort_values(ascending=False)
)

print("IQR de taxa_icsap_10k por região (dispersão interna, comparar com o IQR geral da etapa 3: 11,5):")
dispersao_regiao.round(2)

IQR de taxa_icsap_10k por região (dispersão interna, comparar com o IQR geral da etapa 3: 11,5):

[36]: regiao_intermediaria
São José do Rio Preto    17.51
Marília                  15.08
Presidente Prudente     14.84
Araçatuba                14.16
São José dos Campos      11.51
Araraquara               9.75
Bauru                    9.00
Ribeirão Preto           8.07
Campinas                 7.33
Sorocaba                 6.59
São Paulo                 4.51
Name: iqr_taxa_icsap_10k, dtype: float64
```

Evidência da célula [36] (código e saída)

Etapas 8. Síntese final da AED

[37] Consolidação da AED

Junta numa tabela única o que cada etapa produziu por métrica: mediana, IQR, percentual de outlier, correlação com capacidade, variação sazonal e amplitude geográfica.

Comandos e por que estes:

index.map(dicionario) — Traduz o nome da métrica para o valor correspondente, lidando com o fato de que a variação sazonal só existe para duas das cinco.

Por que consolidar — Coloca as três dimensões na mesma escala de comparação e mostra que a geográfica é a mais explicativa.

```
[37]: fences_df = pd.DataFrame(fences).set_index("metrica")

resumo_etapas = pd.DataFrame({
    "mediana_geral": distribuicao["50%"],
    "iqr": distribuicao["iqr"],
})

resumo_etapas["pct_outlier"] = (fences_df["outliers_total"] / len(tabela_municipio_ano) * 100).round(1)
resumo_etapas["corr_spearman_c_estabelecimento"] = correlacao_spearman.loc[
    metricas_distribuicao, "estabelecimentos_anual_medio"
].round(3)

variacao_sazonal_map = {
    "taxa_internacao_uti_10k": variacao_pct.get("taxa_internacao_uti_10k_dia"),
    "taxa_icsap_10k": variacao_pct.get("taxa_icsap_10k_dia"),
}

resumo_etapas["variacao_sazonal_pct"] = resumo_etapas.index.map(variacao_sazonal_map).round(1)

variacao_regional = metricas_por_regiao[metricas_distribuicao].max() - metricas_por_regiao[metricas_distribuicao].min()
resumo_etapas["variacao_geografica_absoluta"] = variacao_regional.round(2)

resumo_etapas.round(2)
```

	mediana_geral	iqr	pct_outlier	corr_spearman_c_estabelecimento	variacao_sazonal_pct	variacao_geografica_absoluta
taxa_internacao_uti_10k	67.28	40.73	2.1	-0.24	8.1	51.08
taxa_icsap_10k	12.05	11.47	4.4	-0.12	12.1	9.76
pct_icsap	18.32	9.39	2.1	0.02	NaN	6.25
custo_medio_internacao	7933.72	2418.08	1.3	-0.01	NaN	2383.14
dias_medios_uti_internacao	6.35	1.84	2.4	0.21	NaN	1.80

Evidência da célula [37] (código e saída)

Etapa 9. Preparação estatística

[38] Construção dos grupos de comparação

Carrega o painel com SIOPS, calcula despesa per capita e taxa ICSAP, e divide os municípios em alto e baixo investimento pela mediana.

Comandos e por que estes:

dropna(subset=["siops_despesa_paga"]) — Remove 2022, que não tem SIOPS com defasagem utilizável.

groupby("municipio_6").agg(mean) — Média de 2023 e 2024 por município: uma linha por unidade de análise, requisito do teste.

Corte pela mediana — Divide em dois grupos de tamanho praticamente igual (322 e 323), o que preserva poder estatístico. O corte foi definido antes de olhar o resultado.

```
[38]: painel = pd.read_parquet(DATA_DIR / "painel_municipio_ano_sp_2022_2024.parquet")
      regioao_teste_t = pd.read_csv(DATA_DIR / "regiao_municipios_sp.csv", dtype={"municipio_6": str})

      com_siops = painel.dropna(subset=["siops_despesa_paga"]).copy()
      com_siops["despesa_paga_per_capita"] = com_siops["siops_despesa_paga"] / com_siops["populacao"]
      com_siops["taxa_icsap_uti_10k"] = com_siops["qtd_internacoes_icsap_uti"] / com_siops["populacao"] * 10_000
      com_siops["leitos_sus_10k"] = com_siops["qtd_leitos_sus_media"] / com_siops["populacao"] * 10_000

      por_municipio = (
          com_siops.groupby("municipio_6")
          .agg(
              nome_municipio=("nome_municipio", "first"),
              despesa_paga_per_capita=("despesa_paga_per_capita", "mean"),
              taxa_icsap_uti_10k=("taxa_icsap_uti_10k", "mean"),
              populacao=("populacao", "mean"),
              leitos_sus_10k=("leitos_sus_10k", "mean"),
          )
          .reset_index()
      )

      mediana_investimento = por_municipio["despesa_paga_per_capita"].median()
      grupo_alto = por_municipio.loc[por_municipio["despesa_paga_per_capita"] > mediana_investimento].copy()
      grupo_baixo = por_municipio.loc[por_municipio["despesa_paga_per_capita"] <= mediana_investimento].copy()

      print(f"({len(por_municipio)}) municípios com SIOPS válido em pelo menos um dos dois anos (2023 ou 2024)")
      print(f"Mediana de investimento per capita: R$ {mediana_investimento:.2f}")
      print(f"Grupo ALTO investimento: {len(grupo_alto)} municípios | Grupo BAIXO investimento: {len(grupo_baixo)} municípios")

      645 municípios com SIOPS válido em pelo menos um dos dois anos (2023 ou 2024)
      Mediana de investimento per capita: R$ 1,040.81
      Grupo ALTO investimento: 322 municípios | Grupo BAIXO investimento: 323 municípios
```

Evidência da célula [38] (código e saída)

[39] Análise de distribuição dos dois grupos

Histograma sobreposto, boxplot lado a lado e medidas de forma. É o passo 9 do guia, que decide se o teste pode ser paramétrico.

Comandos e por que estes:

hist com alpha=0.6 — A transparência permite ver a sobreposição das duas distribuições.

patch_artist=True — Necessário para colorir a caixa do boxplot do matplotlib.

skew() e **kurtosis()** — Assimetria acima de 1 e curtose positiva nos dois grupos: cauda longa à direita, distribuição não normal.

```
[39]: fig, eixos = plt.subplots(1, 2, figsize=(11, 4.5))

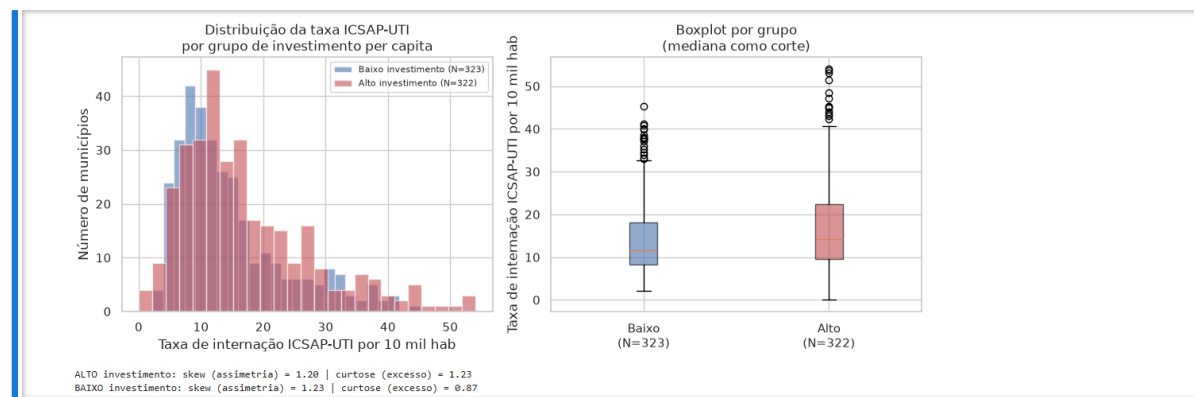
eixos[0].hist(grupo_baixo["taxa_icsap_uti_10k"], bins=25, alpha=0.6, color="#4C72B0",
              label=f"Baixo investimento (N={len(grupo_baixo)})")
eixos[0].hist(grupo_alto["taxa_icsap_uti_10k"], bins=25, alpha=0.6, color="#C44E52",
              label=f"Alto investimento (N={len(grupo_alto)})")
eixos[0].set_xlabel("Taxa de internação ICSAP-UTI por 10 mil hab")
eixos[0].set_ylabel("Número de municípios")
eixos[0].set_title("Distribuição da taxa ICSAP-UTI\ncpor grupo de investimento per capita")
eixos[0].legend(fontsize=8)

caixa = eixos[1].boxplot(
    [grupo_baixo["taxa_icsap_uti_10k"], grupo_alto["taxa_icsap_uti_10k"]],
    tick_labels=[f"Baixo\n(N={len(grupo_baixo)})", f"Alto\n(N={len(grupo_alto)})"],
    patch_artist=True,
)
for caixa_individual, cor in zip(caixa["boxes"], ["#4C72B0", "#C44E52"]):
    caixa_individual.set_facecolor(cor)
    caixa_individual.set_alpha(0.6)
eixos[1].set_ylabel("Taxa de internação ICSAP-UTI por 10 mil hab")
eixos[1].set_title("Boxplot por grupo\n(mediana como corte)")

plt.tight_layout()
plt.show()

for nome, grupo in [("ALTO investimento", grupo_alto), ("BAIXO investimento", grupo_baixo)]:
    taxa = grupo["taxa_icsap_uti_10k"]
    print(f"{nome}: skew (assimetria) = {taxa.skew():.2f} | curtose (excesso) = {taxa.kurtosis():.2f}")
```

Evidência da célula [39] (código)



Evidência da célula [39] (saída (gráficos e medidas de forma))

[40] Métricas descritivas por grupo

Tabela de describe dos dois grupos lado a lado.

Comandos e por que estes:

`pd.DataFrame({rotulo: serie.describe()})` — Monta a comparação em uma expressão, com os grupos como colunas.

Ler média contra mediana — Média 17,09 e mediana 14,28 no grupo alto: a distância entre as duas é a assimetria aparecendo em número.

```
[40]: metricas_descritivas = pd.DataFrame({
    "ALTO investimento": grupo_alto["taxa_icsap_uti_10k"].describe(),
    "BAIXO investimento": grupo_baixo["taxa_icsap_uti_10k"].describe(),
}).round(2)
metricas_descritivas
```

```
[40]:
```

	ALTO investimento	BAIXO investimento
count	322.00	323.00
mean	17.09	14.60
std	10.65	8.84
min	0.00	2.18
25%	9.63	8.28
50%	14.28	11.67
75%	22.38	18.17
max	54.07	45.24

Evidência da célula [40] (código e saída)

[41] Quantificação de missing na base do teste

Confere o nulo de SIOPS por ano e o nulo remanescente na tabela por município.

Comandos e por que estes:

`groupby("ano").apply(lambda s: s.isna().mean())` — Proporção de nulo por ano numa expressão; confirma que 2022 é o ano sem SIOPS.

Checar antes do teste — Nulo não tratado reduz a amostra e pode enviesar o resultado, então a checagem vem antes e não depois.

```
[41]: print("Linhas do painel original:", len(painel))
      print("\n% de siops_despesa_paga nulo, por ano:")
      print(painel.groupby("ano")["siops_despesa_paga"].apply(lambda s: s.isna().mean()))
      print("\nValores ausentes na tabela por_municipio (N = 645):")
      print(por_municipio.isna().sum())
```

```
Linhas do painel original: 1935
```

```
% de siops_despesa_paga nulo, por ano:
```

```
ano
```

```
2022    1.0
```

```
2023    0.0
```

```
2024    0.0
```

```
Name: siops_despesa_paga, dtype: float64
```

```
Valores ausentes na tabela por_municipio (N = 645):
```

```
municipio_6          0
```

```
nome_municipio       0
```

```
despesa_paga_per_capita  0
```

```
taxa_icsap_uti_10k     0
```

```
populacao            0
```

```
leitos_sus_10k        284
```

```
dtype: int64
```

Evidência da célula [41] (código e saída)

[42] Boxplot ajustado por medcouple

Substitui a cerca de Tukey por cercas ajustadas à assimetria, porque Tukey assume distribuição simétrica e marcaria como outlier a própria cauda esperada.

Comandos e por que estes:

`medcouple(serie)` — Medida robusta de assimetria, entre -1 e 1, que entra como expoente na fórmula das cercas.

`np.exp(-4*mc)` e `np.exp(3*mc)` — Fórmula de Hubert e Vandervieren: alarga a cerca do lado da cauda longa e estreita do outro.

Efeito prático — Com Tukey, 86 município-ano seriam outlier; com a cerca ajustada, 3,1% e 5,3% em cada grupo. A diferença é o método, não o dado.

```
[42]: from statsmodels.stats.stattools import medcouple

def fences_boxplot_ajustado(serie):
    q1, q3 = serie.quantile([0.25, 0.75])
    iqr = q3 - q1
    mc = medcouple(serie.to_numpy())
    if mc >= 0:
        inferior = q1 - 1.5 * iqr * np.exp(-4 * mc)
        superior = q3 + 1.5 * iqr * np.exp(3 * mc)
    else:
        inferior = q1 - 1.5 * iqr * np.exp(-3 * mc)
        superior = q3 + 1.5 * iqr * np.exp(4 * mc)
    return inferior, superior, mc

for nome, grupo in [("ALTO investimento", grupo_alto), ("BAIXO investimento", grupo_baixo)]:
    taxa = grupo["taxa_icsap_uti_10k"]
    inferior, superior, mc = fences_boxplot_ajustado(taxa)
    n_outlier = ((taxa < inferior) | (taxa > superior)).sum()
    print(f"{nome}: fence=[{inferior:.2f}, {superior:.2f}] | medcouple={mc:.3f} | "
          f"outliers={n_outlier} ({n_outlier/len(taxa):.1%})")

ALTO investimento: fence=[3.58, 67.78] | medcouple=0.288 | outliers=10 (3.1%)
BAIXO investimento: fence=[5.14, 65.68] | medcouple=0.388 | outliers=17 (5.3%)
```

Evidência da célula [42] (código e saída)

[43] Desenho do boxplot ajustado

Monta manualmente as estatísticas do boxplot para que o gráfico use as cercas ajustadas, e não as de Tukey que o matplotlib aplica por padrão.

Comandos e por que estes:

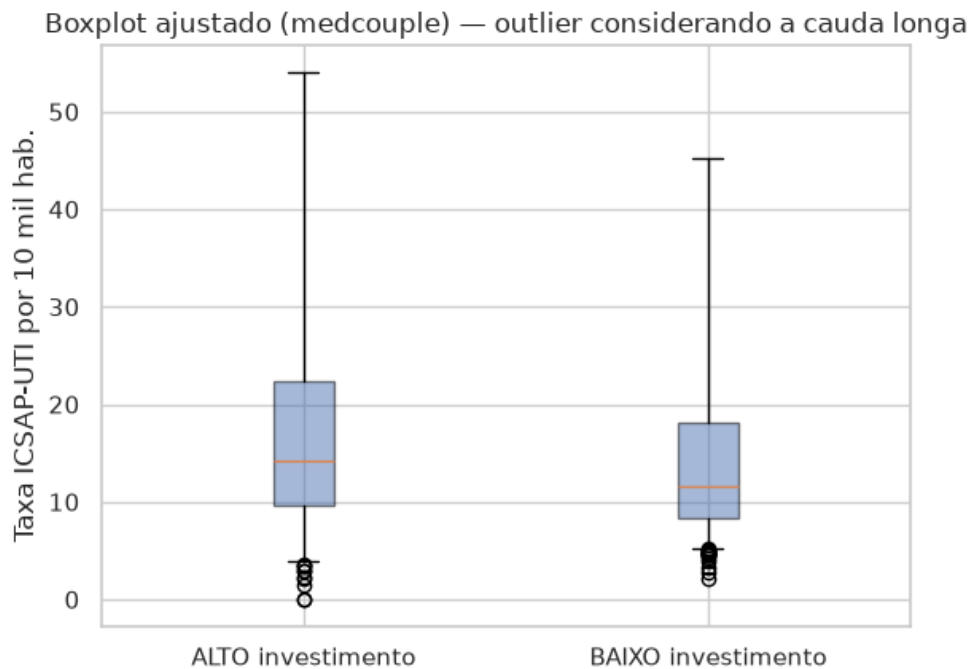
`ax.bxp(lista_de_dicionarios)` — Diferente de `ax.boxplot`, o `bxp` aceita estatísticas já calculadas; é a única forma de plotar cerca customizada.

Recalcular `whislo` e `whishi` — O bigode vai até o valor mais extremo dentro da cerca, não até a cerca em si.

```
[43]: def bxp_stats_ajustado(serie, label):
    q1, q3 = serie.quantile([0.25, 0.75])
    mediana = serie.median()
    inferior, superior, mc = fences_boxplot_ajustado(serie)
    dentro = serie[(serie >= inferior) & (serie <= superior)]
    whislo = dentro.min() if len(dentro) else inferior
    whishi = dentro.max() if len(dentro) else superior
    fliers = serie[(serie < inferior) | (serie > superior)].to_numpy()
    return {"label": label, "med": mediana, "q1": q1, "q3": q3,
            "whislo": whislo, "whishi": whishi, "fliers": fliers}

stats_ajustado = [
    bxp_stats_ajustado(grupo_alto["taxa_icsap_uti_10k"], "ALTO investimento"),
    bxp_stats_ajustado(grupo_baixo["taxa_icsap_uti_10k"], "BAIXO investimento"),
]

fig, ax = plt.subplots(figsize=(6, 4.5))
ax.bxp(stats_ajustado, showfliers=True, patch_artist=True,
        boxprops=dict(facecolor="#4C72B0", alpha=0.5))
ax.set_ylabel("Taxa ICSAP-UTI por 10 mil hab.")
ax.set_title("Boxplot ajustado (medcouple) – outlier considerando a cauda longa")
plt.tight_layout()
plt.show()
```



Evidência da célula [43] (código e saída)

[44] Matriz de correlação entre candidatas

Correlaciona investimento, população, leitos e taxa antes do teste, para procurar variável de confusão.

Comandos e por que estes:

Rodar a matriz antes do teste — É item explícito do passo 9 do guia, e não formalidade: revela correlação de -0,645 entre investimento per capita e população.

O que isso significa — Município pequeno gasta mais por habitante. Qualquer diferença entre os grupos pode ser porte disfarçado de investimento.

```
[44]: candidatas = ["despesa_paga_per_capita", "populacao", "leitos_sus_10k", "taxa_icsap_uti_10k"]
      matriz_correlacao_candidatas = por_municipio[candidatas].corr(method="spearman")
      matriz_correlacao_candidatas.round(3)
```

```
[44]:
```

	despesa_paga_per_capita	populacao	leitos_sus_10k	taxa_icsap_uti_10k
despesa_paga_per_capita	1.000	-0.645	0.056	0.125
populacao	-0.645	1.000	-0.226	-0.211
leitos_sus_10k	0.056	-0.226	1.000	0.320
taxa_icsap_uti_10k	0.125	-0.211	0.320	1.000

Evidência da célula [44] (código e saída)

[45] Investimento e taxa por região

Cruza a divisão de investimento com a regionalização, para ver se o padrão se mantém no recorte que a AED apontou como mais explicativo.

Comandos e por que estes:

agg com três funções nomeadas — count para tamanho da região, median para as duas métricas, tudo numa passada.

```
[45]: regioao_slim = regioao_teste_t[["municipio_6", "regiao_intermediaria"]].drop_duplicates()
      com_regiao = por_municipio.merge(regiao_slim, on="municipio_6", how="left")

      tabela_regiao_investimento = (
          com_regiao.groupby("regiao_intermediaria")
              .agg(
                  n_municipios=("municipio_6", "count"),
                  mediana_investimento_per_capita=("despesa_paga_per_capita", "median"),
                  mediana_taxa_icsap_10k=("taxa_icsap_uti_10k", "median"),
              )
              .sort_values("mediana_taxa_icsap_10k")
      )
      tabela_regiao_investimento.round(2)
```

```
[45]:
```

	n_municipios	mediana_investimento_per_capita	mediana_taxa_icsap_10k
regiao_intermediaria			
São Paulo	50	733.54	8.76
Sorocaba	78	1009.31	8.94
Campinas	87	914.14	9.94
Presidente Prudente	55	1059.40	12.02
Ribeirão Preto	64	966.27	12.37
Bauru	48	1116.51	12.41
Araçatuba	44	1368.66	14.72
Araraquara	26	987.95	15.03
São José dos Campos	39	926.89	17.11
São José do Rio Preto	100	1346.20	18.94
Marília	54	1112.05	20.36

Evidência da célula [45] (código e saída)

[46] Decomposição entre municípios e dentro do município

Separa a correlação em duas: entre municípios (comparando médias) e dentro do mesmo município ao longo do tempo (subtraindo a média de cada um).

Comandos e por que estes:

`groupby(...).transform("mean")` — Diferente do `agg`, o `transform` devolve o resultado alinhado com as linhas originais, o que permite subtrair a média do próprio município de cada linha.

Centrar dentro do grupo — Remove tudo que é característica fixa do município (porte, região, perfil), deixando só a variação temporal.

O resultado — A correlação cai de 0,125 para 0,034. O sinal é quase todo entre municípios, ou seja, é o confundidor por porte e não efeito do investimento.

```
[46]: base_decomposicao = painel.dropna(subset=["siops_despesa_paga", "qtd_internacoes_icsap_uti", "populacao"]).copy()
base_decomposicao["despesa_paga_per_capita"] = base_decomposicao["siops_despesa_paga"] / base_decomposicao["populacao"]
base_decomposicao["taxa_icsap_uti_10k"] = base_decomposicao["qtd_internacoes_icsap_uti"] / base_decomposicao["populacao"] * 10000

medias_municipio = base_decomposicao.groupby("municipio_6")[["despesa_paga_per_capita", "taxa_icsap_uti_10k"]].mean()
corr_entre = medias_municipio["despesa_paga_per_capita"].corr(medias_municipio["taxa_icsap_uti_10k"], method="spearman")

medias_por_linha = base_decomposicao.groupby("municipio_6")[["despesa_paga_per_capita", "taxa_icsap_uti_10k"]].transform("mean")
centrado_dentro = base_decomposicao[["despesa_paga_per_capita", "taxa_icsap_uti_10k"]] - medias_por_linha
corr_dentro = centrado_dentro["despesa_paga_per_capita"].corr(centrado_dentro["taxa_icsap_uti_10k"], method="spearman")

print(f"N (municípios, médias 2023+2024): {len(medias_municipio)}")
print(f"Correlação ENTRE municípios (investimento e taxa per capita, médias por município): {corr_entre:.3f}")
print(f"Correlação DENTRO do município (médias subtraídas): {corr_dentro:.3f}")

N (municípios, médias 2023+2024): 645
Correlação ENTRE municípios (investimento e taxa per capita, médias por município): 0.125
Correlação DENTRO do município (médias subtraídas): 0.034
```

Evidência da célula [46] (código e saída)

Etapa 10. Teste de hipótese

[47] Teste de normalidade

Aplica Shapiro-Wilk nos dois grupos, formalizando o que a etapa 9.1 já mostrava no gráfico.

Comandos e por que estes:

shapiro(serie) — Teste com bom poder para amostras deste tamanho. p da ordem de $1e-13$ e $1e-15$: normalidade rejeitada com folga.

Por que isso decide o teste — Sem normalidade e com N por grupo em torno de 322, o caminho é o teste não paramétrico.

```
[47]: from scipy.stats import shapiro, mannwhitneyu

taxa_alto = grupo_alto["taxa_icsap_uti_10k"]
taxa_baixo = grupo_baixo["taxa_icsap_uti_10k"]

_, p_shapiro_alto = shapiro(taxa_alto)
_, p_shapiro_baixo = shapiro(taxa_baixo)

print(f"Shapiro-Wilk grupo alto investimento: p = {p_shapiro_alto:.2e}")
print(f"Shapiro-Wilk grupo baixo investimento: p = {p_shapiro_baixo:.2e}")

Shapiro-Wilk grupo alto investimento: p = 2.15e-13
Shapiro-Wilk grupo baixo investimento: p = 1.90e-15
```

Evidência da célula [47] (código e saída)

[48] Teste de Mann-Whitney, hipótese pré-registrada

Executa o teste principal com a hipótese definida antes de olhar o dado: o grupo de alto investimento tem taxa menor.

Comandos e por que estes:

`mannwhitneyu(alternative="less")` — Teste não paramétrico para duas amostras independentes; compara postos e não médias, dispensando normalidade.

Teste unilateral — A hipótese do projeto tem direção. Unilateral concentra o poder estatístico nessa direção.

Resultado — $p = 0,9994$. A hipótese original não se confirma.

```
[48]: estatistica_u, p_principal = mannwhitneyu(taxa_alto, taxa_baixo, alternative="less")

print("Teste principal (pré-registrado): H1 = grupo alto tem taxa MENOR que grupo baixo")
print(f"estatística U de Mann-Whitney = {estatistica_u:.1f}")
print(f"p-valor = {p_principal:.4f}")

Teste principal (pré-registrado): H1 = grupo alto tem taxa MENOR que grupo baixo
estatística U de Mann-Whitney = 59710.0
p-valor = 0.9994
```

Evidência da célula [48] (código e saída)

[49] Teste na direção oposta, marcado como exploratório

Roda a hipótese contrária e rotula explicitamente como pós-hoc.

Comandos e por que estes:

`alternative="greater"` — Mesmo teste, direção invertida.

Rotular como exploratório — Hipótese formulada depois de ver o dado não tem a mesma força da pré-registrada. Registrar a diferença é o que separa achado de garimpo de p-valor.

```
[49]: _, p_exploratorio = mannwhitneyu(taxa_alto, taxa_baixo, alternative="greater")

print("[EXPLORATÓRIO, pós-hoc, não é o teste principal]")
print("H1 exploratória: grupo alto tem taxa MAIOR que grupo baixo")
print(f"p-valor = {p_exploratorio:.4f}")

[EXPLORATÓRIO, pós-hoc, não é o teste principal]
H1 exploratória: grupo alto tem taxa MAIOR que grupo baixo
p-valor = 0.0006
```

O resultado na direção oposta é altamente significativo ($p = 0,0006$).

Evidência da célula [49] (código e saída)

[50] Análise de sensibilidade do ponto de corte

Repete o teste com três cortes diferentes (mediana, tercil e quartil extremo) para checar se o resultado depende da escolha do corte.

Comandos e por que estes:

Função `testar_corte` — Encapsula o teste para aplicar a mesma receita aos três cortes sem repetir código.

`quantile([1/3, 2/3])` e `quantile([0.25, 0.75])` — Tercis e quartis; o corte extremo compara os 25% que mais gastam com os 25% que menos gastam.

Resultado — p entre 0,0036 e 0,0078 na direção exploratória nos três cortes. O achado não é artefato do corte.

```
[50]: def testar_corte(alto, baixo):
_, p_principal = mannwhitneyu(alto, baixo, alternative="less")
_, p_exploratorio = mannwhitneyu(alto, baixo, alternative="greater")
return {
    "N cada grupo": f"{len(alto)}/{len(baixo)}",
    "mediana taxa baixo": round(baixo.median(), 2),
    "mediana taxa alto": round(alto.median(), 2),
    "p principal (alto < baixo)": round(p_principal, 4),
    "p exploratório (alto > baixo)": round(p_exploratorio, 4),
}

investimento = por_municipio["despesa_paga_per_capita"]
taxa = por_municipio["taxa_icsap_uti_10k"]

resultados_sensibilidade = {
    "Mediana (PRINCIPAL, pré-registrado)": testar_corte(taxa_alto, taxa_baixo),
}

t1, t2 = investimento.quantile([1 / 3, 2 / 3])
resultados_sensibilidade["Tercil (sensibilidade)"] = testar_corte(
    taxa[investimento >= t2], taxa[investimento <= t1]
)

q1, q3 = investimento.quantile([0.25, 0.75])
resultados_sensibilidade["Quartil/extremos (sensibilidade)"] = testar_corte(
    taxa[investimento >= q3], taxa[investimento <= q1]
)

pd.DataFrame(resultados_sensibilidade).T
```

```
[50]:
```

	N cada grupo	mediana taxa baixo	mediana taxa alto	p principal (alto < baixo)	p exploratório (alto > baixo)
Mediana (PRINCIPAL, pré-registrado)	322/323	11.67	14.28	0.9994	0.0006
Tercil (sensibilidade)	215/215	12.4	14.83	0.9922	0.0078
Quartil/extremos (sensibilidade)	162/162	11.73	15.13	0.9964	0.0036

Evidência da célula [50] (código e saída)

[51] Teste t de Welch como checagem

Roda o teste paramétrico mesmo sem ser exigido, entrega o t-statistic do checklist e compara os dois testes na mesma hipótese.

Comandos e por que estes:

levene(a, b) — Testa igualdade de variância, o pressuposto que separa Student de Welch. $p = 0,0169$ rejeita, então Welch é a versão correta.

ttest_ind(equal_var=False) — Welch: não assume variância igual e ajusta os graus de liberdade (621,3 em vez de 643).

d de Cohen calculado à mão — O scipy não devolve tamanho de efeito. $d = 0,254$, efeito pequeno, coerente com o $r = 0,13$ do Mann-Whitney.

Por que comparar os dois — Os p-valores batem até a terceira casa. Isso mostra que a conclusão vem do dado e não da escolha do teste.

```
[51]: from scipy.stats import ttest_ind, levene

# Pressuposto que separa o t de Student do t de Welch: as duas variancias sao parecidas?
estatistica levene, p levene = levene(taxa_alto, taxa_baixo)
print("Teste de Levene (as variancias dos dois grupos sao iguais?)")
print(f" estatistica = {estatistica levene:.4f} | p-valor = {p levene:.4f}")
print(f" desvio-padrao ALTO investimento = {taxa_alto.std():.2f}")
print(f" desvio-padrao BAIXO investimento = {taxa_baixo.std():.2f}")
print()

# t de Welch: mesma H1 pre-registrada do teste principal, mais a exploratoria
t_welch_less = ttest_ind(taxa_alto, taxa_baixo, equal_var=False, alternative="less")
t_welch_greater = ttest_ind(taxa_alto, taxa_baixo, equal_var=False, alternative="greater")

print("Teste t de Welch (nao assume variancia igual)")
print(f" t-statistic = {t_welch_less.statistic:.4f} | graus de liberdade = {t_welch_less.df:.1f}")
print(f" p-valor H1 PRE-REGISTRADA (alto < baixo) = {t_welch_less.pvalue:.4f}")
print(f" p-valor H1 EXPLORATORIA (alto > baixo) = {t_welch_greater.pvalue:.4f}")
print()

# Lado a lado com o Mann-Whitney, que continua sendo o teste principal
_, p_mu_less = mannwhitneyu(taxa_alto, taxa_baixo, alternative="less")
_, p_mu_greater = mannwhitneyu(taxa_alto, taxa_baixo, alternative="greater")
print("Comparacao dos dois testes, mesma hipotese:")
print(f"{'':26s} {'H1 pre-registrada':>18s} {'H1 exploratoria':>17s}")
print(f"{'Mann-Whitney (principal)':26s} {p_mu_less:18.4f} {p_mu_greater:17.4f}")
print(f"{'t de Welch (checagem)':26s} {t_welch_less.pvalue:18.4f} {t_welch_greater.pvalue:17.4f}")
print()

# Tamanho de efeito em desvios-padrao, para comparar com o r = 0,13 do Mann-Whitney
n_alto, n_baixo = len(taxa_alto), len(taxa_baixo)
desvio_combinado = np.sqrt(
    ((n_alto - 1) * taxa_alto.var() + (n_baixo - 1) * taxa_baixo.var()) / (n_alto + n_baixo - 2)
)
d_cohen = (taxa_alto.mean() - taxa_baixo.mean()) / desvio_combinado
print(f"Media ALTO = {taxa_alto.mean():.2f} | Media BAIXO = {taxa_baixo.mean():.2f} internacoes/10 mil hab")
print(f"d de Cohen = {d_cohen:.3f}")
```

Evidência da célula [51] (código)

```
Teste de Levene (as variancias dos dois grupos sao iguais?)
estatistica = 5.7383 | p-valor = 0.0169
desvio-padrao ALTO investimento = 10.65
desvio-padrao BAIXO investimento = 8.84

Teste t de Welch (nao assume variancia igual)
t-statistic = 3.2287 | graus de liberdade = 621.3
p-valor H1 PRE-REGISTRADA (alto < baixo) = 0.9993
p-valor H1 EXPLORATORIA (alto > baixo) = 0.0007

Comparacao dos dois testes, mesma hipotese:

                                H1 pre-registrada   H1 exploratoria
Mann-Whitney (principal)                0.9994                0.0006
t de Welch (checagem)                   0.9993                0.0007

Media ALTO = 17.09 | Media BAIXO = 14.60 internacoes/10 mil hab
d de Cohen = 0.254
```

Evidência da célula [51] (saída)

Etapa 11. Insights e indicadores

[52] Base a nível de paciente e viés do sobrevivente

Junta perfil de paciente, despesa municipal e tempo de viagem, e compara desfechos entre quem mora perto e longe do polo.

Comandos e por que estes:

merge encadeado com how="inner" — Inner porque só interessa o paciente com as três informações disponíveis.

tempo_viagem_minutos > 60 — Define o grupo tratado por tempo real de estrada, não por distância em linha reta.

O que o resultado expõe — Quem mora longe aparece com MENOS reinternação e MENOS tempo de UTI. A base só contém quem chegou ao hospital, então o resultado é viés de seleção, não benefício.

```
[52]: import pandas as pd

perfil = pd.read_parquet(DATA_DIR / "perfil_paciente_sp_2022_2024.parquet")
painel_desp = pd.read_parquet(DATA_DIR / "painel_municipio_ano_sp_2022_2024.parquet")
rotas = pd.read_csv(DATA_DIR / "rotas_polos_sp_real.csv", dtype={"municipio_6": str})

# So anos com SIOPS valido (2022 fica de fora, ver etapa 9.3), mesma janela
# ja usada na Preparacao Estatistica
painel_slim = painel_desp.dropna(subset=["siops_despesa_paga"]).copy()
painel_slim["despesa_paga_per_capita"] = painel_slim["siops_despesa_paga"] / painel_slim["populacao"]
painel_slim = painel_slim[["municipio_6", "ano", "despesa_paga_per_capita"]]

df_paciente = perfil.merge(painel_slim, on=["municipio_6", "ano"], how="inner")
df_paciente = df_paciente.merge(rotas[["municipio_6", "tempo_viagem_minutos"]], on="municipio_6", how="inner")
df_paciente = df_paciente.dropna(subset=["reinternacao_30d"])
df_paciente["tratado"] = df_paciente["tempo_viagem_minutos"] > 60

print(f"N total: {len(df_paciente):,}")
print(f"Tratado (tempo de viagem > 60min): {df_paciente['tratado'].sum():,} ({df_paciente['tratado'].mean()*100:.1f}%)")
print()
resumo_paciente = df_paciente.groupby("tratado").agg(
    reinternacao_30d=("reinternacao_30d", "mean"),
    uti_mes_to=("uti_mes_to", "mean"),
    dias_perm=("dias_perm", "mean"),
    val_tot=("val_tot", "mean"),
)
resumo_paciente.round(4)
```

```
N total: 535,514
Tratado (tempo de viagem > 60min): 110,619 (20.7%)
```

```
[52]:
```

	reinternacao_30d	uti_mes_to	dias_perm	val_tot
tratado				
False	0.1514	6.7349	11.9995	8092.8315
True	0.1371	6.0265	10.2456	7294.1722

Evidência da célula [52] (código e saída)

[53] Gradiente de taxa por distância ao polo

Volta ao nível de município, onde não existe o condicionamento da célula anterior, e correlaciona distância ao polo com taxa ICSAP.

Comandos e por que estes:

spearmanr(x, y) — Correlação por postos, coerente com o resto do notebook.

pd.qcut(serie, 5) — Cinco faixas de mesmo tamanho, para ver se a relação é monotônica em vez de confiar só no coeficiente.

Resultado — rho = 0,1119 com p = 0,0045, e mediana subindo de 11,06 para 15,35 da faixa mais perto para a mais longe.

```

3]: import pandas as pd
from scipy.stats import spearmanr

painel = pd.read_parquet(DATA_DIR / "painel_municipio_ano_sp_2022_2024.parquet")
rotas = pd.read_csv(DATA_DIR / "rotas_polos_sp_real.csv", dtype={"municipio_6": str})

painel_23_24 = painel[painel["ano"].isin([2023, 2024]).copy()]
painel_23_24["taxa_icsap_uti_10k"] = painel_23_24["qtd_internacoes_icsap_uti"] / painel_23_24["populacao"] * 10000

por_municipio_dist = painel_23_24.groupby("municipio_6")["taxa_icsap_uti_10k"].mean().reset_index()
df_dist = por_municipio_dist.merge(
    rotas[["municipio_6", "distancia_polo_km"]], on="municipio_6", how="inner"
).dropna()

rho, p_rho = spearmanr(df_dist["distancia_polo_km"], df_dist["taxa_icsap_uti_10k"])
print(f"N={len(df_dist)} municípios | Spearman rho={rho:.4f}, p={p_rho:.4f}")

df_dist["faixa_distancia"] = pd.qcut(df_dist["distancia_polo_km"], 5)
print("\nMediana de taxa_icsap_uti_10k por faixa de distância ao polo:")
df_dist.groupby("faixa_distancia", observed=True)["taxa_icsap_uti_10k"].median().round(2)

N=645 municípios | Spearman rho=0.1119, p=0.0045

Mediana de taxa_icsap_uti_10k por faixa de distância ao polo:
3]: faixa_distancia
(-0.001, 23.84]      11.06
(23.84, 40.672]     11.79
(40.672, 58.052]    12.73
(58.052, 86.944]    13.70
(86.944, 204.7]     15.35
Name: taxa_icsap_uti_10k, dtype: float64

```

Evidência da célula [53] (código e saída)

[54] Pareamento por escore de propensão

Constrói pares de municípios semelhantes em porte e investimento, diferindo só na distância ao polo, para isolar o efeito da geografia.

Comandos e por que estes:

LogisticRegression para o escore — O escore de propensão é a probabilidade prevista de ser tratado dadas as covariáveis.

logit do escore — Pareamento em logit é a prática padrão: aproxima a distribuição da normal e melhora o comportamento nas pontas.

ckDTree(...).query(k=1) — Estrutura de busca por vizinho mais próximo; encontra o par de cada tratado em tempo logarítmico em vez de comparar todos contra todos.

caliper = 0,2 x desvio-padrão do logit — Limite máximo de distância aceitável para um par; regra usual da literatura de PSM.

SMD antes e depois — Diferença padronizada de médias mede se o pareamento realmente equilibrou as covariáveis. log_pop vai de -0,583 para -0,026.

```
[54]: import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from scipy.spatial import cKDTree
from scipy.stats import ttest_rel, wilcoxon

painel = pd.read_parquet(DATA_DIR / "painel_municipio_ano_sp_2022_2024.parquet")
rotas = pd.read_csv(DATA_DIR / "rotas_polos_sp_real.csv", dtype={"municipio_6": str})

# 2022 nao tem SIOPS (sem defasagem pra explicar aquele ano, ver etapa 9.3) -> so 2023+2024
painel_23_24 = painel[painel["ano"].isin([2023, 2024])].copy()
painel_23_24["despesa_paga_per_capita"] = painel_23_24["siops_despesa_paga"] / painel_23_24["populacao"]
painel_23_24["taxa_icsap_uti_10k"] = painel_23_24["qtd_internacoes_icsap_uti"] / painel_23_24["populacao"] * 10000

por_municipio = painel_23_24.groupby("municipio_6").agg(
    despesa_paga_per_capita=("despesa_paga_per_capita", "mean"),
    taxa_icsap_uti_10k=("taxa_icsap_uti_10k", "mean"),
    populacao=("populacao", "mean"),
).reset_index()

df_psm = por_municipio.merge(rotas[["municipio_6", "tempo_viagem_minutos"]], on="municipio_6", how="inner")
df_psm = df_psm.dropna(subset=["despesa_paga_per_capita", "taxa_icsap_uti_10k", "populacao", "tempo_viagem_minutos"])
df_psm["log_pop"] = np.log(df_psm["populacao"])
df_psm["tratado"] = (df_psm["tempo_viagem_minutos"] > 60).astype(int)

print(f"N municípios: {len(df_psm)} | Tratados (>60min): {df_psm['tratado'].sum()} | "
      f"Controles: {(df_psm['tratado']==0).sum()}")
print("\nMédias brutas (sem pareamento) de taxa_icsap_uti_10k:")
print(df_psm.groupby("tratado")["taxa_icsap_uti_10k"].mean().round(3))

X = df_psm[["log_pop", "despesa_paga_per_capita"]].values
y = df_psm["tratado"].values
logit = LogisticRegression(max_iter=1000)
logit.fit(X, y)
df_psm["ps"] = logit.predict_proba(X)[:, 1]
df_psm["logit_ps"] = np.log(df_psm["ps"] / (1 - df_psm["ps"]))

caliper = 0.2 * df_psm["logit_ps"].std()
print(f"\nCaliper (0,2 x desvio-padrão do logit do escore): {caliper:.3f}")

tratados = df_psm[df_psm["tratado"] == 1].reset_index(drop=True)
controles = df_psm[df_psm["tratado"] == 0].reset_index(drop=True)

# Pareamento 1:1 vizinho mais proximo em logit(escore), COM reposicao
# (controle pode servir de par pra mais de um tratado)
arvore = cKDTree(controles[["logit_ps"]].values)
```

Evidência da célula [54] (código, parte 1 de 2)

```

distancias, indices = arvore.query(tratados[["logit_ps"]].values, k=1)
dentro_caliper = distancias <= caliper
print(f"Tratados pareados dentro do caliper: {dentro_caliper.sum()} de {len(tratados)}")

trat_pareado = tratados[dentro_caliper].reset_index(drop=True)
ctrl_pareado = controles.iloc[indices[dentro_caliper]].reset_index(drop=True)

def smd(a, b):
    return (a.mean() - b.mean()) / np.sqrt((a.var() + b.var()) / 2)

print(f"\nSMD log_pop: antes={smd(tratados['log_pop'], controles['log_pop']):.3f} -> "
      f"depois={smd(trat_pareado['log_pop'], ctrl_pareado['log_pop']):.3f}")
print(f"SMD despesa_paga_per_capita: antes={smd(tratados['despesa_paga_per_capita'], controles['despesa_paga_per_capita']):.3f} -> "
      f"depois={smd(trat_pareado['despesa_paga_per_capita'], ctrl_pareado['despesa_paga_per_capita']):.3f}")

att = (trat_pareado["taxa_icsap_uti_10k"] - ctrl_pareado["taxa_icsap_uti_10k"]).mean()
print(f"\nATT (tratado pareado - controle pareado): {att:.3f}")
print(f"Média tratado pareado: {trat_pareado['taxa_icsap_uti_10k'].mean():.3f} | "
      f"Média controle pareado: {ctrl_pareado['taxa_icsap_uti_10k'].mean():.3f}")

t_stat, p_t = ttest_rel(trat_pareado["taxa_icsap_uti_10k"], ctrl_pareado["taxa_icsap_uti_10k"])
w_stat, p_w = wilcoxon(trat_pareado["taxa_icsap_uti_10k"], ctrl_pareado["taxa_icsap_uti_10k"])
print(f"\nTeste t pareado: t={t_stat:.3f}, p={p_t:.4f}")
print(f"Wilcoxon (signed-rank): W={w_stat:.3f}, p={p_w:.4f}")

N municípios: 645 | Tratados (>60min): 308 | Controles: 337

Médias brutas (sem pareamento) de taxa_icsap_uti_10k:
tratado
0    15.647
1    16.061
Name: taxa_icsap_uti_10k, dtype: float64

Caliper (0,2 x desvio-padrão do logit do escore): 0.124
Tratados pareados dentro do caliper: 308 de 308

SMD log_pop: antes=-0.583 -> depois=-0.026
SMD despesa_paga_per_capita: antes=0.260 -> depois=0.161

ATT (tratado pareado - controle pareado): 0.453
Média tratado pareado: 16.061 | Média controle pareado: 15.609

Teste t pareado: t=0.597, p=0.5513
Wilcoxon (signed-rank): W=22333.000, p=0.3506

```

Evidência da célula [54] (código, parte 2 de 2, e saída)

[55] Pareamento sem reposição, como robustez

Repete o pareamento impedindo que o mesmo controle seja usado duas vezes, e compara o resultado com a versão com reposição.

Comandos e por que estes:

`usados = set()` e `dif.loc[list(usados)] = np.inf` — Marca como infinitamente distante o controle já usado, forçando o algoritmo a escolher outro.

Por que rodar as duas — Robustez. O ATT muda de +0,453 para -0,806, o que indica sobreposição rasa entre os grupos: 308 tratados para 337 controles.

```
[55]: # Comparacao: pareamento COM reposicao (o que usamos) vs SEM reposicao
usados = set()
pares_trat, pares_ctrl = [], []
for _, row in tratados.iterrows():
    dif = (controles["logit_ps"] - row["logit_ps"]).abs()
    dif.loc[list(usados)] = np.inf
    j = dif.idxmin()
    if dif[j] <= caliper:
        pares_trat.append(row["taxa_icsap_uti_10k"])
        pares_ctrl.append(controles.loc[j, "taxa_icsap_uti_10k"])
        usados.add(j)
att_sem = (np.array(pares_trat) - np.array(pares_ctrl)).mean()

print(f"COM reposicao (usado no resultado principal): ATT={att:+.3f} | pares: {dentro_caliper.sum()} de {len(tratados)}")
print(f"SEM reposicao: ATT={att_sem:+.3f} | pares: {len(pares_trat)} de {len(tratados)}")

COM reposicao (usado no resultado principal): ATT=+0.453 | pares: 308 de 308
SEM reposicao: ATT=-0.806 | pares: 256 de 308
```

Evidência da célula [55] (código e saída)

[56] Modelo de reinternação em 30 dias

Treina um classificador para prever reinternação, com split temporal (2023 treina, 2024 testa), e avalia também na subpopulação onde reinternar é possível.

Comandos e por que estes:

`pd.get_dummies(drop_first=True)` — Converte categórica em binárias, descartando uma categoria por variável para evitar colinearidade perfeita.

`X_tr.align(X_te, join="outer", fill_value=0)` — Garante as mesmas colunas nos dois conjuntos. Categoria presente só em um ano geraria coluna fantasma e quebraria a previsão.

`scale_pos_weight` — Compensa o desbalanceamento (5,76 negativos por positivo) sem descartar linha.

Split temporal em vez de aleatório — O uso real do modelo é prever o ano seguinte; split aleatório inflaria a métrica.

```
[56]: import pandas as pd
import xgboost as xgb
from sklearn.metrics import roc_auc_score
!pip install scikit-learn statsmodels seaborn xgboost shap

perfil = pd.read_parquet(DATA_DIR / "perfil_paciente_sp_2022_2024.parquet")
painel_desp = pd.read_parquet(DATA_DIR / "painel_municipio_ano_sp_2022_2024.parquet")
rotas = pd.read_csv(DATA_DIR / "rotas_polos_sp_real.csv", dtype={"municipio_6": str})

painel_slim = painel_desp.dropna(subset=["siops_despesa_paga"]).copy()
painel_slim["despesa_paga_per_capita"] = painel_slim["siops_despesa_paga"] / painel_slim["populacao"]
painel_slim = painel_slim[["municipio_6", "ano", "despesa_paga_per_capita"]]

df_ml = perfil.merge(painel_slim, on=["municipio_6", "ano"], how="inner")
df_ml = df_ml.merge(rotas[["municipio_6", "tempo_viagem_minutos"]], on="municipio_6", how="inner")
df_ml["grupo_csap_nome"] = df_ml["grupo_csap_nome"].fillna("Nao_ICSAP")

features_cat = ["idade_faixa", "sexo", "grupo_csap_nome", "dia_semana", "feriado"]
features_num = ["val_tot", "dias_perm", "despesa_paga_per_capita", "tempo_viagem_minutos"]
target = "reinternacao_30d"
df_ml = df_ml.dropna(subset=[target])

def treinar_avaliar(df_treino, df_teste):
    X_tr = pd.get_dummies(df_treino[features_cat + features_num], columns=features_cat, drop_first=True)
    X_te = pd.get_dummies(df_teste[features_cat + features_num], columns=features_cat, drop_first=True)
    X_tr, X_te = X_tr.align(X_te, join="outer", axis=1, fill_value=0)
    y_tr = df_treino[target].astype(int)
    y_te = df_teste[target].astype(int)
    peso = (len(y_tr) - y_tr.sum()) / y_tr.sum()
    modelo = xgb.XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1,
                               scale_pos_weight=peso, random_state=42, n_jobs=-1)

    modelo.fit(X_tr, y_tr)
    proba = modelo.predict_proba(X_te[:, 1])
    return roc_auc_score(y_te, proba), len(y_tr), len(y_te)

mask_treino = df_ml["ano"] == 2023
mask_teste = df_ml["ano"] == 2024

auc_full, n_tr, n_te = treinar_avaliar(df_ml[mask_treino], df_ml[mask_teste])
print(f"ROC-AUC (treino e teste na população inteira): {auc_full:.3f} (N_treino={n_tr:,}, N_teste={n_te:,})")

# Boa parte da base nunca pode reinternar por definicao (so 1 internacao no
# periodo todo) -> retreina e reavalua so na populacao onde reinternar e
# fisicamente possivel
```

Evidência da célula [56] (código, parte 1 de 2)

```
df_sub = df_ml[df_ml["qtd_internacoes_paciente_periodo"] > 1]
auc_sub, n_tr_sub, n_te_sub = treinar_avaliar(df_sub[df_sub["ano"] == 2023], df_sub[df_sub["ano"] == 2024])
print(f"ROC-AUC (treino e teste só na população com >1 internação no período): "
      f"{auc_sub:.3f} (N_treino={n_tr_sub:,}, N_teste={n_te_sub:,})")

# Estatística de colisão de chave calculada no perfil ORIGINAL (nao no
# recorte 2023/2024 que entra no modelo), pra nao subestimar por causa do
# filtro de ano/municipio feito acima
print(f"nchave_colisao_suspeita (base completa, todos os anos): "
      f"{perfil['chave_colisao_suspeita'].sum():,} de {len(perfil):,} "
      f"({perfil['chave_colisao_suspeita'].mean()*100:.2f}%)")
```

Evidência da célula [56] (código, parte 2 de 2)

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: scikit-learn in /home/jupyter-vitor/.local/lib/python3.12/site-packages (1.9.0)
Requirement already satisfied: statsmodels in /home/jupyter-vitor/.local/lib/python3.12/site-packages (0.14.6)
Requirement already satisfied: seaborn in /home/jupyter-vitor/.local/lib/python3.12/site-packages (0.13.2)
Requirement already satisfied: xgboost in /home/jupyter-vitor/.local/lib/python3.12/site-packages (3.4.1)
Requirement already satisfied: shap in /home/jupyter-vitor/.local/lib/python3.12/site-packages (0.52.0)
Requirement already satisfied: numpy>=1.24.1 in /opt/tljh/user/lib/python3.12/site-packages (from scikit-learn) (2.5.2)
Requirement already satisfied: scipy>=1.10.0 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from scikit-learn) (1.18.0)
Requirement already satisfied: joblib>=1.4.0 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: numba>=2.0.1 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from statsmodels) (2.25.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from statsmodels) (3.6.0)
Requirement already satisfied: pandas>=2.1.0, <=1.4 in /opt/tljh/user/lib/python3.12/site-packages (from statsmodels) (2.3.3)
Requirement already satisfied: patsy>=0.5.6 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from statsmodels) (1.0.2)
Requirement already satisfied: packaging>=21.3 in /opt/tljh/user/lib/python3.12/site-packages (from statsmodels) (24.1)
Requirement already satisfied: matplotlib>=3.6.1, <=3.4 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from seaborn) (3.11.1)
Requirement already satisfied: nvidia-nccl-cu13 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from xgboost) (2.31.2)
Requirement already satisfied: tqdm>=4.27.0 in /opt/tljh/user/lib/python3.12/site-packages (from shap) (4.70.0)
Requirement already satisfied: slicer==0.0.8 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from shap) (0.67.0)
Requirement already satisfied: llvmlite in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from shap) (0.49.0)
Requirement already satisfied: cloudpickle in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from shap) (3.1.2)
Requirement already satisfied: contourpy>=1.0.1 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from matplotlib>=3.6.1, <=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from matplotlib>=3.6.1, <=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from matplotlib>=3.6.1, <=3.4->seaborn) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from matplotlib>=3.6.1, <=3.4->seaborn) (1.5.0)
Requirement already satisfied: pillow>=9 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from matplotlib>=3.6.1, <=3.4->seaborn) (12.3.0)
Requirement already satisfied: pyparsing>=3 in /home/jupyter-vitor/.local/lib/python3.12/site-packages (from matplotlib>=3.6.1, <=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /opt/tljh/user/lib/python3.12/site-packages (from matplotlib>=3.6.1, <=3.4->seaborn) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /opt/tljh/user/lib/python3.12/site-packages (from pandas>=2.1.0, <=1.4->statsmodels) (2020.3.post1)
Requirement already satisfied: tzdata>=2022.7 in /opt/tljh/user/lib/python3.12/site-packages (from pandas>=2.1.0, <=1.4->statsmodels) (2026.3)
Requirement already satisfied: six>=1.5 in /opt/tljh/user/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib>=3.6.1, <=3.4->seaborn) (1.17.0)
ROC-AUC (treino e teste na população inteira): 0.664 (N_treino=267,940, N_teste=267,574)
ROC-AUC (treino e teste só na população com >1 internação no período): 0.619 (N_treino=95,854, N_teste=91,485)

chave_colisao_suspeita (base completa, todos os anos): 3,059 de 813,422 (0.38%)
```

Evidência da célula [56] (saída)

[57] Efeito esponja: pacientes de fora de SP

Mede quanto o estado gasta com internação de residente de outras UFs e compara custo médio e perfil de gravidade.

Comandos e por que estes:

Filtrar pelos dois lados da flag — Separa as duas populações para comparar em vez de só descrever.

Razão de custo entre os grupos — Expressa a diferença em múltiplo (1,41x), mais legível que dois valores absolutos.

Resultado — 1% das internações, 1,43% do gasto, e só 7,1% de ICSAP contra 18,8%. Quem vem de fora chega mais grave, não por condição evitável.

```
[57]: import pandas as pd

dataset_unificado = pd.read_parquet(DATA_DIR / "dataset_unificado_sp_2022_2024.parquet")

fora = dataset_unificado[dataset_unificado["reside_fora_sp"]]
sp = dataset_unificado[~dataset_unificado["reside_fora_sp"]]

n_fora, n_sp = fora["qtd_internacoes_uti"].sum(), sp["qtd_internacoes_uti"].sum()
val_fora, val_sp = fora["soma_val_tot"].sum(), sp["soma_val_tot"].sum()
val_total = val_fora + val_sp

print(f"Internações de fora de SP: {n_fora:,} ({n_fora/(n_fora+n_sp)*100:.1f}% do total)")
print(f"Gasto com pacientes de fora de SP: R$ {val_fora:,.2f} ({val_fora/val_total*100:.2f}% do gasto total)")
print(f"Custo médio por internação (fora de SP): R$ {val_fora/n_fora:,.2f}")
print(f"Custo médio por internação (SP): R$ {val_sp/n_sp:,.2f}")
print(f"Razão de custo (fora/SP): {(val_fora/n_fora)/(val_sp/n_sp):.2f}x")

print()
fora_icsap = fora[fora["icsap"] == "sis"]["qtd_internacoes_uti"].sum()
sp_icsap = sp[sp["icsap"] == "sis"]["qtd_internacoes_uti"].sum()
print(f"Dos casos de fora de SP, % ICSAP (evitável): {fora_icsap/n_fora*100:.1f}%")
print(f"Dos casos de SP, % ICSAP (evitável): {sp_icsap/n_sp*100:.1f}%")

Internações de fora de SP: 8,304 (1.0% do total)
Gasto com pacientes de fora de SP: R$ 93,883,282.74 (1.43% do gasto total)
Custo médio por internação (fora de SP): R$ 11,305.79
Custo médio por internação (SP): R$ 8,015.71
Razão de custo (fora/SP): 1.41x

Dos casos de fora de SP, % ICSAP (evitável): 7.1%
Dos casos de SP, % ICSAP (evitável): 18.8%
```

Evidência da célula [57] (código e saída)

[58] Top 10 municípios prioritários

Ranqueia municípios por taxa ICSAP com piso mínimo de volume, e gera o gráfico de barras.

Comandos e por que estes:

PISO_VOLUME = 30 — Sem piso, o ranking seria dominado por município minúsculo onde poucos casos disparam a taxa. Elimina 281 dos 645.

Constante nomeada em vez de número solto — Deixa a regra explícita e ajustável num lugar só.

barh com fatiamento [::-1] — Barra horizontal comporta nome de município; a inversão coloca o maior no topo, que é a leitura natural.

```
[58]: import pandas as pd
import matplotlib.pyplot as plt

painel_top10 = pd.read_parquet(DATA_DIR / "painel_municipio_ano_sp_2022_2024.parquet")
painel_top10 = painel_top10[painel_top10["ano"].isin([2023, 2024])].copy()
painel_top10["taxa_icsap_uti_10k"] = (
    painel_top10["qtd_internacoes_icsap_uti"] / painel_top10["populacao"] * 10000
)

por_municipio_top10 = painel_top10.groupby(["municipio_6", "nome_municipio"]).agg(
    taxa_icsap_uti_10k=("taxa_icsap_uti_10k", "mean"),
    populacao=("populacao", "mean"),
    qtd_internacoes_icsap_uti=("qtd_internacoes_icsap_uti", "sum"),
).reset_index()

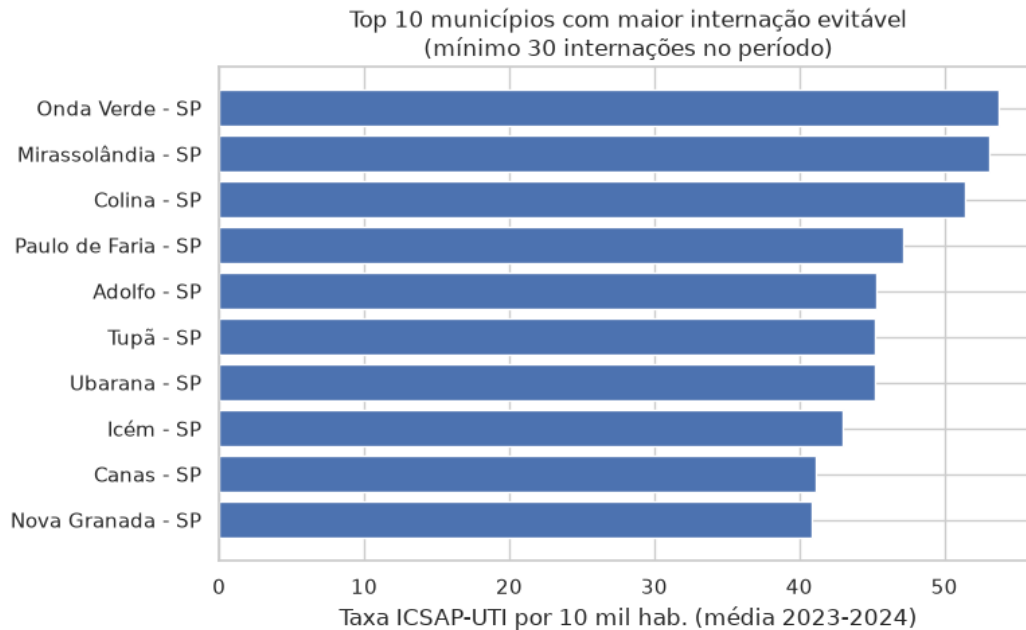
PISO_VOLUME = 30 # mínimo de internacoes ICSAP-UTI no período, pra taxa nao ser so ruído
elegiveis = por_municipio_top10[por_municipio_top10["qtd_internacoes_icsap_uti"] >= PISO_VOLUME]
top10 = elegiveis.sort_values("taxa_icsap_uti_10k", ascending=False).head(10)

print(f"Municípios elegíveis (>= {PISO_VOLUME} internações ICSAP-UTI no período): {len(elegiveis)} de {len(por_municipio_top10)}")
print(top10[["nome_municipio", "taxa_icsap_uti_10k", "populacao", "qtd_internacoes_icsap_uti"]].to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 5))
ax.barh(top10[["nome_municipio"]][::-1], top10["taxa_icsap_uti_10k"][::-1], color="#4C72B0")
ax.set_xlabel("Taxa ICSAP-UTI por 10 mil hab. (média 2023-2024)")
ax.set_title(f"Top 10 municípios com maior internação evitável\n(mínimo {PISO_VOLUME} internações no período)")
plt.tight_layout()
plt.show()
```

Municípios elegíveis (>= 30 internações ICSAP-UTI no período): 364 de 645	nome_municipio	taxa_icsap_uti_10k	populacao	qtd_internacoes_icsap_uti
Onda Verde - SP	53.743837	4841.5	52	
Mirassolândia - SP	53.086760	4720.0	50	
Colina - SP	51.399715	18664.0	192	
Paulo de Faria - SP	47.204540	7415.0	70	
Adolfo - SP	45.314620	4414.5	40	
Tupã - SP	45.236884	64672.0	585	
Ubarana - SP	45.200031	5410.5	49	
Icém - SP	43.014391	7894.0	68	
Canas - SP	41.170851	4990.0	41	
Nova Granada - SP	40.829285	19572.0	160	

Evidência da célula [58] (código e saída)



Evidência da célula [58] (gráfico gerado)

Etapa 12. O Machine Learning explicado

[59] Escada de modelos e justificativa do algoritmo

Treina quatro algoritmos de famílias diferentes na mesma matriz e no mesmo split, do chute proporcional ao XGBoost.

Comandos e por que estes:

`DummyClassifier(strategy="stratified")` — Piso teórico. Se não desse 0,500, haveria erro na montagem da avaliação.

`StandardScaler` antes da `logística` — Modelo linear é sensível à escala; árvore não é, e por isso só a logística recebe o escalonamento.

`class_weight="balanced"` — Equivalente do `scale_pos_weight` nos modelos do scikit-learn, para comparação justa.

O que a escada prova — Logística 0,619, árvore 0,633, XGBoost 0,664. O ganho de 0,045 justifica a escolha, e o teto em 0,66 mostra que o limite é do dado, não do algoritmo.

```
[59]: import time
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

# Mesma matriz de features, mesmo split temporal e mesmo tratamento de desbalanceamento
# da secão 11.4. A ÚNICA coisa que muda entre as linhas da tabela e o algoritmo.
X_treino = pd.get_dummies(df_ml[mask_treino][features_cat + features_num],
                          columns=features_cat, drop_first=True)
X_teste = pd.get_dummies(df_ml[mask_teste][features_cat + features_num],
                          columns=features_cat, drop_first=True)
X_treino, X_teste = X_treino.align(X_teste, join="outer", axis=1, fill_value=0)
y_treino = df_ml[mask_treino][target].astype(int)
y_teste = df_ml[mask_teste][target].astype(int)
peso_classe = (len(y_treino) - y_treino.sum()) / y_treino.sum()

print(f"{X_treino.shape[1]} colunas depois do one-hot | treino (2023) = {len(y_treino):,} | teste (2024) = {len(y_teste):,}")
print(f"Prevalencia de reinternacao em 30 dias no teste: {y_teste.mean():.2%}")
print(f"scale_pos_weight (razao negativo/positivo no treino): {peso_classe:.2f}")
print()

def avaliar(nome, modelo, escalona=False):
    inicio = time.time()
    if escalona:
        escala = StandardScaler().fit(X_treino.astype(float))
        modelo.fit(escala.transform(X_treino.astype(float)), y_treino)
        proba = modelo.predict_proba(escala.transform(X_teste.astype(float)))[:, 1]
    else:
        modelo.fit(X_treino, y_treino)
        proba = modelo.predict_proba(X_teste)[:, 1]
    segundos = time.time() - inicio
    auc = roc_auc_score(y_teste, proba)
    print(f"{nome:42s} ROC-AUC = {auc:.3f} (treino: {segundos:.1f}s)")
    return modelo, proba, auc

print("Escada de modelos, do mais simples ao escolhido:")
avaliar("Chute proporcional (piso teorico)",
        DummyClassifier(strategy="stratified", random_state=42))
avaliar("Regressao logistica (baseline linear)",
        LogisticRegression(max_iter=1000, class_weight="balanced"), escalona=True)
avaliar("Arvore de decisao unica (profundidade 6)",
        DecisionTreeClassifier(max_depth=6, class_weight="balanced", random_state=42))
modelo_xgb, proba_xgb, auc_xgb = avaliar(
    "XGBoost (o escolhido, secão 11.4)",
    xgb.XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1,
                      scale_pos_weight=peso_classe, random_state=42, n_jobs=-1))
```

Evidência da célula [59] (código)

```
modelo_xgb, proba_xgb, auc_xgb = avaliar(
    "XGBoost (o escolhido, secão 11.4)",
    xgb.XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1,
                      scale_pos_weight=peso_classe, random_state=42, n_jobs=-1))
```

49 colunas depois do one-hot | treino (2023) = 267,940 | teste (2024) = 267,574
 Prevalencia de reinternacao em 30 dias no teste: 14.92%
 scale_pos_weight (razao negativo/positivo no treino): 5.76

Escada de modelos, do mais simples ao escolhido:

Chute proporcional (piso teorico)	ROC-AUC = 0.500	(treino: 0.3s)
Regressao logistica (baseline linear)	ROC-AUC = 0.619	(treino: 17.9s)
Arvore de decisao unica (profundidade 6)	ROC-AUC = 0.633	(treino: 4.9s)
XGBoost (o escolhido, secão 11.4)	ROC-AUC = 0.664	(treino: 27.6s)

Evidência da célula [59] (saída)

Etapa 13. Metricas do modelo

[60] Painel completo de métricas de classificação

Aplica o corte de 0,5 e reporta precisão, recall, F1, matriz de confusão e ganho sobre a prevalência, nas duas populações.

Comandos e por que estes:

`confusion_matrix(...).ravel()` — O ravel achata a matriz 2x2 em quatro valores, permitindo desempacotar em VN, FP, FN e VP numa linha.

Reportar o ganho sobre a prevalência — Precisão isolada engana quando a prevalência muda: 0,514 na subpopulação parece melhor que 0,215, mas o ganho cai de 1,44x para 1,18x.

Contar quantos o modelo sinaliza — 121.113 de 267.574, ou 45,3% da base. É o número que mostra por que o score não serve para decisão individual.

```
[60]: from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix

def painel_de_metricas(y_verdadeiro, probabilidade, nome, corte=0.5):
    predito = (probabilidade >= corte).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_verdadeiro, predito).ravel()
    prevalencia = y_verdadeiro.mean()
    precisao = precision_score(y_verdadeiro, predito)
    print(f"--- {nome}")
    print(f" N = {len(y_verdadeiro):,} | reinternacoes reais = {int(y_verdadeiro.sum()):,} "
          f"| prevalencia = {prevalencia:.2%}")
    print(f" ROC-AUC {roc_auc_score(y_verdadeiro, probabilidade):.3f}")
    print(f" Precisao {precisao:.3f} (de cada 100 sinalizados, {precisao*100:.0f} reinternam de fato)")
    print(f" Recall {recall_score(y_verdadeiro, predito):.3f} "
          f"(pega {recall_score(y_verdadeiro, predito)*100:.0f}% das reinternacoes que existiram)")
    print(f" F1 {f1_score(y_verdadeiro, predito):.3f}")
    print(f" Ganho sobre chutar na prevalencia: {precisao/prevalencia:.2f}x")
    print(f" Sinalizados pelo modelo: {tp+fp:,} de {len(y_verdadeiro):,} ({(tp+fp)/len(y_verdadeiro):.1%} da base)")
    print(f" Matriz de confusao: VN={tn:,} | FP={fp:,} | FN={fn:,} | VP={tp:,}")
    print()

painel_de_metricas(y_teste, proba_xgb, "Populacao inteira (o numero de cabecalho da secao 11.4)")

# Mesmo modelo, mesma receita, so na populacao onde reinternar e fisicamente possivel
X_sub_treino = pd.get_dummies(df_sub[df_sub["ano"] == 2023][features_cat + features_num],
                             columns=features_cat, drop_first=True)
X_sub_teste = pd.get_dummies(df_sub[df_sub["ano"] == 2024][features_cat + features_num],
                             columns=features_cat, drop_first=True)
X_sub_treino, X_sub_teste = X_sub_treino.align(X_sub_teste, join="outer", axis=1, fill_value=0)
y_sub_treino = df_sub[df_sub["ano"] == 2023][target].astype(int)
y_sub_teste = df_sub[df_sub["ano"] == 2024][target].astype(int)
peso_sub = (len(y_sub_treino) - y_sub_treino.sum()) / y_sub_treino.sum()

modelo_sub = xgb.XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1,
                              scale_pos_weight=peso_sub, random_state=42, n_jobs=-1)
modelo_sub.fit(X_sub_treino, y_sub_treino)
proba_sub = modelo_sub.predict_proba(X_sub_teste)[: , 1]

painel_de_metricas(y_sub_teste, proba_sub, "So pacientes com mais de 1 internacao no periodo")
```

Evidência da célula [60] (código)

```
--- Populacao inteira (o numero de cabecalho da secao 11.4)
N = 267,574 | reinternacoes reais = 39,912 | prevalencia = 14.92%
ROC-AUC    0.664
Precisao    0.215 (de cada 100 sinalizados, 22 reinternam de fato)
Recall      0.654 (pega 65% das reinternacoes que existiram)
F1          0.324
Ganho sobre chutar na prevalencia: 1.44x
Sinalizados pelo modelo: 121,113 de 267,574 (45.3% da base)
Matriz de confusao: VN=132,641 | FP=95,021 | FN=13,820 | VP=26,092

--- So pacientes com mais de 1 internacao no periodo
N = 91,485 | reinternacoes reais = 39,912 | prevalencia = 43.63%
ROC-AUC    0.619
Precisao    0.514 (de cada 100 sinalizados, 51 reinternam de fato)
Recall      0.612 (pega 61% das reinternacoes que existiram)
F1          0.559
Ganho sobre chutar na prevalencia: 1.18x
Sinalizados pelo modelo: 47,468 de 91,485 (51.9% da base)
Matriz de confusao: VN=28,521 | FP=23,052 | FN=15,496 | VP=24,416
```

Evidência da célula [60] (saída)

6. Uso de IA e pesquisa como ferramenta

Somos alunos de primeiro ano de Ciência de Dados. Parte dos recursos usados neste notebook está acima do que a grade cobriu até aqui, e seria desonesto apresentá-los sem dizer como chegamos neles.

Usamos IA e pesquisa aprofundada como ferramenta de construção e de entendimento de sintaxe, com o incentivo explícito da Oracle ao uso de IA no Challenge. Os casos concretos foram: o boxplot ajustado por medcouple (fórmula de Hubert e Vandervieren, célula 42), o pareamento por escore de propensão e o uso de cKDTree para busca de vizinho mais próximo (célula 54), a diferença padronizada de médias como diagnóstico de balanceamento (célula 54), a escolha do t de Welch a partir do teste de Levene (célula 51), e a decomposição de correlação entre e dentro de grupo via transform (célula 46).

O que a ferramenta entregou foi sintaxe e nome de método: qual função existe, quais argumentos ela aceita, o que cada um faz. O que ela não entrega, e que é o conteúdo do notebook, é a decisão de qual pergunta fazer a este dado.

Três exemplos de onde essa fronteira fica visível. Primeiro, o `min_count=1` da célula 15: nenhuma ferramenta avisa que a soma padrão do pandas transformaria 853 combinações município-ano em 'zero leito' fabricado; isso apareceu porque conferimos o resultado contra o dado bruto. Segundo, a decomposição entre e dentro do município da célula 46, que só faz sentido depois de a matriz de correlação da célula 44 ter revelado o confundidor por porte. Terceiro, a decisão da célula 52 de trocar a unidade de análise ao perceber viés de seleção, em vez de reportar o resultado bruto que parecia favorável.

Em todos os pontos onde um recurso foi usado sem que dominássemos a sua teoria completa, ele aparece no notebook com o resultado declarado e a limitação registrada, e não como afirmação fechada. O pareamento da célula 55 é o exemplo mais claro: rodamos as duas versões, elas discordam, e isso está impresso no notebook em vez de escondido.

7. Evidência de execução do script .py

O script foi executado do início ao fim, sem erro, com código de saída 0. Todos os números conferem com a execução do notebook. Abaixo, os quatro gráficos gerados automaticamente na pasta figuras/ e um recorte do log de execução.

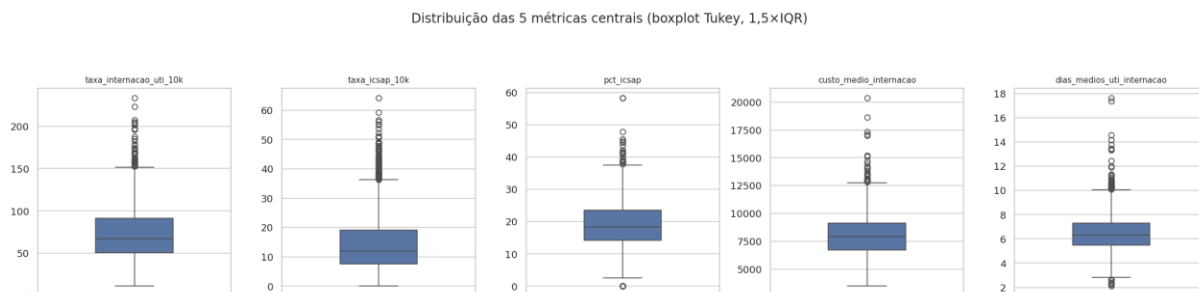


Figura 1 gerada pelo script

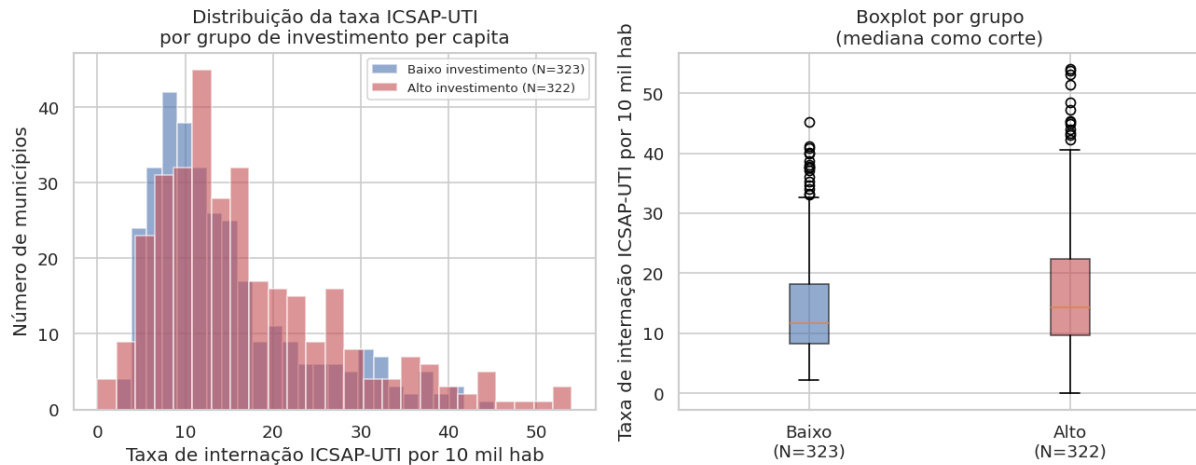


Figura 2 gerada pelo script

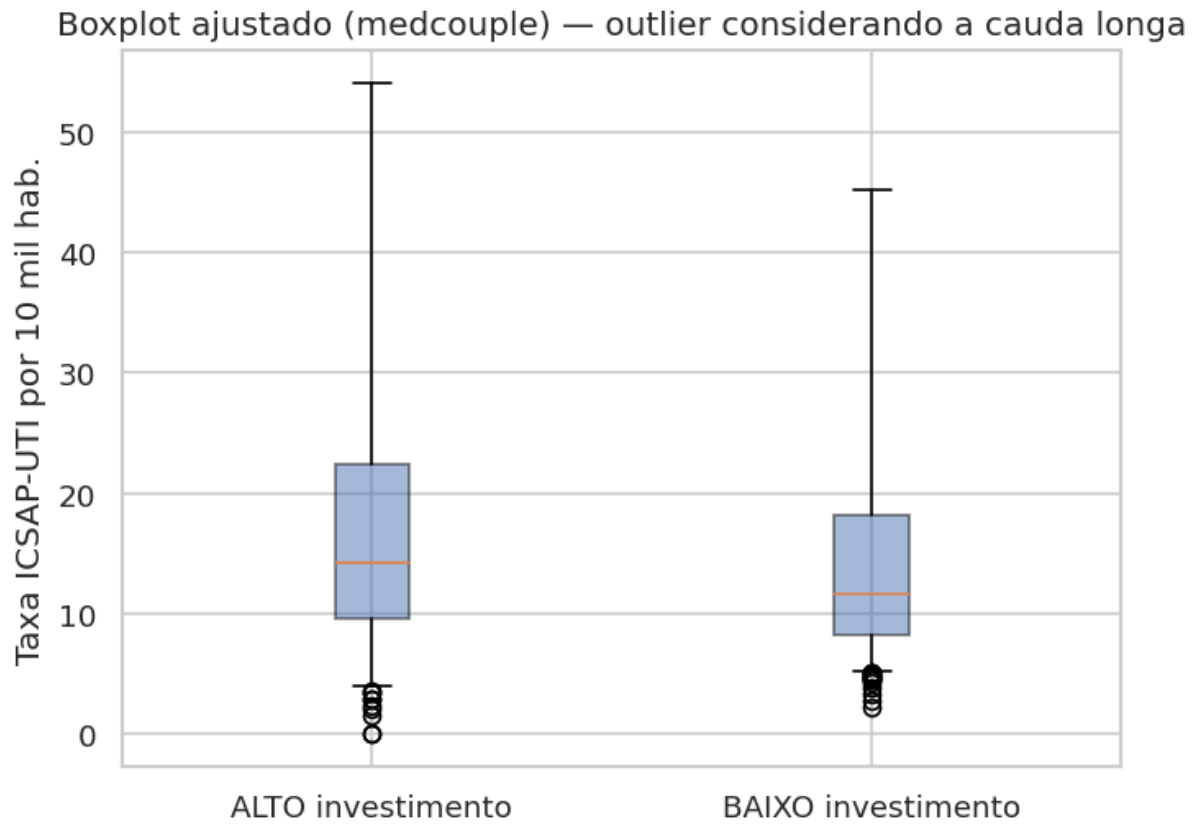


Figura 3 gerada pelo script

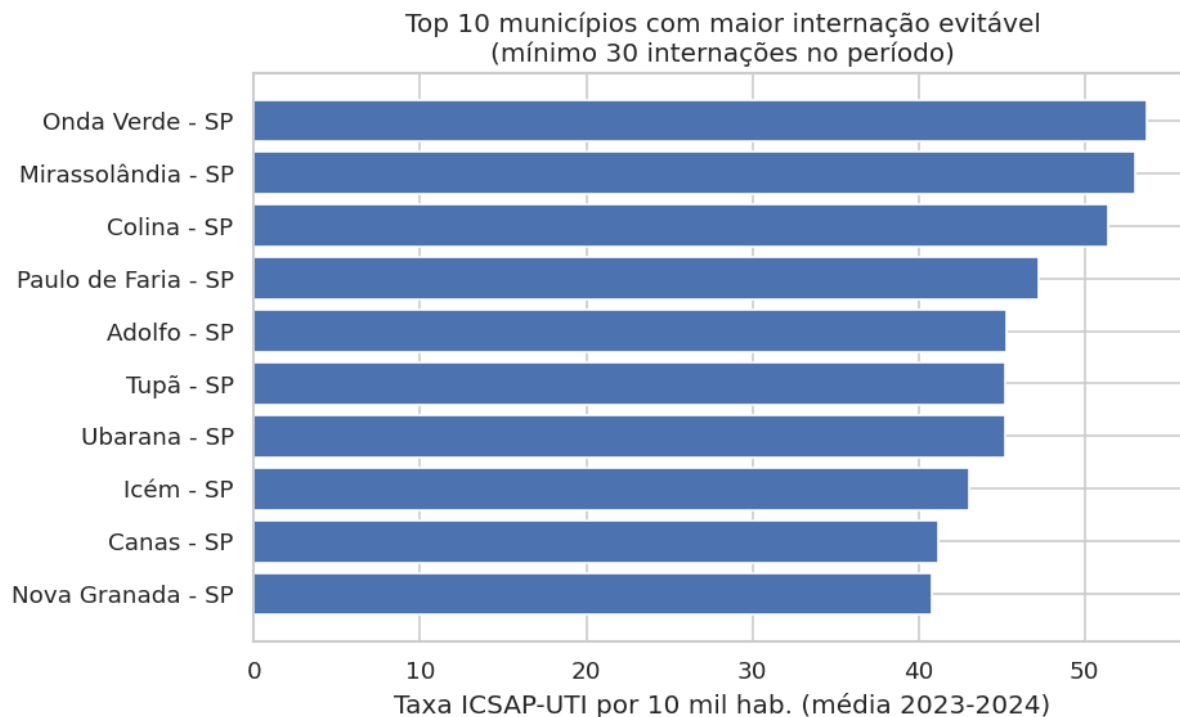


Figura 4 gerada pelo script

Recorte do log de execução (últimas linhas):

```
Escada de modelos, do mais simples ao escolhido:
Chute proporcional (piso teorico)      ROC-AUC = 0.500 (treino: 0.0s)
Regressao logistica (baseline linear)  ROC-AUC = 0.619 (treino: 1.8s)
Arvore de decisao unica (profundidade 6) ROC-AUC = 0.633 (treino: 0.8s)
XGBoost (o escolhido, secao 11.4)      ROC-AUC = 0.664 (treino: 3.3s)
--- Populacao inteira (o numero de cabecalho da secao 11.4)
N = 267,574 | reinternacoes reais = 39,912 | prevalencia = 14.92%
ROC-AUC 0.664
Precisao 0.215 (de cada 100 sinalizados, 22 reinternam de fato)
Recall 0.654 (pega 65% das reinternacoes que existiram)
F1 0.324
Ganho sobre chutar na prevalencia: 1.44x
Sinalizados pelo modelo: 121,113 de 267,574 (45.3% da base)
Matriz de confusao: VN=132,641 | FP=95,021 | FN=13,820 | VP=26,092

--- So pacientes com mais de 1 internacao no periodo
N = 91,485 | reinternacoes reais = 39,912 | prevalencia = 43.63%
ROC-AUC 0.619
Precisao 0.514 (de cada 100 sinalizados, 51 reinternam de fato)
Recall 0.612 (pega 61% das reinternacoes que existiram)
F1 0.559
Ganho sobre chutar na prevalencia: 1.18x
Sinalizados pelo modelo: 47,468 de 91,485 (51.9% da base)
Matriz de confusao: VN=28,521 | FP=23,052 | FN=15,496 | VP=24,416
```

8. Código-fonte completo

Conteúdo integral de clariti_aed_icsap_util_sp.py, entregue também como arquivo separado.

```
# -*- coding: utf-8 -*-
"""
CLARITI - Challenge FIAP-Oracle 2026
Analise Exploratoria, Preparacao Estatistica, Teste de Hipotese e Machine Learning
sobre internacoes com UTI classificadas por ICSAP no estado de Sao Paulo, 2022-2024.

Building Data-Driven Applications for Data Science - Sprint 3

Requisitos:
    pandas, numpy, matplotlib, seaborn, scipy, statsmodels, scikit-learn, xgboost

Uso:
    python clariti_aed_icsap_util_sp.py

Arquivos de entrada esperados em DATA_DIR:
    dataset_unificado_sp_2022_2024.parquet
    painel_municipio_ano_sp_2022_2024.parquet
    perfil_paciente_sp_2022_2024.parquet
    regioao_municipios_sp.csv
    rotas_polos_sp_real.csv
"""

import calendar
import time
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import xgboost as xgb
from scipy.spatial import cKDTree
from scipy.stats import (levene, mannwhitneyu, shapiro, spearmanr, ttest_ind,
                        ttest_rel, wilcoxon)
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, f1_score, precision_score,
                            recall_score, roc_auc_score)
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from statsmodels.stats.stattools import medcouple

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

DATA_DIR = Path("../data")
FIG_DIR = Path("figuras")
FIG_DIR.mkdir(exist_ok=True)

ARQUIVOS_DO_PROJETO = [
    "dataset_unificado_sp_2022_2024.parquet",
    "painel_municipio_ano_sp_2022_2024.parquet",
    "perfil_paciente_sp_2022_2024.parquet",
    "regiao_municipios_sp.csv",
    "rotas_polos_sp_real.csv",
]

faltando = [nome for nome in ARQUIVOS_DO_PROJETO if not (DATA_DIR / nome).exists()]
print(f"DATA_DIR = {DATA_DIR}")
print(f"Arquivos encontrados: {len(ARQUIVOS_DO_PROJETO) - len(faltando)} de {len(ARQUIVOS_DO_PROJETO)}")
if faltando:
    print("FALTANDO (o script nao roda sem eles):")
    for nome in faltando:
        print(f" - {nome}")
    raise SystemExit(1)

df = pd.read_parquet(DATA_DIR / "dataset_unificado_sp_2022_2024.parquet")
painel = pd.read_parquet(DATA_DIR / "painel_municipio_ano_sp_2022_2024.parquet")
perfil = pd.read_parquet(DATA_DIR / "perfil_paciente_sp_2022_2024.parquet")
```

```

regiao_municipios = pd.read_csv(DATA_DIR / "regiao_municipios_sp.csv", dtype={"municipio_6": str})
rotas = pd.read_csv(DATA_DIR / "rotas_polos_sp_real.csv", dtype={"municipio_6": str})

dataset_unificado = df
painel_desp = painel
regiao_teste_t = regiao_municipios

print(f"{len(df):,} linhas, {df.shape[1]} colunas")

# =====
# 1. Perfil geral do dataset
# =====

# --- [3] ---
print("Período:", df["ano"].min(), "a", df["ano"].max())
print("Meses por ano:", sorted(int(m) for m in df["mes"].unique()))
print()
print("Combinações (município, diag_princ, ano, mês) = linhas do dataset:", f"{len(df):,}")
print("Códigos de diagnóstico distintos (diag_princ):", df["diag_princ"].nunique())
print("Códigos de município distintos (municipio_6):", df["municipio_6"].nunique())

# --- [4] ---
total_internacoes = df["qtd_internacoes_uti"].sum()
print(f"Total de internações com UTI (soma de qtd_internacoes_uti, não linha): {total_internacoes:,}")
print()
print("Por ano:")
print(df.groupby("ano")["qtd_internacoes_uti"].sum().apply(lambda x: f"{x:,}"))

# --- [5] ---
residentes_sp = df.loc[~df["reside_fora_sp"], "qtd_internacoes_uti"].sum()
residentes_fora = df.loc[df["reside_fora_sp"], "qtd_internacoes_uti"].sum()

municipios_sp = df.loc[~df["reside_fora_sp"], "municipio_6"].nunique()
municipios_fora = df.loc[df["reside_fora_sp"], "municipio_6"].nunique()

print(f"Municípios de SP que aparecem no dataset: {municipios_sp} (de 645 no total do estado)")
print(f"Municípios de fora de SP que aparecem: {municipios_fora}")
print()
print(f"Internações de residente de SP: {residentes_sp:,} ({residentes_sp/total_internacoes:.1%})")
print(f"Internações de residente de fora de SP: {residentes_fora:,} ({residentes_fora/total_internacoes:.1%})")

# --- [6] ---
icsap = df.loc[df["csap"] == "sim", "qtd_internacoes_uti"].sum()
nao_icsap = df.loc[df["csap"] == "não", "qtd_internacoes_uti"].sum()

print(f"ICSAP: {icsap:,} ({icsap/total_internacoes:.1%})")
print(f"Não-ICSAP: {nao_icsap:,} ({nao_icsap/total_internacoes:.1%})")
print()
print("% ICSAP por ano:")
tab = df.groupby(["ano", "csap"])["qtd_internacoes_uti"].sum().unstack(fill_value=0)
tab["% icsap"] = (tab["sim"] / (tab["sim"] + tab["não"]) * 100).round(1)
print(tab)

# =====
# 2. Missing (valores ausentes)
# =====

# --- [7] ---
nulos = df.isna().sum()
nulos = nulos[nulos > 0].sort_values(ascending=False)
pct = (nulos / len(df) * 100).round(2)

resumo_nulos = pd.DataFrame({"linhas_nulas": nulos, "pct_do_total": pct})
print(resumo_nulos)

# --- [8] ---
# confere a regra: grupo_csap_cod é nulo se e só se csap == "não"
inconsistente = ((df["csap"] == "não") != df["grupo_csap_cod"].isna()).sum()
print(f"Linhas onde a regra 'grupo_csap_cod nulo <=> csap=não' NÃO vale: {inconsistente}")

# --- [9] ---

```

```

pop_nula = df["populacao"].isna()

fora_sp = (pop_nula & df["reside_fora_sp"]).sum()
bug_sp = (pop_nula & ~df["reside_fora_sp"]).sum()

print(f"Total nulo: {pop_nula.sum():,}")
print(f"    Residente de fora de SP (esperado – não faz sentido ter população/CNES de SP pra quem não mora aqui): {fora_sp:,}")
print(f"    Residente de SP sem população (bug de ordem de merge, documentado no .docx seção 2.8, não corrigido): {bug_sp:,}")

# --- [10] ---
leito_nulo = df["qtd_leitos_existentes"].isna()

junto_com_grupo_781 = (leito_nulo & df["qtd_estabelecimentos"].isna()).sum()
so_leito = (leito_nulo & df["qtd_estabelecimentos"].notna()).sum()

print(f"Total nulo: {leito_nulo.sum():,}")
print(f"    Mesmas linhas do grupo anterior (fora de SP + bug): {junto_com_grupo_781:,}")
print(f"    SÓ leito nulo, estabelecimento preenchido (gap CNES-LT vs CNES-ST): {so_leito:,}")

# --- [11] ---
so_estabelecimento_nulo_leito_preenchido = (
    df["qtd_estabelecimentos"].isna() & df["qtd_leitos_existentes"].notna()
).sum()
print(f"Linhas com estabelecimento nulo MAS leito preenchido: {so_estabelecimento_nulo_leito_preenchido}")
print(f"(esperado 0 se o padrão for monotônico – nulo de leito 'engloba' o de estabelecimento)")

# --- [12] ---
tabela_cruzada = pd.crosstab(df["reside_fora_sp"], df["populacao"].isna())
tabela_cruzada.columns = ["populacao_preenchida", "populacao_nula"]
tabela_cruzada.index.name = "reside_fora_sp"
print(tabela_cruzada)

# --- [13] ---
sub = df[df["qtd_estabelecimentos"].notna()].copy()
sub["leito_nulo"] = sub["qtd_leitos_existentes"].isna()
sub["porte"] = pd.qcut(
    sub["qtd_estabelecimentos"], q=4,
    labels=["Q1 (menor)", "Q2", "Q3", "Q4 (maior)"], duplicates="drop"
)

taxa_nulo_por_porte = (sub.groupby("porte", observed=True)["leito_nulo"].mean() * 100).round(1)
print(taxa_nulo_por_porte)

# =====
# 3. Distribuicao das metricas centrais (municipio x ano)
# =====

# --- [14] ---
df_sp_mun = df[~df["reside_fora_sp"]].copy()

metricas_centrais = (
    df_sp_mun.groupby(["municipio_6", "ano"])
    .agg(
        n_internacoes=("qtd_internacoes_uti", "sum"),
        soma_val_tot=("soma_val_tot", "sum"),
        soma_uti_mes_to=("soma_uti_mes_to", "sum"),
        populacao=("populacao", "median"),
        nome_municipio=("nome_municipio", "first"),
    )
    .reset_index()
)

n_icsap = (
    df_sp_mun[df_sp_mun["csap"] == "sim"]
    .groupby(["municipio_6", "ano"])["qtd_internacoes_uti"]
    .sum()
    .rename("n_icsap")
)

metricas_centrais = metricas_centrais.merge(n_icsap, on=["municipio_6", "ano"], how="left")
metricas_centrais["n_icsap"] = metricas_centrais["n_icsap"].fillna(0)

print(f"{len(metricas_centrais):,} combinações município x ano (máximo possível: 645 x 3 = 1.935)")

```

```

print(metricas_centrais.head())

# --- [15] ---
mensal = df_sp_mun.drop_duplicates(subset=["municipio_6", "ano", "mes"])[
    ["municipio_6", "ano", "mes", "qtd_leitos_existentes", "qtd_estabelecimentos"]
]

por_municipio_ano = mensal.groupby(["municipio_6", "ano"])

# leitos: convenção oficial CONASS/DATASUS (soma dos 12 meses / 12).
# min_count=1 é essencial: sem isso, um grupo com os 12 meses nulos soma "0" por
# padrão do pandas, e viraria "0 leitos" fabricado em vez de nulo de verdade.
leitos_soma_sem_min_count = por_municipio_ano["qtd_leitos_existentes"].sum()
leitos_soma = por_municipio_ano["qtd_leitos_existentes"].sum(min_count=1)
leitos_anual = (leitos_soma / 12).rename("leitos_anual_media")

# estabelecimentos: média dos meses disponíveis (mean já ignora nulo por padrão)
estabelecimentos_anual =
por_municipio_ano["qtd_estabelecimentos"].mean().rename("estabelecimentos_anual_media")

anualizado = pd.concat([leitos_anual, estabelecimentos_anual], axis=1).reset_index()

todos_nulos = (leitos_soma_sem_min_count == 0) & (leitos_soma.isna())
total_combinacoes = por_municipio_ano.ngroups
print(f"Combinações município-ano com os 12 meses de leito nulos: {todos_nulos.sum():,} de
{total_combinacoes:,} ({todos_nulos.sum()/total_combinacoes:.1%})")
print("Sem min count=1, essas linhas virariam '0 leitos' por engano.")
print(anualizado.head())

# --- [16] ---
tabela_municipio_ano = metricas_centrais.merge(anualizado, on=["municipio_6", "ano"], how="left")

tabela_municipio_ano["taxa_internacao_uti_10k"] = (
    tabela_municipio_ano["n_internacoes"] / tabela_municipio_ano["populacao"] * 10000
)
tabela_municipio_ano["taxa_icsap_10k"] = (
    tabela_municipio_ano["n_icsap"] / tabela_municipio_ano["populacao"] * 10000
)
tabela_municipio_ano["pct_icsap"] = (
    tabela_municipio_ano["n_icsap"] / tabela_municipio_ano["n_internacoes"] * 100
)
tabela_municipio_ano["custo_medio_internacao"] = (
    tabela_municipio_ano["soma_val_tot"] / tabela_municipio_ano["n_internacoes"]
)
tabela_municipio_ano["dias_medios_uti_internacao"] = (
    tabela_municipio_ano["soma_uti_mes_to"] / tabela_municipio_ano["n_internacoes"]
)

print(f"{len(tabela_municipio_ano):,} linhas, {tabela_municipio_ano.shape[1]} colunas")
print(tabela_municipio_ano.head())

# --- [17] ---
metricas_distribuicao = [
    "taxa_internacao_uti_10k", "taxa_icsap_10k", "pct_icsap",
    "custo_medio_internacao", "dias_medios_uti_internacao",
]

distribuicao = tabela_municipio_ano[metricas_distribuicao].describe(percentiles=[0.25, 0.5, 0.75]).T
distribuicao["iqr"] = distribuicao["75%"] - distribuicao["25%"]
print(distribuicao[["50%", "25%", "75%", "iqr", "min", "max"]].round(2))

# =====
# 4. Outliers (municipio x ano)
# =====

# --- [18] ---
metricas_distribuicao = [
    "taxa_internacao_uti_10k", "taxa_icsap_10k", "pct_icsap",
    "custo_medio_internacao", "dias_medios_uti_internacao",
]

fences = []
outlier_cols = []
for m in metricas_distribuicao:
    q1, q3 = tabela_municipio_ano[m].quantile([0.25, 0.75])

```

```

iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
col = f"outlier_{m}"
tabela_municipio_ano[col] = (tabela_municipio_ano[m] < lower) | (tabela_municipio_ano[m] > upper)
outlier_cols.append(col)
fences.append({
    "metrica": m, "fence_inferior": round(lower, 2), "fence_superior": round(upper, 2),
    "outliers_total": int(tabela_municipio_ano[col].sum()),
    "outliers_baixo": int((tabela_municipio_ano[m] < lower).sum()),
    "outliers_alto": int((tabela_municipio_ano[m] > upper).sum()),
})

print(pd.DataFrame(fences))

# --- [19] ---
fig, eixos = plt.subplots(1, 5, figsize=(19, 4.5))
for eixo, m in zip(eixos, metricas_distribuicao):
    sns.boxplot(y=tabela_municipio_ano[m], ax=eixo, color="#4C72B0", width=0.4)
    eixo.set_title(m, fontsize=9)
    eixo.set_ylabel("")
fig.suptitle("Distribuição das 5 métricas centrais (boxplot Tukey, 1,5×IQR)", y=1.04)
plt.tight_layout(rect=[0, 0, 1, 0.94])
plt.savefig(FIG_DIR / "fig01_boxplot_5_metricas.png", dpi=120, bbox_inches="tight")
plt.close()

# --- [20] ---
tabela_municipio_ano["n_metricas_outlier"] = tabela_municipio_ano[outlier_cols].sum(axis=1)
print(tabela_municipio_ano["n_metricas_outlier"].value_counts().sort_index())

# --- [21] ---
cols_show = ["nome_municipio", "ano", "populacao", "n_internacoes", "n_icsap"] + metricas_distribuicao
tres_metricas = tabela_municipio_ano[tabela_municipio_ano["n_metricas_outlier"] == 3]
print(tres_metricas[cols_show].sort_values("populacao"))

# --- [22] ---
tem_algun_outlier = tabela_municipio_ano["n_metricas_outlier"] > 0

mediana_n_outlier = tabela_municipio_ano.loc[tem_algun_outlier, "n_internacoes"].median()
mediana_n_nao_outlier = tabela_municipio_ano.loc[~tem_algun_outlier, "n_internacoes"].median()

print(f"Mediana de n_internacoes | é outlier em alguma métrica: {mediana_n_outlier:.0f}")
print(f"Mediana de n_internacoes | não é outlier em nenhuma: {mediana_n_nao_outlier:.0f}")

# --- [23] ---
colunas_tupa = [
    "nome_municipio", "ano", "populacao", "n_internacoes", "n_icsap",
    "pct_icsap", "taxa_internacao_uti_10k", "taxa_icsap_10k",
]
print(tabela_municipio_ano[tabela_municipio_ano["nome_municipio"].str.contains("Tupã",
na=False)][colunas_tupa])

# =====
# 5. Correlacao (municipio x ano)
# =====

# --- [24] ---
colunas_correlacao = metricas_distribuicao + ["leitos_anual_media", "estabelecimentos_anual_media"]

n_leito_valido = tabela_municipio_ano["leitos_anual_media"].notna().sum()
print(f"Correlações com leitos_anual_media usam {n_leito_valido:,} de {len(tabela_municipio_ano):,} linhas
"
f"({n_leito_valido/len(tabela_municipio_ano):.1%}) – o resto é nulo (achado da etapa 3, gap CNES-
LT).")
print(f"Correlações com estabelecimentos_anual_media usam
{tabela_municipio_ano['estabelecimentos_anual_media'].notna().sum():,} linhas (sem nulo relevante).")

correlacao_spearman = tabela_municipio_ano[colunas_correlacao].corr(method="spearman")
correlacao_pearson = tabela_municipio_ano[colunas_correlacao].corr(method="pearson")

print(correlacao_spearman.loc[metricas_distribuicao, ["leitos_anual_media",
"estabelecimentos_anual_media"]].round(3))

# --- [25] ---
diferenca_metodo = (
    correlacao_spearman.loc[metricas_distribuicao, ["leitos_anual_media", "estabelecimentos_anual_media"]]

```

```

- correlacao_pearson.loc[metricas_distribuicao, ["leitos_anual_media",
"estabelecimentos_anual_media"]]
).abs()

print(f"Maior diferença absoluta entre Spearman e Pearson: {diferenca_metodo.values.max():.3f}")
print(diferenca_metodo.round(3))

# --- [26] ---
tabela_municipio_ano["leitos_10k_hab"] = (
    tabela_municipio_ano["leitos_anual_media"] / tabela_municipio_ano["populacao"] * 10000
)

mediana_leitos_10k = tabela_municipio_ano["leitos_10k_hab"].median()
tupa_leitos_10k = tabela_municipio_ano[tabela_municipio_ano["nome_municipio"].str.contains("Tupã",
na=False)][
    ["ano", "leitos_10k_hab", "taxa_icsap_10k"]
]

print(f"Mediana geral de leitos por 10 mil hab: {mediana_leitos_10k:.1f}")
print(tupa_leitos_10k)

# =====
# 6. Sazonalidade (mes x estado inteiro)
# =====

# --- [27] ---
pop_dim_sp = (
    df[~df["reside_fora_sp"]]
    .drop_duplicates(subset=["municipio_6", "ano"])[["municipio_6", "ano", "populacao"]]
)
populacao_sp_ano = pop_dim_sp.groupby("ano")["populacao"].sum()

tabela_ano_mes = (
    df.groupby(["ano", "mes"])
    .agg(
        n_internacoes=("qtd_internacoes_uti", "sum"),
        soma_val_tot=("soma_val_tot", "sum"),
        soma_uti_mes_to=("soma_uti_mes_to", "sum"),
    )
    .reset_index()
)

n_icsap_mes = (
    df[df["csap"] == "sim"].groupby(["ano", "mes"])["qtd_internacoes_uti"].sum().rename("n_icsap")
)
tabela_ano_mes = tabela_ano_mes.merge(n_icsap_mes, on=["ano", "mes"], how="left")
tabela_ano_mes["n_icsap"] = tabela_ano_mes["n_icsap"].fillna(0)

tabela_ano_mes = tabela_ano_mes.merge(populacao_sp_ano.rename("populacao_sp"), on="ano", how="left")

tabela_ano_mes["taxa_internacao_uti_10k"] = tabela_ano_mes["n_internacoes"] /
tabela_ano_mes["populacao_sp"] * 10000
tabela_ano_mes["taxa_icsap_10k"] = tabela_ano_mes["n_icsap"] / tabela_ano_mes["populacao_sp"] * 10000
tabela_ano_mes["pct_icsap"] = tabela_ano_mes["n_icsap"] / tabela_ano_mes["n_internacoes"] * 100
tabela_ano_mes["custo_medio_internacao"] = tabela_ano_mes["soma_val_tot"] /
tabela_ano_mes["n_internacoes"]
tabela_ano_mes["dias_medios_uti_internacao"] = tabela_ano_mes["soma_uti_mes_to"] /
tabela_ano_mes["n_internacoes"]

metricas_sazonais = ["taxa_internacao_uti_10k", "taxa_icsap_10k", "pct_icsap", "custo_medio_internacao",
"dias_medios_uti_internacao"]

print(f"Tabela ano+mês: {tabela_ano_mes.shape[0]} linhas (esperado 36 = 3 anos x 12 meses).")
print(tabela_ano_mes[["ano", "mes"] + metricas_sazonais].round(2))

# --- [28] ---
tabela_ano_mes["dias_no_mes"] = tabela_ano_mes.apply(
    lambda r: calendar.monthrange(int(r["ano"]), int(r["mes"]))[1], axis=1
)

padrao_sazonal = tabela_ano_mes.groupby("mes")[metricas_sazonais].median()
padrao_sazonal["n_anos"] = tabela_ano_mes.groupby("mes").size()

print("Padrão sazonal por mês (mediana dos 3 anos, N=3 por mês – leitura exploratória, não teste
estatístico):")

```

```

print(padrao_sazonal.round(3))

# --- [29] ---
tabela_ano_mes["taxa_internacao_uti_10k_dia"] = tabela_ano_mes["taxa_internacao_uti_10k"] /
tabela_ano_mes["dias_no_mes"]
tabela_ano_mes["taxa_icsap_10k_dia"] = tabela_ano_mes["taxa_icsap_10k"] / tabela_ano_mes["dias_no_mes"]

padrao_sazonal_dia = tabela_ano_mes.groupby("mes")[["taxa_internacao_uti_10k_dia",
"taxa_icsap_10k_dia"]].median()
variacao_pct = (padrao_sazonal_dia.max() - padrao_sazonal_dia.min()) / padrao_sazonal_dia.median() * 100

print("Taxa por DIA (corrige o viés de mês com menos dias, ex: fevereiro):")
print(padrao_sazonal_dia.sort_values("taxa_internacao_uti_10k_dia").round(4))
print()
print("Variação (mês mais alto - mês mais baixo) / mediana:")
print(variacao_pct.round(1))

# --- [30] ---
grupos_sazonalidade = {"g12": "Cerebrovasculares", "g11": "Insuf. cardíaca", "g10": "Angina", "g08":
"Pulmonares"}

sub_grupos = df[df["grupo_csap_cod"].isin(grupos_sazonalidade)].copy()
tabela_grupos_mes = (
    sub_grupos.groupby(["grupo_csap_cod", "ano", "mes"])["qtd_internacoes_uti"].sum().reset_index()
)
tabela_grupos_mes["dias_no_mes"] = tabela_grupos_mes.apply(
    lambda r: calendar.monthrange(int(r["ano"]), int(r["mes"]))[1], axis=1
)
tabela_grupos_mes["internacoes_dia"] = tabela_grupos_mes["qtd_internacoes_uti"] /
tabela_grupos_mes["dias_no_mes"]

resumo_grupos = []
for cod, nome in grupos_sazonalidade.items():
    pool_grupo = tabela_grupos_mes[tabela_grupos_mes["grupo_csap_cod"] ==
cod].groupby("mes")["internacoes_dia"].median()
    variacao = (pool_grupo.max() - pool_grupo.min()) / pool_grupo.median() * 100
    resumo_grupos.append({
        "grupo": f"{cod} ({nome})",
        "mes_mais_baixo": int(pool_grupo.idxmin()),
        "mes_mais_alto": int(pool_grupo.idxmax()),
        "variacao_pct": round(variacao, 1),
    })

resumo_grupos_df = pd.DataFrame(resumo_grupos).sort_values("variacao_pct")
print(resumo_grupos_df)

# =====
# 7. Recorte geografico
# =====

# --- [31] ---
ranking_leitos = (
    tabela_municipio_ano.groupby(["municipio_6", "nome_municipio"])["leitos_10k_hab"]
        .median()
        .reset_index()
        .sort_values("leitos_10k_hab", ascending=False)
)

mediana_geral_leitos = ranking_leitos["leitos_10k_hab"].median()
print(f"Mediana geral de leitos por 10 mil hab: {mediana_geral_leitos:.1f}")
print(f"Total de municípios no ranking: {len(ranking_leitos)}")
print(ranking_leitos.head(15).reset_index(drop=True))

# --- [32] ---
resumo_municipio = (
    tabela_municipio_ano.groupby(["municipio_6", "nome_municipio"])
        .agg(leitos_10k_hab=("leitos_10k_hab", "median"), n_internacoes=("n_internacoes", "median"),
populacao=("população", "median"))
        .dropna()
        .reset_index()
)

corr_leito_volume = resumo_municipio[["leitos_10k_hab", "n_internacoes"]].corr(method="spearman").iloc[0,
1]
corr_leito_populacao = resumo_municipio[["leitos_10k_hab", "populacao"]].corr(method="spearman").iloc[0,
1]

```

```

print(f"N de municípios com leito válido nos 3 anos: {len(resumo_municipio)} de
{tabela_municipio_ano['municipio_6'].nunique()}")
print(f"Correlação (Spearman) leitos_10k_hab x n_internacoes: {corr_leito_volume:.3f}")
print(f"Correlação (Spearman) leitos_10k_hab x população: {corr_leito_populacao:.3f}")

# --- [33] ---
p75_leito = resumo_municipio["leitos_10k_hab"].quantile(0.75)
p75_volume = resumo_municipio["n_internacoes"].quantile(0.75)

candidatos_polo = resumo_municipio[
    (resumo_municipio["leitos_10k_hab"] >= p75_leito) & (resumo_municipio["n_internacoes"] >= p75_volume)
].sort_values("n_internacoes", ascending=False)

print(f"Candidatos a polo regional (top 25% em leito per capita E top 25% em volume):
{len(candidatos_polo)} de {len(resumo_municipio)} municípios")
print(candidatos_polo.round(1).reset_index(drop=True))

# --- [34] ---
tabela_municipio_ano_regiao = tabela_municipio_ano.merge(
    regioao_municipios[["municipio_6", "regiao_intermediaria"]], on="municipio_6", how="left"
)

print(f"Municípios sem região identificada:
{tabela_municipio_ano_regiao['regiao_intermediaria'].isna().sum()} de {len(tabela_municipio_ano_regiao)}
linhas.")
print(f"Regiões intermediárias cobrindo SP:
{tabela_municipio_ano_regiao['regiao_intermediaria'].nunique()}")

# --- [35] ---
metricas_por_regiao = (
    tabela_municipio_ano_regiao.groupby("regiao_intermediaria")[metricas_distribuicao].median().round(2)
)
n_municipios_regiao = regioao_municipios.groupby("regiao_intermediaria").size().rename("n_municipios")
metricas_por_regiao = metricas_por_regiao.merge(n_municipios_regiao, on="regiao_intermediaria")

print(metricas_por_regiao.sort_values("taxa_icsap_10k", ascending=False))

# --- [36] ---
dispersao_regiao = (
    tabela_municipio_ano_regiao.groupby("regiao_intermediaria")["taxa_icsap_10k"]
    .apply(lambda s: s.quantile(0.75) - s.quantile(0.25))
    .rename("iqr_taxa_icsap_10k")
    .sort_values(ascending=False)
)

print("IQR de taxa_icsap_10k por região (dispersão interna, comparar com o IQR geral da etapa 3: 11,5):")
print(dispersao_regiao.round(2))

# =====
# 8. Síntese final da AED
# =====

# --- [37] ---
fences_df = pd.DataFrame(fences).set_index("metrica")

resumo_etapas = pd.DataFrame({
    "mediana_geral": distribuicao["50%"],
    "iqr": distribuicao["iqr"],
})
resumo_etapas["pct_outlier"] = (fences_df["outliers_total"] / len(tabela_municipio_ano) * 100).round(1)
resumo_etapas["corr_spearman_c_estabelecimento"] = correlacao_spearman.loc[
    metricas_distribuicao, "estabelecimentos_anual_media"
].round(3)

variacao_sazonal_map = {
    "taxa_internacao_uti_10k": variacao_pct.get("taxa_internacao_uti_10k_dia"),
    "taxa_icsap_10k": variacao_pct.get("taxa_icsap_10k_dia"),
}
resumo_etapas["variacao_sazonal_pct"] = resumo_etapas.index.map(variacao_sazonal_map).round(1)

variacao_regional = metricas_por_regiao[metricas_distribuicao].max() -
metricas_por_regiao[metricas_distribuicao].min()
resumo_etapas["variacao_geografica_absoluta"] = variacao_regional.round(2)

```

```

print(resumo_etapas.round(2))

# =====
# 9. Preparacao estatistica (pre-requisitos do teste)
# =====

# --- [38] ---
com_siops = painel.dropna(subset=["siops_despesa_paga"]).copy()
com_siops["despesa_paga_per_capita"] = com_siops["siops_despesa_paga"] / com_siops["populacao"]
com_siops["taxa_icsap_uti_10k"] = com_siops["qtd_internacoes_icsap_uti"] / com_siops["populacao"] * 10_000
com_siops["leitos_sus_10k"] = com_siops["qtd_leitos_sus_media"] / com_siops["populacao"] * 10_000

por_municipio = (
    com_siops.groupby("municipio_6")
    .agg(
        nome_municipio=("nome_municipio", "first"),
        despesa_paga_per_capita=("despesa_paga_per_capita", "mean"),
        taxa_icsap_uti_10k=("taxa_icsap_uti_10k", "mean"),
        populacao=("populacao", "mean"),
        leitos_sus_10k=("leitos_sus_10k", "mean"),
    )
    .reset_index()
)

mediana_investimento = por_municipio["despesa_paga_per_capita"].median()
grupo_alto = por_municipio.loc[por_municipio["despesa_paga_per_capita"] > mediana_investimento].copy()
grupo_baixo = por_municipio.loc[por_municipio["despesa_paga_per_capita"] <= mediana_investimento].copy()

print(f"{len(por_municipio)} municípios com SIOPS válido em pelo menos um dos dois anos (2023 ou 2024)")
print(f"Mediana de investimento per capita: R$ {mediana_investimento:,.2f}")
print(f"Grupo ALTO investimento: {len(grupo_alto)} municípios | Grupo BAIXO investimento: {len(grupo_baixo)} municípios")

# --- [39] ---
fig, eixos = plt.subplots(1, 2, figsize=(11, 4.5))

eixos[0].hist(grupo_baixo["taxa_icsap_uti_10k"], bins=25, alpha=0.6, color="#4C72B0",
              label=f"Baixo investimento (N={len(grupo_baixo)})")
eixos[0].hist(grupo_alto["taxa_icsap_uti_10k"], bins=25, alpha=0.6, color="#C44E52",
              label=f"Alto investimento (N={len(grupo_alto)})")
eixos[0].set_xlabel("Taxa de internação ICSAP-UTI por 10 mil hab")
eixos[0].set_ylabel("Número de municípios")
eixos[0].set_title("Distribuição da taxa ICSAP-UTI\ncpor grupo de investimento per capita")
eixos[0].legend(fontsize=8)

caixa = eixos[1].boxplot(
    [grupo_baixo["taxa_icsap_uti_10k"], grupo_alto["taxa_icsap_uti_10k"]],
    tick_labels=[f"Baixo\N(N={len(grupo_baixo)})", f"Alto\N(N={len(grupo_alto)})"],
    patch_artist=True,
)
for caixa_individual, cor in zip(caixa["boxes"], ["#4C72B0", "#C44E52"]):
    caixa_individual.set_facecolor(cor)
    caixa_individual.set_alpha(0.6)
eixos[1].set_ylabel("Taxa de internação ICSAP-UTI por 10 mil hab")
eixos[1].set_title("Boxplot por grupo\N(mediana como corte)")

plt.tight_layout()
plt.savefig(FIG_DIR / "fig02_distribuicao_por_grupo.png", dpi=120, bbox_inches="tight")
plt.close()

for nome, grupo in [("ALTO investimento", grupo_alto), ("BAIXO investimento", grupo_baixo)]:
    taxa = grupo["taxa_icsap_uti_10k"]
    print(f"{nome}: skew (assimetria) = {taxa.skew():.2f} | curtose (excesso) = {taxa.kurtosis():.2f}")

# --- [40] ---
metricas_descritivas = pd.DataFrame({
    "ALTO investimento": grupo_alto["taxa_icsap_uti_10k"].describe(),
    "BAIXO investimento": grupo_baixo["taxa_icsap_uti_10k"].describe(),
}).round(2)
print(metricas_descritivas)

# --- [41] ---
print("Linhas do painel original:", len(painel))
print("\n% de siops_despesa_paga nulo, por ano:")

```

```

print(painel.groupby("ano")["siops_despesa_paga"].apply(lambda s: s.isna().mean()))
print("\nValores ausentes na tabela por_municipio (N = 645):")
print(por_municipio.isna().sum())

# --- [42] ---
def fences_boxplot_ajustado(serie):
    q1, q3 = serie.quantile([0.25, 0.75])
    iqr = q3 - q1
    mc = medcouple(serie.to_numpy())
    if mc >= 0:
        inferior = q1 - 1.5 * iqr * np.exp(-4 * mc)
        superior = q3 + 1.5 * iqr * np.exp(3 * mc)
    else:
        inferior = q1 - 1.5 * iqr * np.exp(-3 * mc)
        superior = q3 + 1.5 * iqr * np.exp(4 * mc)
    return inferior, superior, mc

for nome, grupo in [("ALTO investimento", grupo_alto), ("BAIXO investimento", grupo_baixo)]:
    taxa = grupo["taxa_icsap_uti_10k"]
    inferior, superior, mc = fences_boxplot_ajustado(taxa)
    n_outlier = ((taxa < inferior) | (taxa > superior)).sum()
    print(f"{nome}: fence=[{inferior:.2f}, {superior:.2f}] | medcouple={mc:.3f} | "
          f"outliers={n_outlier} ({n_outlier/len(taxa):.1%})")

# --- [43] ---
def bxp_stats_ajustado(serie, label):
    q1, q3 = serie.quantile([0.25, 0.75])
    mediana = serie.median()
    inferior, superior, mc = fences_boxplot_ajustado(serie)
    dentro = serie[(serie >= inferior) & (serie <= superior)]
    whislo = dentro.min() if len(dentro) else inferior
    whishi = dentro.max() if len(dentro) else superior
    fliers = serie[(serie < inferior) | (serie > superior)].to_numpy()
    return {"label": label, "med": mediana, "q1": q1, "q3": q3,
            "whislo": whislo, "whishi": whishi, "fliers": fliers}

stats_ajustado = [
    bxp_stats_ajustado(grupo_alto["taxa_icsap_uti_10k"], "ALTO investimento"),
    bxp_stats_ajustado(grupo_baixo["taxa_icsap_uti_10k"], "BAIXO investimento"),
]

fig, ax = plt.subplots(figsize=(6, 4.5))
ax.bxp(stats_ajustado, showfliers=True, patch_artist=True,
        boxprops=dict(facecolor="#4C72B0", alpha=0.5))
ax.set_ylabel("Taxa ICSAP-UTI por 10 mil hab.")
ax.set_title("Boxplot ajustado (medcouple) - outlier considerando a cauda longa")
plt.tight_layout()
plt.savefig(FIG_DIR / "fig03_boxplot_ajustado_medcouple.png", dpi=120, bbox_inches="tight")
plt.close()

# --- [44] ---
candidatas = ["despesa_paga_per_capita", "populacao", "leitos_sus_10k", "taxa_icsap_uti_10k"]
matriz_correlacao_candidatas = por_municipio[candidatas].corr(method="spearman")
print(matriz_correlacao_candidatas.round(3))

# --- [45] ---
regiao_slim = regiao_teste_t[["municipio_6", "regiao_intermediaria"]].drop_duplicates()
com_regiao = por_municipio.merge(regiao_slim, on="municipio_6", how="left")

tabela_regiao_investimento = (
    com_regiao.groupby("regiao_intermediaria")
    .agg(
        n_municipios=("municipio_6", "count"),
        mediana_investimento_per_capita=("despesa_paga_per_capita", "median"),
        mediana_taxa_icsap_10k=("taxa_icsap_uti_10k", "median"),
    )
    .sort_values("mediana_taxa_icsap_10k")
)
print(tabela_regiao_investimento.round(2))

# --- [46] ---
base_decomposicao = painel.dropna(subset=["siops_despesa_paga", "qtd_internacoes_icsap_uti",
    "populacao"]).copy()
base_decomposicao["despesa_paga_per_capita"] = base_decomposicao["siops_despesa_paga"] /
    base_decomposicao["populacao"]
base_decomposicao["taxa_icsap_uti_10k"] = base_decomposicao["qtd_internacoes_icsap_uti"] /
    base_decomposicao["populacao"] * 10000

```

```

medias_municipio = base_decomposicao.groupby("municipio_6")[["despesa_paga_per_capita",
"taxa_icsap_uti_10k"]].mean()
corr_entre = medias_municipio["despesa_paga_per_capita"].corr(medias_municipio["taxa_icsap_uti_10k"],
method="spearman")

medias_por_linha = base_decomposicao.groupby("municipio_6")[["despesa_paga_per_capita",
"taxa_icsap_uti_10k"]].transform("mean")
centrado_dentro = base_decomposicao[["despesa_paga_per_capita", "taxa_icsap_uti_10k"]] - medias_por_linha
corr_dentro = centrado_dentro["despesa_paga_per_capita"].corr(centrado_dentro["taxa_icsap_uti_10k"],
method="spearman")

print(f"N (municípios, médias 2023+2024): {len(medias_municipio)}")
print(f"Correlação ENTRE municípios (investimento e taxa per capita, médias por município):
{corr_entre:.3f}")
print(f"Correlação DENTRO do município (médias subtraídas): {corr_dentro:.3f}")

# =====
# 10. Teste de hipotese
# =====

# --- [47] ---
taxa_alto = grupo_alto["taxa_icsap_uti_10k"]
taxa_baixo = grupo_baixo["taxa_icsap_uti_10k"]

_, p_shapiro_alto = shapiro(taxa_alto)
_, p_shapiro_baixo = shapiro(taxa_baixo)

print(f"Shapiro-Wilk grupo alto investimento: p = {p_shapiro_alto:.2e}")
print(f"Shapiro-Wilk grupo baixo investimento: p = {p_shapiro_baixo:.2e}")

# --- [48] ---
estatistica_u, p_principal = mannwhitneyu(taxa_alto, taxa_baixo, alternative="less")

print("Teste principal (pré-registrado): H1 = grupo alto tem taxa MENOR que grupo baixo")
print(f"estatística U de Mann-Whitney = {estatistica_u:.1f}")
print(f"p-valor = {p_principal:.4f}")

# --- [49] ---
_, p_exploratorio = mannwhitneyu(taxa_alto, taxa_baixo, alternative="greater")

print("[EXPLORATÓRIO, pós-hoc, não é o teste principal]")
print("H1 exploratória: grupo alto tem taxa MAIOR que grupo baixo")
print(f"p-valor = {p_exploratorio:.4f}")

# --- [50] ---
def testar_corte(alto, baixo):
    _, p_principal = mannwhitneyu(alto, baixo, alternative="less")
    _, p_exploratorio = mannwhitneyu(alto, baixo, alternative="greater")
    return {
        "N cada grupo": f"{len(alto)}/{len(baixo)}",
        "mediana taxa baixo": round(baixo.median(), 2),
        "mediana taxa alto": round(alto.median(), 2),
        "p principal (alto < baixo)": round(p_principal, 4),
        "p exploratório (alto > baixo)": round(p_exploratorio, 4),
    }

investimento = por_municipio["despesa_paga_per_capita"]
taxa = por_municipio["taxa_icsap_uti_10k"]

resultados_sensibilidade = {
    "Mediana (PRINCIPAL, pré-registrado)": testar_corte(taxa_alto, taxa_baixo),
}

t1, t2 = investimento.quantile([1 / 3, 2 / 3])
resultados_sensibilidade["Tercil (sensibilidade)"] = testar_corte(
    taxa[investimento >= t2], taxa[investimento <= t1]
)

q1, q3 = investimento.quantile([0.25, 0.75])
resultados_sensibilidade["Quartil/extremos (sensibilidade)"] = testar_corte(
    taxa[investimento >= q3], taxa[investimento <= q1]
)

print(pd.DataFrame(resultados_sensibilidade).T)

```

```

# --- [51] ---
# Pressuposto que separa o t de Student do t de Welch: as duas variancias sao parecidas?
estatistica_levene, p_levene = levene(taxa_alto, taxa_baixo)
print("Teste de Levene (as variancias dos dois grupos sao iguais?)")
print(f" estatistica = {estatistica_levene:.4f} | p-valor = {p_levene:.4f}")
print(f" desvio-padrao ALTO investimento = {taxa_alto.std():.2f}")
print(f" desvio-padrao BAIXO investimento = {taxa_baixo.std():.2f}")
print()

# t de Welch: mesma H1 pre-registrada do teste principal, mais a exploratoria
t_welch_less = ttest_ind(taxa_alto, taxa_baixo, equal_var=False, alternative="less")
t_welch_greater = ttest_ind(taxa_alto, taxa_baixo, equal_var=False, alternative="greater")

print("Teste t de Welch (nao assume variancia igual)")
print(f" t-statistic = {t_welch_less.statistic:.4f} | graus de liberdade = {t_welch_less.df:.1f}")
print(f" p-valor H1 PRE-REGISTRADA (alto < baixo) = {t_welch_less.pvalue:.4f}")
print(f" p-valor H1 EXPLORATORIA (alto > baixo) = {t_welch_greater.pvalue:.4f}")
print()

# Lado a lado com o Mann-Whitney, que continua sendo o teste principal
_, p_mw_less = mannwhitneyu(taxa_alto, taxa_baixo, alternative="less")
_, p_mw_greater = mannwhitneyu(taxa_alto, taxa_baixo, alternative="greater")
print("Comparacao dos dois testes, mesma hipotese:")
print(f" {'':26s} {'H1 pre-registrada':>18s} {'H1 exploratoria':>17s}")
print(f" {'Mann-Whitney (principal)':26s} {p_mw_less:18.4f} {p_mw_greater:17.4f}")
print(f" {'t de Welch (checagem)':26s} {t_welch_less.pvalue:18.4f} {t_welch_greater.pvalue:17.4f}")
print()

# Tamanho de efeito em desvios-padrao, para comparar com o r = 0,13 do Mann-Whitney
n_alto, n_baixo = len(taxa_alto), len(taxa_baixo)
desvio_combinado = np.sqrt(
    ((n_alto - 1) * taxa_alto.var() + (n_baixo - 1) * taxa_baixo.var()) / (n_alto + n_baixo - 2)
)
d_cohen = (taxa_alto.mean() - taxa_baixo.mean()) / desvio_combinado
print(f"Media ALTO = {taxa_alto.mean():.2f} | Media BAIXO = {taxa_baixo.mean():.2f} internacoes/10 mil hab")
print(f"d de Cohen = {d_cohen:.3f}")

# =====
# 11. Insights e indicadores
# =====

# --- [52] ---
# So anos com SIOPS valido (2022 fica de fora, ver etapa 9.3), mesma janela
# ja usada na Preparacao Estatistica
painel_slim = painel_desp.dropna(subset=["siops_despesa_paga"]).copy()
painel_slim["despesa_paga_per_capita"] = painel_slim["siops_despesa_paga"] / painel_slim["populacao"]
painel_slim = painel_slim[["municipio_6", "ano", "despesa_paga_per_capita"]]

df_paciente = perfil.merge(painel_slim, on=["municipio_6", "ano", "how="inner"])
df_paciente = df_paciente.merge(rotas[["municipio_6", "tempo_viagem_minutos"]], on="municipio_6",
how="inner")
df_paciente = df_paciente.dropna(subset=["reinternacao_30d"])
df_paciente["tratado"] = df_paciente["tempo_viagem_minutos"] > 60

print(f"N total: {len(df_paciente):,}")
print(f"Tratado (tempo de viagem > 60min): {df_paciente['tratado'].sum():,}")
print(f"({df_paciente['tratado'].mean()*100:.1f}%)")
print()
resumo_paciente = df_paciente.groupby("tratado").agg(
    reinternacao_30d=("reinternacao_30d", "mean"),
    uti_mes_to=("uti_mes_to", "mean"),
    dias_perm=("dias_perm", "mean"),
    val_tot=("val_tot", "mean"),
)
print(resumo_paciente.round(4))

# --- [53] ---
painel_23_24 = painel[painel["ano"].isin([2023, 2024])].copy()
painel_23_24["taxa_icsap_uti_10k"] = painel_23_24["qtd_internacoes_icsap_uti"] / painel_23_24["populacao"]
* 10000

por_municipio_dist = painel_23_24.groupby("municipio_6")["taxa_icsap_uti_10k"].mean().reset_index()
df_dist = por_municipio_dist.merge(
    rotas[["municipio_6", "distancia_polo_km"]], on="municipio_6", how="inner"
)

```

```

).dropna()

rho, p_rho = spearmanr(df_dist["distancia_polo_km"], df_dist["taxa_icsap_uti_10k"])
print(f"N={len(df_dist)} municípios | Spearman rho={rho:.4f}, p={p_rho:.4f}")

df_dist["faixa_distancia"] = pd.qcut(df_dist["distancia_polo_km"], 5)
print("\nMediana de taxa_icsap_uti_10k por faixa de distância ao polo:")
print(df_dist.groupby("faixa_distancia", observed=True)["taxa_icsap_uti_10k"].median().round(2))

# --- [54] ---
# 2022 não tem SIOPS (sem defasagem pra explicar aquele ano, ver etapa 9.3) -> só 2023+2024
painel_23_24 = painel[painel["ano"].isin([2023, 2024]).copy()]
painel_23_24["despesa_paga_per_capita"] = painel_23_24["siops_despesa_paga"] / painel_23_24["populacao"]
painel_23_24["taxa_icsap_uti_10k"] = painel_23_24["qtd_internacoes_icsap_uti"] / painel_23_24["populacao"]
* 10000

por_municipio = painel_23_24.groupby("municipio_6").agg(
    despesa_paga_per_capita=("despesa_paga_per_capita", "mean"),
    taxa_icsap_uti_10k=("taxa_icsap_uti_10k", "mean"),
    populacao=("populacao", "mean"),
).reset_index()

df_psm = por_municipio.merge(rotas[["municipio_6", "tempo_viagem_minutos"]], on="municipio_6",
how="inner")
df_psm = df_psm.dropna(subset=["despesa_paga_per_capita", "taxa_icsap_uti_10k", "populacao",
"tempo_viagem_minutos"])
df_psm["log_pop"] = np.log(df_psm["populacao"])
df_psm["tratado"] = (df_psm["tempo_viagem_minutos"] > 60).astype(int)

print(f"N municípios: {len(df_psm)} | Tratados (>60min): {df_psm['tratado'].sum()} | "
f"Controles: {(df_psm['tratado']==0).sum()}")
print("\nMédias brutas (sem pareamento) de taxa_icsap_uti_10k:")
print(df_psm.groupby("tratado")["taxa_icsap_uti_10k"].mean().round(3))

X = df_psm[["log_pop", "despesa_paga_per_capita"]].values
y = df_psm["tratado"].values
logit = LogisticRegression(max_iter=1000)
logit.fit(X, y)
df_psm["ps"] = logit.predict_proba(X)[:, 1]
df_psm["logit_ps"] = np.log(df_psm["ps"] / (1 - df_psm["ps"]))

caliper = 0.2 * df_psm["logit_ps"].std()
print(f"\nCaliper (0,2 x desvio-padrão do logit do escore): {caliper:.3f}")

tratados = df_psm[df_psm["tratado"] == 1].reset_index(drop=True)
controles = df_psm[df_psm["tratado"] == 0].reset_index(drop=True)

# Pareamento 1:1 vizinho mais próximo em logit(escore), COM reposicao
# (controle pode servir de par pra mais de um tratado)
arvore = cKDTree(controles[["logit_ps"]].values)
distancias, indices = arvore.query(tratados[["logit_ps"]].values, k=1)
dentro_caliper = distancias <= caliper
print(f"Tratados pareados dentro do caliper: {dentro_caliper.sum()} de {len(tratados)}")

trat_pareado = tratados[dentro_caliper].reset_index(drop=True)
ctrl_pareado = controles[indices[dentro_caliper]].reset_index(drop=True)

def smd(a, b):
    return (a.mean() - b.mean()) / np.sqrt((a.var() + b.var()) / 2)

print(f"\nSMD log_pop: antes={smd(tratados['log_pop'], controles['log_pop']):.3f} -> "
f"depois={smd(trat_pareado['log_pop'], ctrl_pareado['log_pop']):.3f}")
print(f"SMD despesa_paga_per_capita: antes={smd(tratados['despesa_paga_per_capita'],
controles['despesa_paga_per_capita']):.3f} -> "
f"depois={smd(trat_pareado['despesa_paga_per_capita'],
ctrl_pareado['despesa_paga_per_capita']):.3f}")

att = (trat_pareado["taxa_icsap_uti_10k"] - ctrl_pareado["taxa_icsap_uti_10k"]).mean()
print(f"\nATT (tratado pareado - controle pareado): {att:.3f}")
print(f"Média tratado pareado: {trat_pareado['taxa_icsap_uti_10k'].mean():.3f} | "
f"Média controle pareado: {ctrl_pareado['taxa_icsap_uti_10k'].mean():.3f}")

t_stat, p_t = ttest_rel(trat_pareado["taxa_icsap_uti_10k"], ctrl_pareado["taxa_icsap_uti_10k"])
w_stat, p_w = wilcoxon(trat_pareado["taxa_icsap_uti_10k"], ctrl_pareado["taxa_icsap_uti_10k"])
print(f"\nTeste t pareado: t={t_stat:.3f}, p={p_t:.4f}")
print(f"Wilcoxon (signed-rank): W={w_stat:.3f}, p={p_w:.4f}")

```

```

# --- [55] ---
# Comparacao: pareamento COM reposicao (o que usamos) vs SEM reposicao
usados = set()
pares_trat, pares_ctrl = [], []
for _, row in tratados.iterrows():
    dif = (controles["logit_ps"] - row["logit_ps"]).abs()
    dif.loc[list(usados)] = np.inf
    j = dif.idxmin()
    if dif[j] <= caliper:
        pares_trat.append(row["taxa_icsap_uti_10k"])
        pares_ctrl.append(controles.loc[j, "taxa_icsap_uti_10k"])
        usados.add(j)
att_sem = (np.array(pares_trat) - np.array(pares_ctrl)).mean()

print(f"COM reposicao (usado no resultado principal): ATT={att:+.3f} | pares: {dentro_caliper.sum()} de {len(tratados)}")
print(f"SEM reposicao: ATT={att_sem:+.3f} | pares: {len(pares_trat)} de {len(tratados)}")

# --- [56] ---
painel_slim = painel_desp.dropna(subset=["siops_despesa_paga"]).copy()
painel_slim["despesa_paga_per_capita"] = painel_slim["siops_despesa_paga"] / painel_slim["populacao"]
painel_slim = painel_slim[["municipio_6", "ano", "despesa_paga_per_capita"]]

df_ml = perfil.merge(painel_slim, on=["municipio_6", "ano"], how="inner")
df_ml = df_ml.merge(rotas[["municipio_6", "tempo_viagem_minutos"]], on="municipio_6", how="inner")
df_ml["grupo_csap_nome"] = df_ml["grupo_csap_nome"].fillna("Nao_ICSAP")

features_cat = ["idade_faixa", "sexo", "grupo_csap_nome", "dia_semana", "feriado"]
features_num = ["val_tot", "dias_perm", "despesa_paga_per_capita", "tempo_viagem_minutos"]
target = "reinternacao_30d"
df_ml = df_ml.dropna(subset=[target])

def treinar_avaliar(df_treino, df_teste):
    X_tr = pd.get_dummies(df_treino[features_cat + features_num], columns=features_cat, drop_first=True)
    X_te = pd.get_dummies(df_teste[features_cat + features_num], columns=features_cat, drop_first=True)
    X_tr, X_te = X_tr.align(X_te, join="outer", axis=1, fill_value=0)
    y_tr = df_treino[target].astype(int)
    y_te = df_teste[target].astype(int)
    peso = (len(y_tr) - y_tr.sum()) / y_tr.sum()
    modelo = xgb.XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1,
                               scale_pos_weight=peso, random_state=42, n_jobs=-1)
    modelo.fit(X_tr, y_tr)
    proba = modelo.predict_proba(X_te)[:, 1]
    return roc_auc_score(y_te, proba), len(y_tr), len(y_te)

mask_treino = df_ml["ano"] == 2023
mask_teste = df_ml["ano"] == 2024

auc_full, n_tr, n_te = treinar_avaliar(df_ml[mask_treino], df_ml[mask_teste])
print(f"ROC-AUC (treino e teste na população inteira): {auc_full:.3f} (N_treino={n_tr:}, N_teste={n_te:})")

# Boa parte da base nunca pode reinternar por definicao (so 1 internacao no
# periodo todo) -> retreina e reavalia so na populacao onde reinternar e
# fisicamente possivel
df_sub = df_ml[df_ml["qtd_internacoes_paciente_periodo"] > 1]
auc_sub, n_tr_sub, n_te_sub = treinar_avaliar(df_sub[df_sub["ano"] == 2023], df_sub[df_sub["ano"] == 2024])
print(f"ROC-AUC (treino e teste só na população com >1 internação no período): "
      f"{auc_sub:.3f} (N_treino={n_tr_sub:}, N_teste={n_te_sub:})")

# Estatistica de colisao de chave calculada no perfil ORIGINAL (nao no
# recorte 2023/2024 que entra no modelo), pra nao subestimar por causa do
# filtro de ano/municipio feito acima
print(f"\nchave_colisao_suspeita (base completa, todos os anos): "
      f"{perfil['chave_colisao_suspeita'].sum():} de {len(perfil):}, "
      f"{(perfil['chave_colisao_suspeita'].mean()*100:.2f)%}")

# --- [57] ---
fora = dataset_unificado[dataset_unificado["reside_fora_sp"]]
sp = dataset_unificado[~dataset_unificado["reside_fora_sp"]]

n_fora, n_sp = fora["qtd_internacoes_uti"].sum(), sp["qtd_internacoes_uti"].sum()
val_fora, val_sp = fora["soma_val_tot"].sum(), sp["soma_val_tot"].sum()
val_total = val_fora + val_sp

```

```

print(f"Internações de fora de SP: {n_fora:,.2f} ({n_fora/(n_fora+n_sp)*100:.1f}% do total)")
print(f"Gasto com pacientes de fora de SP: R$ {val_fora:,.2f} ({val_fora/val_total*100:.2f}% do gasto total)")
print(f"Custo médio por internação (fora de SP): R$ {val_fora/n_fora:,.2f}")
print(f"Custo médio por internação (SP): R$ {val_sp/n_sp:,.2f}")
print(f"Razão de custo (fora/SP): {(val_fora/n_fora)/(val_sp/n_sp):.2f}x")
print()
fora_icsap = fora[fora["csap"] == "sim"]["qtd_internacoes_uti"].sum()
sp_icsap = sp[sp["csap"] == "sim"]["qtd_internacoes_uti"].sum()
print(f"Dos casos de fora de SP, % ICSAP (evitável): {fora_icsap/n_fora*100:.1f}%")
print(f"Dos casos de SP, % ICSAP (evitável): {sp_icsap/n_sp*100:.1f}%")

# --- [58] ---
painel_top10 = painel[painel["ano"].isin([2023, 2024])].copy()
painel_top10["taxa_icsap_uti_10k"] = (
    painel_top10["qtd_internacoes_icsap_uti"] / painel_top10["populacao"] * 10000
)

por_municipio_top10 = painel_top10.groupby(["municipio_6", "nome_municipio"]).agg(
    taxa_icsap_uti_10k=("taxa_icsap_uti_10k", "mean"),
    populacao=("populacao", "mean"),
    qtd_internacoes_icsap_uti=("qtd_internacoes_icsap_uti", "sum"),
).reset_index()

PISO_VOLUME = 30 # minimo de internacoes ICSAP-UTI no periodo, pra taxa nao ser so ruido
elegiveis = por_municipio_top10[por_municipio_top10["qtd_internacoes_icsap_uti"] >= PISO_VOLUME]
top10 = elegiveis.sort_values("taxa_icsap_uti_10k", ascending=False).head(10)

print(f"Municípios elegíveis (>= {PISO_VOLUME} internações ICSAP-UTI no período): {len(elegiveis)} de {len(por_municipio_top10)}")
print(top10[["nome_municipio", "taxa_icsap_uti_10k", "populacao", "qtd_internacoes_icsap_uti"]].to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 5))
ax.barh(top10["nome_municipio"][:10], top10["taxa_icsap_uti_10k"][:10], color="#4C72B0")
ax.set_xlabel("Taxa ICSAP-UTI por 10 mil hab. (média 2023-2024)")
ax.set_title(f"Top 10 municípios com maior internação evitável\n(mínimo {PISO_VOLUME} internações no período)")
plt.tight_layout()
plt.savefig(FIG_DIR / "fig04_top10_municipios.png", dpi=120, bbox_inches="tight")
plt.close()

# =====
# 12. O Machine Learning explicado (escada de modelos)
# =====

# --- [59] ---
# Mesma matriz de features, mesmo split temporal e mesmo tratamento de desbalanceamento
# da secao 11.4. A UNICA coisa que muda entre as linhas da tabela e o algoritmo.
X_treino = pd.get_dummies(df_ml[mask_treino][features_cat + features_num],
    columns=features_cat, drop_first=True)
X_teste = pd.get_dummies(df_ml[mask_teste][features_cat + features_num],
    columns=features_cat, drop_first=True)
X_treino, X_teste = X_treino.align(X_teste, join="outer", axis=1, fill_value=0)
y_treino = df_ml[mask_treino][target].astype(int)
y_teste = df_ml[mask_teste][target].astype(int)
peso_classe = (len(y_treino) - y_treino.sum()) / y_treino.sum()

print(f"{X_treino.shape[1]} colunas depois do one-hot | treino (2023) = {len(y_treino):,} | teste (2024) = {len(y_teste):,}")
print(f"Prevalência de reinternacao em 30 dias no teste: {y_teste.mean():.2%}")
print(f"scale_pos_weight (razao negativo/positivo no treino): {peso_classe:.2f}")
print()

def avaliar(nome, modelo, escalona=False):
    inicio = time.time()
    if escalona:
        escala = StandardScaler().fit(X_treino.astype(float))
        modelo.fit(escala.transform(X_treino.astype(float)), y_treino)
        proba = modelo.predict_proba(escala.transform(X_teste.astype(float)))[:, 1]
    else:
        modelo.fit(X_treino, y_treino)
        proba = modelo.predict_proba(X_teste)[:, 1]
    segundos = time.time() - inicio
    auc = roc_auc_score(y_teste, proba)
    print(f"{nome:42s} ROC-AUC = {auc:.3f} (treino: {segundos:.1f}s)")
    return modelo, proba, auc

```

```

print("Escada de modelos, do mais simples ao escolhido:")
avaliar("Chute proporcional (piso teorico)",
        DummyClassifier(strategy="stratified", random_state=42))
avaliar("Regressao logistica (baseline linear)",
        LogisticRegression(max_iter=1000, class_weight="balanced"), escalona=True)
avaliar("Arvore de decisao unica (profundidade 6)",
        DecisionTreeClassifier(max_depth=6, class_weight="balanced", random_state=42))
modelo_xgb, proba_xgb, auc_xgb = avaliar(
    "XGBoost (o escolhido, secao 11.4)",
    xgb.XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1,
                     scale_pos_weight=peso_classe, random_state=42, n_jobs=-1))

# =====
# 13. Metricas e interpretacao do modelo
# =====

# --- [60] ---
def painel_de_metricas(y_verdadeiro, probabilidade, nome, corte=0.5):
    predito = (probabilidade >= corte).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_verdadeiro, predito).ravel()
    prevalencia = y_verdadeiro.mean()
    precisao = precision_score(y_verdadeiro, predito)
    print(f"--- {nome}")
    print(f"  N = {len(y_verdadeiro):,} | reinternacoes reais = {int(y_verdadeiro.sum()):,} "
          f"| prevalencia = {prevalencia:.2%}")
    print(f"  ROC-AUC {roc_auc_score(y_verdadeiro, probabilidade):.3f}")
    print(f"  Preciso {precisao:.3f} (de cada 100 sinalizados, {precisao*100:.0f} reinternam de fato)")
    print(f"  Recall {recall_score(y_verdadeiro, predito):.3f} "
          f"  (pega {recall_score(y_verdadeiro, predito)*100:.0f}% das reinternacoes que existiram)")
    print(f"  F1 {f1_score(y_verdadeiro, predito):.3f}")
    print(f"  Ganho sobre chutar na prevalencia: {precisao/prevalencia:.2f}x")
    print(f"  Sinalizados pelo modelo: {tp+fp:,} de {len(y_verdadeiro):,} ({(tp+fp)/len(y_verdadeiro):.1%} da base)")
    print(f"  Matriz de confusao: VN={tn:,} | FP={fp:,} | FN={fn:,} | VP={tp:,}")
    print()

painel_de_metricas(y_teste, proba_xgb, "Populacao inteira (o numero de cabecalho da secao 11.4)")

# Mesmo modelo, mesma receita, so na populacao onde reinternar e fisicamente possivel
X_sub_treino = pd.get_dummies(df_sub[df_sub["ano"] == 2023][features_cat + features_num],
                             columns=features_cat, drop_first=True)
X_sub_teste = pd.get_dummies(df_sub[df_sub["ano"] == 2024][features_cat + features_num],
                             columns=features_cat, drop_first=True)
X_sub_treino, X_sub_teste = X_sub_treino.align(X_sub_teste, join="outer", axis=1, fill_value=0)
y_sub_treino = df_sub[df_sub["ano"] == 2023][target].astype(int)
y_sub_teste = df_sub[df_sub["ano"] == 2024][target].astype(int)
peso_sub = (len(y_sub_treino) - y_sub_treino.sum()) / y_sub_treino.sum()

modelo_sub = xgb.XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1,
                              scale_pos_weight=peso_sub, random_state=42, n_jobs=-1)
modelo_sub.fit(X_sub_treino, y_sub_treino)
proba_sub = modelo_sub.predict_proba(X_sub_teste)[: , 1]

painel_de_metricas(y_sub_teste, proba_sub, "So pacientes com mais de 1 internacao no periodo")

```

9. Limitações e reprodução

Do dado: o SIH-SUS é registro administrativo e não contém variável clínica (gravidade, comorbidade, exame), o que limita o modelo da etapa 11. O gap de cadastro entre CNES-LT e CNES-ST deixa a coluna de leitos nula em 10,7% do dataset. O SIOPS capturado é o total em Ações e Serviços Públicos de Saúde, sem a abertura por subfunção que separaria atenção primária do restante do gasto.

Do método: com apenas dois anos de SIOPS utilizável, o desenho é transversal na prática, mesmo com a defasagem construída no painel. O pareamento da etapa 11 corrobora a direção mas não fecha significância.

Reprodução: colocar os cinco arquivos de dados em uma pasta, apontar DATA_DIR para ela e executar. No notebook, Kernel > Restart & Run All. No script, `python clariti_aed_icsap_uti_sp.py`. A execução completa do script levou cerca de 3 minutos e gravou os quatro gráficos em figuras/.